SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

CSA1737 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 5 (CO5 – Unit V: Agents, Case Studies and Real-Time Projects)

Agents, Intelligent Systems and Real-Time AI: Architecture, Engineering and Deployment

Item	Details
Student	**Saai Vardhan D** – Register No. **192525205**
Nature	Individual assignment, 50 marks (Part A design 25, Part B implementation 25)
Course	CSA1737 – Artificial Intelligence (L-T-P-C: 3-0-2-4)
Applied case chosen (A5 and B3)	(a) AI-based intrusion detection on an edge device
Language / tools	Python 3 with NumPy, SciPy, scikit-learn (random-forest training and metrics only) and Matplotlib; the agent, swarm, expert-system shell, flat-array inference engine, mini-RDD engine and chatbot are written from scratch; fixed seeds and PYTHONHASHSEED=0
Date of submission	19 November 2026

Workflow followed: understand → design → architect → implement → measure → stress → compress → decide → defend. Part A is written without a compiler, with pseudocode in the SIMATS style; Part B implements what Part A designed and reports the evidence chain *source code → test input → expected behaviour → actual output → performance metrics (several seeds or sizes) → boundary / failure test → interpretation*. Wherever the assignment says "predict", the prediction is written before the run and the program prints it ahead of the result; several predictions were wrong and are reported as they are.

> **Two limits of this working environment that the reader should know**
>
> **1. No PySpark.** The sandbox in which the code was run could not install PySpark (the package index was unreachable). Task B3.3 therefore uses `minirdd.py`, a small RDD-style engine written for this assignment (partitions, lazy lineage, narrow versus wide dependencies, map-side combine, caching, lineage recovery, process pool). Its numbers are **not Spark numbers**. The equivalent PySpark program is given in Appendix C; it is unrun here and should be run on Colab before the viva.
>
> **2. No public dataset.** No intrusion dataset could be downloaded. Task B3 uses a flow **simulator** written by me with the feature names of the KDD Cup 99 / NSL-KDD schema (Tavallaee et al., 2009). The assignment allows "a public dataset or simulator", but results on this simulator are not results on NSL-KDD. `ids.py` is organised so that `gen_stream` can be replaced by a loader for NSL-KDD.

Contents and Summary of Findings

Part	Task	Title	Marks
A	A1	Design an agent architecture (parcel-delivery drone)	7
A	A2	Don't centralise what can be local (flood swarm)	5
A	A3	Discover the expert-system architecture pattern	5
A	A4	Design a distributed data pipeline with RDDs	4
A	A5	Design when the full model is too expensive (edge IDS)	4
B	B1	Implement and instrument the drone agent and the swarm	7
B	B2	Engineer an adversarial input for the expert system	5
B	B3	Applied case: full model versus edge model	8
B	B4	Conversational interface and deployment decision	5
		Total	**50**

Key findings (all measured by the programs in the appendices)

B1. The hybrid drone agent passes four scenarios (normal flight, sudden obstacle, goal blocked, battery too small) with 0 incidents; its program trace matches the hand trace of A1.5 step for step except for a path tie-break and a hovering detail. A wind-drift test found a real bug (a stale plan after a drift made the drone hover until the battery ran out); after the fix, 20% drift with a fixed reserve left 28/30 flights landing home and an adaptive return margin 30/30, at the cost of fewer deliveries (16/30 against 29/30 at battery 50). **Swarm:** with 30 seeds the success rate (coverage ≥ 70%) is 0.00 for 1 and 5 drones and 1.00 for 10, 25 and 50; redundancy rises from 0.04 (N = 1) to 0.86 (N = 50); with 30% of the drones failing N = 10 still covers 0.814. A central controller loses everything when it fails (coverage 0.208 against 0.944 for the swarm).

B2. The 16-rule pump expert system scores 1.00 on the 15 development cases and **0.867** on the 15 hold-out cases. Of 20 adversarial predictions 19 were right; the wrong one (recency on a case with a derived fact) is reported. Rule order changes the diagnosis under the "order" strategy in 4 cases. Run-time rule edits without touching the engine: one added rule fixed a hold-out case, a careless edit silently broke a correct diagnosis (accuracy 0.933 → 0.900), which the regression run exposed.

B3. On a time-ordered split of simulated flows the 200-tree forest reaches F1 0.968 (AUC 0.999) and the edge cascade F1 0.945 (AUC 0.950), with a **170× smaller model** (21.5 KiB against 3.6 MiB) and **152× fewer node visits** (18 against 2,666). The unseen attack type slow_dos is detected 36.6% of the time by the full model and 0.0% by the edge model; adding 200 labelled examples lifts the full model to 95.3% but **not** the cascade (still 0.0%). The mini-RDD engine shows a 2.3× speed-up at 1e7 records and no gain at 1e5 and 1e6; map-side combine moves 60× less shuffle data (0.26 MB against 15.7 MB); caching helps an expensive step (2.68 s → 1.30 s) and hurts a cheap one (0.28 s → 0.58 s).

B4. The retrieval chatbot (9 intents) first accepted 55% of out-of-scope questions; adding question words to the stop list cut this to 15% (and 8% on a fresh second test set) while accuracy rose from 0.757 to 0.784 (0.633 → 0.833 on the second set). Under simulated recognition errors accuracy falls from 0.757 to 0.505 at WER 0.3. **Decision:** run the detector and the chatbot on the edge device; reverse it if the model grows past the device memory or if the attack mix drifts faster than the device can be updated.

PART A – DESIGN AND REASONING (25 marks)

Task A1 – Design an Agent Architecture: Autonomous Parcel-Delivery Drone (7 marks)

A1.1 Model the agent problem

PEAS	Specification
Performance measure	Parcel delivered; zero collisions and zero no-fly violations (hard constraints); drone lands at home with the battery reserve intact; fewer moves and less battery as a secondary goal; delivery time.
Environment	City grid with a home base and a delivery cell; static no-fly zones (known map); static and sudden obstacles (not known in advance); wind that sometimes moves the drone to a different neighbouring cell.
Actuators	Move N, E, S or W by one cell; Hold; Drop the parcel; Land.
Sensors	Position estimate (GPS), battery gauge, proximity sensors for the four neighbouring cells, clock, the stored no-fly map.

Environment property	Value	Reason
Observability	Partially observable	obstacles are seen only in the four adjacent cells; the wind is not observed, only its effect on the position
Determinism	Stochastic	a move sometimes ends in another cell because of wind drift
Episodic?	Sequential	a move changes the battery and position that every later decision depends on
Static?	Dynamic	obstacles appear while the drone is flying
Discrete?	Discrete	grid cells, time steps and a finite action set
Agents	Single agent (A1); multi-agent in A2	other drones appear only in the swarm task

Agent types required. A **simple reflex** component is needed for safety (a sensed obstacle must stop a move immediately, whatever the plan says). A **model-based** component is needed because the percept is partial: the drone must remember which obstacles it has seen and estimate the wind. A **goal-based** component plans a route to the delivery cell and home. A **utility-based** component is needed to trade the competing aims (delivery against battery, speed against safety). A learning component is not required for the base problem (the wind rate is estimated online, a minimal form of adaptation). The result is a **hybrid (layered) architecture**: a reactive safety layer below a deliberative planning layer.

A1.2 Percepts, internal state and actions

Category	Content
Percepts (every step)	pos = (row, col) from GPS; battery (moves left); t (step number); sensed = the set of neighbouring cells that currently hold an obstacle.
Internal state remembered between steps	home, goal, no-fly set (given); known_obst (every obstacle ever sensed); mode ∈ {MISSION, RETURN, DONE}; path (the planned cells still to visit); delivered (flag); expected (the cell the last move was meant to reach); moves and drift_hits (to estimate the wind rate p_est = drift_hits / (moves + 5)); reserve (safety battery margin).
Actions	N, E, S, W (one cell, cost 1 battery), Hold (cost 1), Drop (cost 1), Land (cost 0).

A1.3 Decision rule and priorities

Conflicts are resolved by a **fixed priority order**: (1) **safety** (never enter a sensed obstacle, a no-fly cell or leave the grid), then (2) **battery reserve** (if the battery is no more than the estimated energy to get home plus the reserve, abort and return), then (3) **mission** (go to the goal and drop the parcel), then (4) **economy** (among equal plans prefer the shorter). Inside the mission layer the planner minimises a utility:

$$U = 100 \cdot delivered - 1 \cdot moves - 1000 \cdot violations - 500 \cdot forced_landing$$

The weights make a violation or a forced landing far worse than any saving of moves, so the order above is respected also in the utility; the planner itself is a shortest-path search (A*), which maximises U among safe plans. The return rule uses the estimated cost of going home: need = d_home · (1 + 2p̂)/(1 − p̂) (p̂ is the drift estimate; with no wind p̂ = 0 and need = d_home), plus the reserve; the factor (1 + 2p)/(1 − p) is the expected number of moves per cell when each move drifts with probability p and every drift costs two extra moves to undo.

A1.4 Pseudocode of the sense–decide–act cycle

```
ALGORITHM DroneControlCycle(HOME, GOAL, NOFLY, RESERVE)
STATE : known_obst <- empty set ; mode <- MISSION ; path <- empty list ; delivered <- FALSE ; expected <- NONE
; moves <- 0 ; drift_hits <- 0
BEGIN
  1   WHILE the mission has not terminated DO                                  // termination: mode = DONE,
battery = 0 or time limit
  2       (pos, battery, sensed) <- SENSE()                                     // ---- sense
  3       IF expected is not NONE THEN                                          // ---- state update
  4           moves <- moves + 1 ; IF pos != expected THEN drift_hits <- drift_hits + 1
  5       expected <- NONE ; known_obst <- known_obst UNION sensed
  6       IF mode = DONE THEN ACT(Hold) ; CONTINUE
  7       // ---------- LAYER 0: reactive safety and battery (highest priority) ----------
  8       IF mode = RETURN AND pos = HOME THEN mode <- DONE ; ACT(Land) ; CONTINUE
  9       path_home <- ASTAR(pos, HOME, NOFLY UNION known_obst) ; d_home <- LENGTH(path_home)
 10       p_est <- drift_hits / (moves + 5)
 11       need <- d_home * (1 + 2 * p_est) / (1 - p_est)
 12       IF mode = MISSION AND battery <= need + RESERVE THEN mode <- RETURN ; path <- empty list     //
battery overrides mission
 13       IF mode = RETURN AND pos = HOME THEN mode <- DONE ; ACT(Land) ; CONTINUE
 14       IF mode = MISSION AND pos = GOAL THEN delivered <- TRUE ; mode <- RETURN ; path <- empty ; ACT(Drop)
; CONTINUE
 15       IF path is not empty AND first cell of path is in (NOFLY UNION known_obst) THEN path <- empty list
// veto: never step into an obstacle
 16       IF path is not empty AND DISTANCE(pos, first cell of path) != 1 THEN path <- empty list
// wind moved the drone off the plan
 17       // ---------- LAYER 1: deliberative planning ----------
 18       IF path is empty THEN
 19           target <- GOAL IF mode = MISSION ELSE HOME
 20           path <- ASTAR(pos, target, NOFLY UNION known_obst)
 21           IF path = NO PATH THEN
 22               IF mode = MISSION THEN mode <- RETURN ; path <- ASTAR(pos, HOME, NOFLY UNION known_obst)
// goal unreachable: abort safely
 23               IF path = NO PATH THEN ACT(Hold) ; CONTINUE
 24       next <- REMOVE first cell of path ; expected <- next                  // ---- arbitration done: L0
had priority on lines 8-16
 25       ACT( the move from pos to next )                                      // ---- act
 26   END WHILE
END
```

Arbitration. Layer 0 acts first on every cycle; layer 1 is reached only if layer 0 did not return an action (lines 8, 13, 14) and may be overridden by the veto (15) and the off-plan test (16). Mode changes caused by the battery rule persist, which is how safety and battery outrank the mission. **Boundary conditions covered:** no path to the goal (line 22), no path home (line 23), the drone back at home while still in MISSION mode (line 13), drift off the plan (line 16).

A1.5 Hand trace (obstacle appears, then the battery falls below the threshold)

Scenario: 8 × 8 grid, HOME = (0,0), GOAL = (0,6), battery 14, RESERVE = 3, no wind ($\hat{p} = 0$ so need = d_home), no no-fly zones. An obstacle appears at (0,4) at the end of step 3. A* tie-break order N, E, S, W.

t	Percept: pos / battery / sensed	Internal state before acting	Test that decides	Layer	Action
0	(0,0) / 14 / –	MISSION; path empty	battery 14 > 0 + 3; plan east: 6 cells	L1	E
1	(0,1) / 13 / –	MISSION; 4 cells left	13 > 1 + 3	L1	E
2	(0,2) / 12 / –	MISSION; 3 cells left	12 > 2 + 3	L1	E
3	(0,3) / 11 / {(0,4)}	MISSION; path (0,4),(0,5),(0,6); known_obst = {(0,4)}	11 > 3 + 3; next cell (0,4) is a known obstacle: **veto**, replan: detour S,E,E,N,E (5 moves)	**L0** then L1	S
4	(1,3) / 10 / –	MISSION; detour 4 cells left	10 > 4 + 3 = 7	L1	E
5	(1,4) / 9 / {(0,4)}	MISSION; 3 cells left	9 > 5 + 3 = 8	L1	E
6	(1,5) / 8 / –	MISSION → **RETURN**; path cleared	8 ≤ 6 + 3 = 9: **battery rule**; plan home, 6 cells	**L0**	W
7	(1,4) / 7 / {(0,4)}	RETURN; 5 cells left	no further test; follow plan	L1	W
8	(1,3) / 6 / –	RETURN; 4 cells left	follow plan	L1	next cell of the path (a shortest path home)
12	(0,0) / 2 / –	RETURN; at home	pos = HOME	L0	Land (battery 2 left)

The trace shows the three required events: the sudden obstacle is handled by the reactive veto and a replan (t = 3), the battery rule overrides the mission (t = 6) and the drone lands with 2 units of battery. Part B (B1.2) compares this hand trace with the program's trace.

Task A2 – Don't Centralise What Can Be Local: Flood Search-and-Rescue with 50 Drones (5 marks)

A2.1 Diagnose the weakness of the centralised design

Failure	Effect on coverage	Effect on communication load	Effect on mission success
Controller fails	Every drone stops receiving cells: coverage freezes at the value reached (simulated: 0.208 against 0.944)	load collapses to zero, but so does all work	The mission fails (success 0.00 in the simulation)
Links are lossy (30% of commands lost)	Drones idle on every lost command; coverage drops (0.811 against 0.897 with perfect links)	Retransmissions needed; the load of 50 drones × 1 command per step must reach one point	Success falls from 0.97 to 0.83
Drones fail	The controller keeps assigning cells to a drone that is dead unless it detects the failure	Wasted commands to dead drones	Lower coverage (0.755 with 30% failures)

The central design has a **single point of failure**, a communication load that grows with the number of drones, and a latency between sensing and command, which matters in a disaster where links are the first thing to break.

A2.2 Decentralised redesign: pheromone marks and frontier selection

Idea. Each drone marks the cells it visits (a digital pheromone in the shared environment, a count per cell) and always moves toward the least-marked neighbour, so drones are repelled from visited cells and spread

out without being told where to go. When all neighbours are marked the drone looks inside a sensing radius RS = 4 for the nearest unmarked cell (frontier) and walks toward it.

> **Information an agent needs**
>
> **From its own sensors:** its position and battery. **From the environment (stigmergy):** the pheromone mark of every cell within radius RS = 4 (read locally, written when visiting). **From neighbours:** only whether a drone currently occupies each of the four adjacent cells (separation). **Not needed:** any identity, any plan or any message from other drones or from a controller.

```
ALGORITHM SwarmStep(pos, mark, occupied, FREE, RS)
INPUT : pos - my cell; mark[c] - number of visits to cell c (0 = never visited), readable within radius
RS;
        occupied - set of cells holding another drone (my four neighbours); FREE(c) - c is inside the
grid and not a no-fly cell
OUTPUT: the cell to move to (or pos to hover); the cell is marked by the caller
BEGIN
  1   candidates <- the cells c in {up, right, down, left of pos} (in this fixed order) with FREE(c) AND
c not in occupied
  2   IF candidates is empty THEN RETURN pos                                    // blocked: hover
(separation)
  3   best <- the smallest mark[c] over candidates
  4   IF best = 0 THEN RETURN the first (or a random) candidate with mark 0     // unvisited neighbour:
take it
  5   // all neighbours visited: frontier search with breadth-first search limited to radius RS
  6   frontier <- the nearest cell f within distance RS of pos with mark[f] = 0, reached through FREE
cells
  7   IF frontier exists THEN
  8       first <- the first step of the shortest path from pos to frontier
  9       IF first is in candidates THEN RETURN first
 10   RETURN the first (or a random) candidate whose mark equals best              // follow the
pheromone gradient
END
```

Properties. Cost per decision is constant (4 neighbours and a bounded search); the rule uses no global information; the failure of any drone only removes that drone's work, the others keep covering. A drone must also carry a battery check (return before it is empty), omitted here for clarity.

A2.3 Emergent behaviour: 5 × 5 grid, three drones (hand trace)

Drones A (0,0), B (0,4), C (4,2); all start cells have mark 1; order of moving A, B, C within a step; marks are updated immediately; ties are broken in the order up, right, down, left; coordinates (row, col).

Step	A moves	B moves	C moves	Reasoning (first lowest mark in the order U, R, D, L)	Visited cells
1	(0,0) → (0,1)	(0,4) → (1,4)	(4,2) → (3,2)	A: up is outside, right and down both have mark 0, right comes first. B: up and right unavailable, down comes before left. C: up (3,2) is first with mark 0.	6 of 25
2	→ (0,2)	→ (2,4)	→ (2,2)	A: right (0,2) is unvisited. B: up (1,4)… mark 1 (its own trail), down (2,4) mark 0. C: up (2,2).	9
3	→ (0,3)	→ (3,4)	→ (1,2)	same logic: every drone keeps moving to a neighbour with mark 0	12
4	→ (1,3)	→ (4,4)	→ (1,1)	A: right (0,4) has mark 1 (B's start), so down (1,3) with mark 0. C: up (0,2) mark 1, right (1,3) is now marked by A, down (2,2) mark 1, left (1,1) mark 0.	15 = 60%

After four steps 12 new cells have been covered by 12 moves: no cell was visited twice (redundancy 0) although no drone was assigned anything. The drones separated naturally, one working down the right edge, one along the top, one up the middle. (Steps 5 and 6 are checked by the program in B1.3: 18 and 20 cells.)

A2.4 Challenge the rule: clustering and redundant coverage

Configuration. 5 × 5 grid; every cell has mark 1 except (4,0); drones B at (0,4) and C at (0,3); RS = 8 (the frontier is visible); order B then C; ties U, R, D, L.

Prediction before tracing. Both drones see the same single unmarked cell (4,0) as their frontier. Neither knows the other has chosen it, because the rule has no claim mechanism; both walk toward it side by side (I wrote down 7 and 6 steps; the true Manhattan distances are 8 and 7, an off-by-one in my arithmetic) and the first to arrive covers the single new cell, so almost every move is redundant: I predict about 13 of 13 moves redundant.

Trace. Step 1: B: up and right unavailable, left (0,3) is occupied by C, so the only candidate is (1,4) (mark 1, all candidates marked): frontier search finds (4,0) and the first step on the path is down or left; left is not a candidate, so B goes to (1,4). C: candidates right (0,4) (free now), down (1,3), left (0,2): the first step toward (4,0) is down, so C goes to (1,3). Steps 2–4: both drones move down together. The program trace in B1.3 shows that the drones do arrive in tandem, and in addition B oscillates between (3,4) and (4,4) because C is blocking its path (14 moves in total, redundancy 0.929). **Repair (stated, not implemented):** let a drone write a *claim* mark on its chosen frontier cell, read by others within RS, so that the second drone chooses a different frontier or stays out.

Task A3 – Discover the Expert-System Architecture Pattern (5 marks)

A3.1 The common pattern

Three rule sets (conditions are tests on observations; a rule that concludes an intermediate fact lets the engine chain):

System	Rules
1. Pump-fault diagnosis (symptoms → fault)	P1: IF noise = rattling AND suction = low THEN fault = cavitation. P2: IF noise = grinding AND bearing_temp = high THEN fault = bearing_wear. P3: IF leak = visible THEN fault = seal_leak. P4: IF flow ∈ {low, zero} THEN flow_deficit = yes. P5: IF flow_deficit = yes AND pressure = high THEN fault = clogged_impeller.
2. Crop-disease advisor (observations → disease → treatment)	C1: IF leaf_spots = brown AND humidity = high THEN disease = leaf_blight. C2: IF leaf_colour = yellow AND roots = rotten THEN disease = root_rot. C3: IF white_powder = yes THEN disease = powdery_mildew. C4: IF disease = leaf_blight THEN treatment = copper_fungicide. C5: IF disease = root_rot THEN treatment = improve_drainage. C6: IF disease = powdery_mildew THEN treatment = sulphur_spray.
3. Flood-relief triage (reports → priority)	F1: IF people_trapped = yes AND water_level = rising THEN priority = critical. F2: IF no_drinking_water = yes AND children = yes THEN priority = high. F3: IF road_blocked = yes AND people_trapped = no THEN priority = medium. F4: IF only_property_damage = yes THEN priority = low. F5: IF medical_need = yes AND priority != critical THEN priority = high.

Common to all three (reusable without changing any code)	Domain-specific
Working memory of attribute–value facts with timestamps; rules of the form IF conditions THEN conclusion; forward chaining that matches rules, builds a conflict set, resolves it, fires and repeats; chaining through intermediate facts (flow_deficit, disease, priority); conflict-resolution strategies (specificity, priority, recency, rule order); HOW and WHY explanation; the knowledge-acquisition interface (add, edit, remove, load, save rules); the evaluation harness (accuracy, regression run).	The attribute vocabulary and allowed values; the rules themselves; the priority values of the conclusions; the question wording used by WHY; the cost of a wrong answer (a wrong triage is more serious than a wrong crop treatment, which influences the chosen conflict strategy).

A3.2 Generic expert-system shell

Component	Content	Exchanges
Knowledge base	Rules stored as data (id, conditions, conclusion, kind, priority, text), loaded from and saved to JSON	gives rules to the inference engine; receives new/edited rules from the acquisition interface
Working memory	Facts (attribute = value) with insertion time and justification (observed or rule id)	receives observations; engine reads and asserts facts; explanation reads justifications
Inference engine	Forward-chaining cycle with conflict resolution	reads KB and WM, returns diagnosis and the trace of fired rules
Explanation facility	HOW: the rule and facts behind a conclusion; WHY: why the shell asks for a missing fact	reads the trace and the KB
Knowledge-acquisition interface	add / edit / remove rule, validate, regression test against stored cases	changes the KB without touching the engine

```
ALGORITHM InferenceCycle(KB, WM, STRATEGY)          // STRATEGY in {specificity, priority, recency,
order}
OUTPUT: diagnosis (or NONE), the list of fired rules
BEGIN
  1   fired <- empty list
  2   REPEAT                                                    // phase 1: derivation rules fire
until nothing new is derived
  3       changed <- FALSE
  4       FOR every rule r in KB with r.kind = derive AND r not in fired DO
  5           IF every condition of r holds in WM THEN ASSERT r.conclusion in WM (justification r) ;
APPEND r to fired ; changed <- TRUE
  6   UNTIL changed = FALSE
  7   conflict_set <- every rule r in KB with r.kind = diagnosis whose conditions all hold in WM
// match
  8   IF conflict_set is empty THEN RETURN NONE
// missing evidence
  9   FOR every rule r in conflict_set DO
// conflict resolution
 10       IF STRATEGY = specificity THEN key[r] <- ( - number of conditions of r , position of r in KB )
 11       IF STRATEGY = priority    THEN key[r] <- ( - priority of r , position of r in KB )
 12       IF STRATEGY = recency     THEN key[r] <- ( - the latest insertion time among the facts that r
tests , position of r in KB )
 13       IF STRATEGY = order       THEN key[r] <- ( position of r in KB )
 14   winner <- the rule in conflict_set with the smallest key
// ties: earlier in the KB
 15   APPEND winner to fired ; RETURN winner.conclusion, fired, conflict_set
// act
END
```

A3.3 Why explanation and knowledge acquisition are essential

The users are pump engineers, agronomists and relief coordinators, not programmers. **Explanation (HOW and WHY)** turns a verdict into something that can be checked by the expert: HOW shows the rule and the observations that produced “bearing wear”, so the engineer can accept it or see that a reading was wrong; WHY tells the user why the shell is asking for one more measurement, so the user trusts the dialogue and can answer correctly. Without explanation an expert system is a black box that experts will not rely on and that cannot be debugged when it is wrong. **Knowledge acquisition** matters because the knowledge belongs to the experts and changes: new failure modes appear, a rule turns out to be too broad. If every change needs a programmer the system stagnates; with a rule editor and a regression run the expert can add or repair a rule and see immediately which earlier diagnoses it changed (shown in B2.3).

Task A4 – Design a Distributed Data Pipeline with RDDs (4 marks)

Problem. About 10^7 readings (sensorId, timestamp, value) per day; required: per-sensor mean and maximum and the ten noisiest sensors (noise = standard deviation of the values).

```
PIPELINE AirQuality(PATH, PARTITIONS = 8)
BEGIN
  1   raw       <- textFile(PATH, minPartitions = PARTITIONS)                  // TRANSFORMATION (lazy):
one record per line
  2   parsed    <- raw.map( line -> (sensorId, timestamp, value) )              // narrow
  3   valid     <- parsed.filter( value is a number AND 0 <= value <= 500 )     // narrow
  4   pairs     <- valid.map( (id, ts, v) -> (id, (v, 1, v, v * v)) )           // narrow: (sum, count,
max, sum of squares)
  5   pairs.cache()                                                             // marked for caching
only if two or more actions use it (see below)
  6   stats     <- pairs.reduceByKey( (a, b) -> (a.sum + b.sum, a.count + b.count, MAX(a.max, b.max),
a.sq + b.sq) )     // WIDE: one shuffle, with map-side combine
  7   perSensor <- stats.mapValues( (s, n, m, q) -> (mean = s / n, max = m, std = SQRT(q / n - (s /
n)^2)) )          // narrow
  8   result    <- perSensor.collect()                                          // ACTION 1: runs the job
(all lines above execute now)
  9   noisiest  <- perSensor.takeOrdered(10, key = -std)                        // ACTION 2: top 10 on
the small per-sensor RDD
 10   RETURN result, noisiest
END
```

Item	Answer
Transformations and actions	textFile, map, filter, reduceByKey, mapValues and cache are transformations: they only extend the lineage graph. Nothing is read or computed until an action: collect (line 8) and takeOrdered (line 9) are the actions; evaluation happens there.
Narrow dependencies	map (2, 4), filter (3), mapValues (7): each output partition depends on one input partition, no data moves between nodes; they are pipelined into one stage.
Wide dependency (shuffle)	reduceByKey (6): all values of a sensor must meet, so the data are repartitioned by key. The combiner (sum, count, max, sum of squares) runs before the shuffle, so each partition ships at most one tuple per sensor instead of all of its readings.
Stages	Stage 1: textFile → map → filter → map → partial reduce (map side). Stage 2: reduce side, mapValues, collect. One shuffle in total.
Where caching helps	pairs is used by two actions if the means and the top 10 are computed by separate jobs from it: caching avoids re-reading and re-parsing the 300 MB of text and re-running steps 2–4. If perSensor (only about 1,000 to 10^4 tuples) is reused it should also be cached because it is tiny.
Where caching wastes memory	raw (the text lines) is used once and is larger than pairs; parsed and valid are intermediate in a narrow pipeline and are never reused; if only one action is run, nothing should be cached.
Lineage and recovery	Each RDD remembers how it was computed from its parent (textFile → map → filter → map → reduceByKey). If a partition of pairs is lost with a failed executor, Spark re-reads only the corresponding input split and re-runs the narrow steps for that partition; the other partitions are untouched. After a wide dependency a lost reduce partition needs the map outputs of all input partitions, so those are kept as shuffle files; checkpointing would be needed only for very long lineages.
Choice of partitions	10^7 readings at about 30 bytes ≈ 300 MB: 8 partitions of about 40 MB, roughly 2–4 per core, enough for load balance without scheduling overhead.

Task A5 – Design When the Full Model Is Too Expensive (4 marks)

Case chosen: AI-based intrusion detection. The detector classifies a network flow (12 KDD-style features) as attack or normal. **Target device (assumed, stated for the estimate):** a small gateway with 512 KB of RAM available for the model, a 100 MHz class core, and a latency budget of 50 ms per flow.

A5.1 Full-model perspective

Full model: a random forest of 200 unpruned trees. Training on about 36,000 flows with noisy labels, an unpruned tree has about 1,000 nodes (2 × the number of leaves); this gives the **estimate** parameters ≈ 200 × 1,000 = 2×10^5 nodes. Each node stores a feature index (2 bytes), a threshold (4 bytes), two child indices (4 + 4 bytes) and a leaf probability (4 bytes) = 18 bytes: **memory ≈ 3.6 MB**. Operations per sample: one

comparison per level of each tree, 200 trees × about 13 levels ≈ **2,600 node visits** plus 200 additions. (Part B3 measures 208,438 nodes, 3,664 KiB and 2,666 node visits per flow, so the estimate was within 5%.)

Why it exceeds the budget. On the assumed device the **memory** is the binding constraint: 3.6 MB against 512 KB is a factor of 7 too large. The latency estimate gives about 2,600 visits × 30–50 cycles ≈ 1.3 ms at 100 MHz, which is *inside* the 50 ms budget; so the honest statement is that the full model violates the memory limit and not the time limit, and the time budget becomes critical only on a much slower device or when a flow rate of thousands of flows per second must be sustained (throughput 1000/1.3 ≈ 770 flows/s).

A5.2 Compressed pipeline

Approximation: an early-exit cascade with a smaller, quantised second stage. Stage 1 is a depth-3 decision tree on five cheap features (count, serror_rate, diff_srv_rate, failed_logins, num_compromised) that decides the clear cases; only uncertain flows go to stage 2, a 15-tree forest of depth at most 8 with 8-bit thresholds and 8-bit inputs.

```
ALGORITHM EdgeDetect(flow x)
INPUT : x - 12 feature values of one flow; stage1 - tree of depth 3; stage2 - 15 trees (depth <= 8) with
thresholds scaled to 0..255;
        LOW = 0.03 ; HIGH = 0.97 ; ALARM = 0.5 ; lo[j], scale[j] - quantisation constants per feature
OUTPUT: attack (TRUE / FALSE) and the probability p
BEGIN
  1   p <- LEAF-PROBABILITY( stage1 , x )                                        // at most 3
comparisons
  2   IF p <= LOW OR p >= HIGH THEN RETURN ( p >= ALARM , p )                    // early exit:
confident, no stage 2
  3   FOR every feature j DO  q[j] <- MIN(255, MAX(0, FLOOR( (x[j] - lo[j]) / scale[j] )))   // quantise
the input
  4   total <- 0
  5   FOR every tree t in stage2 DO
  6       node <- root of t
  7       WHILE node is not a leaf DO
  8           IF q[ feature of node ] <= threshold of node THEN node <- left child ELSE node <- right
child
  9       total <- total + leaf probability of node
 10   p <- total / 15
 11   RETURN ( p >= ALARM , p )
END
```

Operations saved. About 88% of flows leave at line 2 after 3–4 comparisons; the rest need at most 15 × 8 = 120 comparisons instead of 2,600: expected visits ≈ 0.88 × 4 + 0.12 × 120 ≈ 18 (measured 18), a reduction of about 150×. **Memory:** 15 + 3,133 nodes at 7 bytes (1-byte feature id, 1-byte threshold, two 2-byte child indices, 1-byte value) ≈ 21.5 KiB, which fits in 512 KB with a wide margin; the model is about 170× smaller.

A5.3 What have we sacrificed?

Loss	Why it may happen	How I would detect it
Accuracy on subtle attack classes	A depth-3 stage 1 and a depth-8 stage 2 with 8-bit thresholds cannot represent fine boundaries; the r2l and u2r classes (few flows, small differences) are the first to suffer	per-class recall on a held-out time-ordered test set, not only overall accuracy
Probability quality (AUC)	Few trees and early exits give coarse probabilities, which lowers ROC-AUC and makes thresholds less tunable	compare ROC-AUC and the threshold sweep with the full model
Robustness to unseen attacks	A shallow stage 1 that exits early may send a new attack straight to “normal” with high confidence and never reach stage 2	evaluate on a held-out attack type that was never in training; check the early-exit share per class
Adaptability	Retraining the cascade may not help if stage 1 cannot isolate the new class	test whether adding labelled examples of a new class improves recall

Loss	Why it may happen	How I would detect it
Explainability	A smaller model gives simpler explanations, but the quantised thresholds are less readable	inspect the decision path of the flagged flows

PART B – IMPLEMENTATION, INSTRUMENTATION AND EVIDENCE (25 marks)

All programs are in Appendix A and in the accompanying zip file. Every experiment uses fixed seeds, so the numbers below can be reproduced with `PYTHONHASHSEED=0 python3 <file>`. The code keeps the **agent logic** (`drone.py`, `swarm_decide` in `swarm.py`, `es.py`, `ids.py`, `chatbot.py`) apart from the **simulator or harness** (`MissionEnv`, `Field`, `run`, the t_*.py test drivers).

File	Role	Produces
drone.py	A* planner, `DroneAgent` (hybrid layers) and `MissionEnv` simulator	t_b11_output.txt
swarm.py	pheromone swarm rule, centralised controller, field simulator	t_b13_output.txt, fig/b1_swarm.png
es.py, pump_kb.py	generic expert-system shell; pump knowledge base and 30 cases	t_b2_output.txt
ids.py	flow simulator, full forest, cascade, flat-array engine	t_b3_output.txt, t_b3b_output.txt, fig/b3_edge.png
minirdd.py	mini-RDD engine (partitions, lineage, shuffle, cache)	t_b34_output.txt, fig/b3_rdd.png
chatbot.py	TF-IDF retrieval chatbot, WER corruption	t_b4_output.txt, fig/b4_chat.png

Task B1 – Implement and Instrument the Drone Agent and the Swarm (7 marks)

B1.1 Scenarios with logs (2 marks)

`DroneAgent.decide(percept)` returns an action, the layer that decided and a note; `MissionEnv` is the simulator. Five scenarios, each with the expectation written beforehand (Appendix B gives the full step-by-step logs):

Scenario	Expected behaviour (before the run)	Actual output	Verdict
S1 normal flight, goal (6,6), no-fly block, battery 40	routes around the no-fly block, delivers, returns, 0 incidents, 26 steps (12 + 12 + drop + land)	delivered, LANDED_HOME, 26 steps, battery left 15, incidents 0	as expected
S2 sudden obstacle at (0,4) after step 3	L0 veto at (0,3), replan around it, still delivers, replans ≥ 2	delivered, 18 steps, 1 veto, 3 replans, incidents 0	as expected
S3 boundary: goal cell becomes an obstacle	mission aborted safely, returns and lands, not delivered	not delivered, LANDED_HOME, 9 steps, incidents 0	as expected
S4 boundary: battery 14 and a goal 9 cells away	battery rule turns the drone back with the reserve intact	RETURN at t = 5 (note printed: battery 9 ≤ distance home 5 + reserve 3, with the 1.6 detour margin), landed with 4 left	as expected
S5 wind drift 20% per move, 30 seeds	drone should still get home most of the time	see the table below: a **bug was found**	prediction partly wrong

Bug found by S5. The first version lost the drone to battery exhaustion in many wind runs: after a drift the stored path was no longer adjacent to the drone, the agent classed the plan as inconsistent and chose Hold, burning one battery unit per step. The fix (line 16 of the pseudocode: drop the path when its first cell is not adjacent, then replan) and the adaptive margin of line 11 were added; the table gives the results after the fix:

Configuration (20% drift, 30 seeds)	Delivered	Landed home	Battery exhausted	Mean steps
battery 36, fixed reserve 3	3/30	28/30	2	31.9

Configuration (20% drift, 30 seeds)	Delivered	Landed home	Battery exhausted	Mean steps
battery 36, adaptive margin	1/30	**30/30**	0	26.2
battery 50, fixed reserve 3	29/30	29/30	1	36.6
battery 50, adaptive margin	16/30	**30/30**	0	33.7

Interpretation. The adaptive margin trades deliveries for safety: it never loses the drone (30/30) but delivers only 16/30 at battery 50, because it turns back whenever the wind estimate makes the way home look expensive. Whether that trade is right depends on the cost of a lost drone against a failed delivery, which the utility in A1.3 expresses with its weights (−500 per forced landing). “Incidents” counts move attempts that, after the wind drift, would have entered an obstacle, a no-fly cell or the outside of the grid (the move is blocked and the drone stays where it is, so there is no physical collision in the simulator); the mean of about 1 per flight in the wind runs is these blocked drifts. In all scenarios without wind it is 0.

B1.2 Compare the A1.5 hand trace with the program trace (1 mark)

The program was run on exactly the A1.5 scenario (8 × 8, home (0,0), goal (0,6), battery 14, reserve 3, obstacle at (0,4) appearing before the move of step 4). Result: 11 steps, not delivered, landed with battery 4, 1 veto, 0 incidents.

t	Hand trace (A1.5)	Program trace	Same?
0–2	E, E, E from (0,0) to (0,3)	E, E, E	yes
3	(0,3), obstacle (0,4) sensed: veto, replan, move S	same: L0 veto, move S	yes
4	(1,3): E	(1,3): E	yes
5	(1,4), battery 9: 9 > 5 + 3, continue E	(1,4), battery 9: **RETURN** (“battery 9 ≤ distance home 5 + reserve 3 + margin”)	**no**
6–	(1,5) battery 8: RETURN; path W, W, … (along row 1 first)	(1,3) N to (0,3), then W, W, W along row 0	**no** (route)
end	land at (0,0) with battery 2 (12 steps)	landed with battery 4 (11 steps)	**no**

Explanation of every difference. (1) *The turn-back came one step earlier (t = 5).* In the hand trace I used the pure rule need = d_home; the implementation multiplies the distance by a 1.6 safety factor for the detour that an unknown obstacle may force (line 11 of the program, comment “+60% safety”). With d_home = 5 the threshold is 1.6 · 5 + 3 = 11 and the battery 9 is below it. The program is the safer of the two; the hand trace was too optimistic and would have used one more unit of battery before turning back. (2) *Different landing battery (4 against 2) and one step fewer.* This follows entirely from (1): the program turned back from (1,4), five cells from home, the hand trace from (1,5), six cells from home, so the return was one move shorter and the landing battery one unit higher; the remaining unit is the program printing the battery after the last move while the hand trace counted the Land row separately. (3) *Different route home.* From (1,4) both a path along row 1 and a path that climbs to row 0 early are shortest paths of equal length; the program’s A* tie-break order (N, E, S, W) chose to climb at (1,3) whereas the hand trace assumed that the drone would walk west along row 1 first. The difference is a tie-break only. No difference is a logic error of the program.

B1.3 The swarm for N drones (2 marks)

What the improvement targets. The A2 redesign targets three weaknesses of the central controller: its single point of failure, the lossy command link and the growth of communication with N. `swarm_decide`

uses only the local pheromone marks within RS = 4 and the four neighbour cells; the centralised baseline `central_targets` assigns every drone the nearest unvisited cell from a global view. The same field simulator (30 × 30 cells, 5% no-fly, battery 120 moves, 150 steps) runs both.

A2.3 check. The program reproduces the hand trace of A2.3 move by move (steps 1–4: 6, 9, 12 and 15 visited cells; steps 5–6: 18 and 20 cells = 80%). **A2.4 check.** The failure configuration was run after the prediction was written: the predicted behaviour (both drones walk toward the single unvisited cell in tandem) occurred; the prediction of the number of steps was wrong by one (the distances are 8 and 7, I wrote 7 and 6), and the real run took 14 moves to gain one new cell (redundancy ratio 0.929), with one drone oscillating between (3,4) and (4,4) because the other blocked its path.

B1.4 Experiment: single drone against swarm (2 marks)

30 seeds for every cell of the table; success = coverage ≥ 70% of the free cells within 150 steps; redundancy = share of moves onto a cell that was already visited; "t to 50%" is the median step at which 50% coverage was reached (in brackets the number of seeds that reached it); latency = time of one decision (µs, this machine).

N	fail	success	coverage	t to 50%	redundancy	battery/drone	latency µs
1	0%	0.00	0.136	never	0.043	120.0	2.5
1	10%	0.00	0.044	never	0.038	38.0	2.4
1	30%	0.00	0.044	never	0.038	38.0	2.5
5	0%	0.00	0.592	102 (30/30)	0.159	120.0	3.6
5	10%	0.00	0.518	114 (21/30)	0.141	102.8	3.5
5	30%	0.00	0.453	never	0.104	86.3	2.8
10	0%	1.00	0.944	61 (30/30)	0.329	120.0	5.4
10	10%	1.00	0.902	62 (30/30)	0.308	111.2	5.3
10	30%	1.00	0.814	69 (30/30)	0.256	93.3	5.1
25	0%	1.00	1.000	46 (30/30)	0.716	120.0	15.4
25	10%	1.00	1.000	48 (30/30)	0.686	108.9	15.8
25	30%	1.00	0.998	49 (30/30)	0.622	90.2	11.8
50	0%	1.00	1.000	46 (30/30)	0.857	119.6	16.9
50	10%	1.00	1.000	46 (30/30)	0.844	109.5	14.8
50	30%	1.00	1.000	48 (30/30)	0.809	89.6	14.5

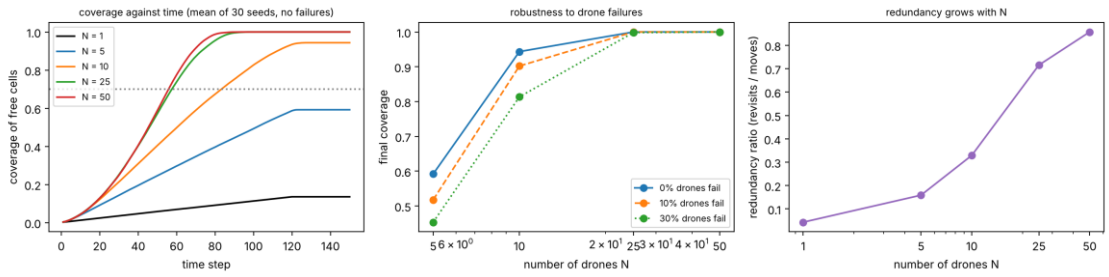


Figure 1 – Coverage against time and the effect of drone failures (30 seeds; B1.4).

Condition (N = 10)	Design	Coverage	Success
perfect links	central / swarm	0.897 / 0.944	0.97 / 1.00
30% command loss	central / swarm	0.811 / 0.944	0.83 / 1.00

Condition (N = 10)	Design	Coverage	Success
controller fails at step 30	central / swarm	**0.208 / 0.944**	**0.00 / 1.00**
10% drones fail	central / swarm	0.866 / 0.902	0.97 / 1.00
30% drones fail	central / swarm	0.755 / 0.814	0.83 / 1.00

Interpretation. (1) *Scaling:* success jumps from 0.00 at N = 5 to 1.00 at N = 10; there is a threshold number of drones below which 150 steps cannot cover 70% of the field. Beyond N = 25 more drones add almost nothing to coverage (1.000) and shorten the time to 50% only slightly (46 steps); instead **redundancy** rises from 0.33 to 0.86 and the work per drone falls, so N = 10–25 is the economical range for this field. (2) *Robustness:* with 30% of the drones failing N = 10 still covers 0.814, N ≥ 25 is unaffected; the loss is graceful. (3) *Latency* per decision grows slowly (2.5 → 17 µs) because the rule is local; the cost per drone does not depend on N, only the simulator's occupancy check does. (4) *Central against swarm:* the central controller is slightly worse even with perfect links, and loses everything when it fails (0.208). The swarm is not better in every respect: its redundancy is high for large N and it has no guarantee of full coverage. **Artefact in the table:** for N = 1 the 10% and 30% failure rows are identical (0.044) because ⌈0.1·1⌉ = 1, so "10% failure" already kills the only drone; the rows are valid for the N = 1 case but should not be compared with the larger N.

Task B2 – Engineer an Adversarial Input for the Expert System (5 marks)

B2.1 The pump expert system (2 marks)

`es.py` is the generic shell of A3.2 (rules as data, working memory with timestamps, two-phase forward chaining, four conflict strategies, HOW and WHY, run-time rule editing). `pump_kb.py` holds the pump knowledge: **16 rules** (2 derivation rules D1–D2 and 14 diagnosis rules R1–R14) for **6 faults** (cavitation, bearing wear, seal leak, clogged impeller, motor overload, misalignment) and **30 labelled cases written by me** (they are not real plant data). The rules were written from the 15 development cases only; the other 15 cases were kept aside and not looked at until the rules were fixed.

Rule	Condition → conclusion	Rule	Condition → conclusion
D1 (derive)	flow ∈ {low, zero} → flow_deficit	R8	leak = visible AND pressure = low → seal_leak
D2 (derive)	motor_temp = high AND current = high → electrical_stress	R9	flow_deficit AND pressure = high → clogged_impeller
R1	noise = rattling AND suction = low → cavitation	R10	flow_deficit AND current = high AND pressure = high → clogged_impeller
R2	rattling AND pressure = low AND vibration = high → cavitation	R11	electrical_stress → motor_overload
R3	rattling AND vibration = high → cavitation	R12	motor_temp = high AND current = high AND flow = normal → motor_overload
R4	grinding AND bearing_temp = high → bearing_wear	R13	axial_vib = high AND vibration = high → misalignment
R5	grinding AND vibration = high → bearing_wear	R14	axial_vib = high AND bearing_temp = normal → misalignment
R6	bearing_temp = high AND vibration = high → bearing_wear	R7	leak = visible → seal_leak

Result (specificity, ties by rule order)	Value
Development accuracy (15 cases)	1.000 (15/15) – expected, the rules were fitted to these

Result (specificity, ties by rule order)	Value
Hold-out accuracy (15 cases)	**0.867 (13/15)**
Mean rules matched / fired per hold-out case	2.33 / 1.47

Fault	Precision	Recall	Support (hold-out)
cavitation	1.00	1.00	3
bearing_wear	0.75	1.00	3
seal_leak	1.00	1.00	2
clogged_impeller	1.00	1.00	2
motor_overload	1.00	0.67	3
misalignment	1.00	0.50	2

The two errors and why they happened. *H12 (motor overload, predicted None):* the case has a hot motor and normal flow but a *normal* current reading, so neither R11 (needs current = high) nor R12 fires; my development cases always had both high. A gap in the knowledge, not a conflict. *H14 (misalignment, predicted bearing wear):* the case has axial vibration with grinding noise and high vibration; R5 (grinding AND high vibration) and R13 both match, and under specificity both have two conditions, so the earlier rule R5 wins. A conflict resolved wrongly by the tie-break.

HOW and WHY (examples from the run).

```
case H3 {'noise': 'grinding', 'vibration': 'high', 'bearing_temp': 'high', 'flow': 'normal'}:
HOW: diagnosis 'bearing_wear' was concluded by rule R4 (grinding noise with hot bearing).
   because noise = grinding  [observed]
   because bearing_temp = high  [observed]
   other rules that also matched and lost the conflict resolution: ['R5', 'R6']
WHY ask 'suction'? Because rule R1 (rattling noise with low suction pressure) could conclude
'cavitation' and it needs
suction = low; the other conditions are already satisfied or unknown.
```

B2.2 Adversarial cases and conflict resolution (2 marks)

Five adversarial cases were designed, each aimed at a particular weakness, with the predicted outcome for each of four conflict-resolution strategies written *before* the run: **X1** two faults with equal specificity (R4 and R13 both apply), **X2** missing evidence (only a weak rule matches), **X3** order dependence (a seal-leak symptom together with a motor-overload pattern), **X4** dominance (a high-priority rule against a more specific rule), **X5** no evidence at all.

Case	specificity	priority	recency	order	Truth / comment
X1 conflicting rules	bearing_wear ✓	bearing_wear ✓	misalignment ✓	bearing_wear ✓	ambiguous; two true faults possible
X2 missing evidence	cavitation ✓	cavitation ✓	cavitation ✓	cavitation ✓	only R3 matches: right answer from thin evidence
X3 order dependence	motor_overload ✓	motor_overload ✓	motor_overload ✓	**seal_leak** ✓	only “order” lets rule position decide
X4 dominance	clogged_impeller ✓	motor_overload ✓	**motor_overload (predicted clogged_impeller) ✗**	clogged_impeller ✓	the only wrong prediction
X5 no evidence	None ✓	None ✓	None ✓	None ✓	the system refuses to guess

19 of 20 predictions were right. The wrong one (X4, recency): I assumed the newest fact would belong to R9/R10, but the derived fact `electrical_stress` is asserted by the derivation rule *after* the observations, so R11 became the "most recent" rule. Recency counts derived facts as new, which is a property of the engine that I had not reckoned with. This was found by running the experiment, not by thinking.

Strategy	Hold-out accuracy	Adversarial correct (of 5)	Cases whose diagnosis changes when the rule order is reversed
specificity	0.867	4/5	H14, X1
priority	0.867	3/5	none (most stable)
recency	0.867	2/5	X4
order	0.867	3/5	H14, X1, X3, X4 (least stable)

Interpretation. On the hold-out set all four strategies are equal (0.867), so the 15 hold-out cases cannot separate them; the adversarial cases can. Specificity is the best on the adversarial set (4/5) because the more specific rule usually encodes more evidence. Priority is the most **stable** (no case changes when the rule order is reversed) and is the right choice when the cost of a wrong diagnosis differs by fault (safety-critical faults get high priority). "Order" is the worst because the answer depends on how the knowledge engineer happened to type the rules. A recommendation: specificity first, priority as the tie-break, rule order only as the last resort.

B2.3 Knowledge acquisition at run time with a regression run (1 mark)

Four edits were applied through the acquisition interface **without changing the engine**; after each, all 30 cases were run again and the cases whose diagnosis changed were listed:

Edit	Accuracy on 30 cases	Diagnoses that changed	Reading
1 ADD R15: motor_temp = high AND flow = normal → motor_overload (to cover H12)	0.933 → **0.967**	H12: None → motor_overload	the added rule repairs the gap and breaks nothing
2 EDIT R5: add 'axial_vib ≠ high' (to cover H14)	0.933 → 0.933	H14 repaired; **H5 broken** (bearing_wear → None)	a gain and a loss cancel: the regression run shows the side effect
3 careless EDIT of R6: bearing-temperature condition dropped	0.933 → **0.900**	H7: seal_leak → bearing_wear	an over-general rule silently steals a correct case
4 REMOVE R14	0.933 → 0.933	none	R14 was redundant with R13

Interpretation. Edit 2 is the typical trap of knowledge acquisition: the expert fixes the case in front of them and breaks another that nobody was looking at. Only the regression run reveals it. This is why A3.3 states that explanation alone is not enough, and why a stored case library is part of the shell.

Task B3 – Applied Case: Full Model against Edge Model (8 marks)

Applied case: (a) AI-based intrusion detection. Decision: is a network flow an attack? **Data:** a flow simulator written by me (`ids.gen_stream`) with 12 features named as in the KDD Cup 99 / NSL-KDD schema (duration, src_bytes, dst_bytes, count, srv_count, serror_rate, rerror_rate, same_srv_rate, diff_srv_rate, failed_logins, logged_in, num_compromised); schema from Tavallaee, Bagheri, Lu and Ghorbani (2009). Seven traffic types: normal, burst_normal (flash crowds), dos, probe, r2l, u2r and **slow_dos**, which exists only in the last 30% of the stream. Normal traffic drifts (higher counts) and bursts

become more frequent over time. **This is simulated data**: the numbers show how the two models behave on a stream with drift and an unseen attack, and are not NSL-KDD results.

B3.1 Leakage-safe time split and the metrics (2 marks)

The stream of 60,000 flows is ordered by time and split **by time, not at random**: the first 60% train the models, the next 10% choose thresholds (validation), the last 30% are the test set, so no future flow influences training, and the normalisation constants of the 8-bit quantisation come from the training part only. Three stream seeds were used. The flat-array inference engine (the one that is timed) was checked against scikit-learn: maximum absolute difference of the probabilities 0.0 in all three seeds.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Full: 200-tree forest	0.981 ± 0.001	0.997 ± 0.000	0.940 ± 0.002	0.968 ± 0.001	0.999 ± 0.000
Edge: depth-3 tree + 15 quantised trees	0.968 ± 0.000	0.993 ± 0.001	0.901 ± 0.001	0.945 ± 0.000	0.950 ± 0.004

Mean ± standard deviation over 3 stream seeds, test period, threshold 0.5.

Seed-0 confusion matrix (test)	Predicted normal	Predicted attack
Full – actual normal	12,604 (TN)	16 (FP)
Full – actual attack	336 (FN)	5,044 (TP)
Edge – actual normal	12,587 (TN)	33 (FP)
Edge – actual attack	531 (FN)	4,849 (TP)

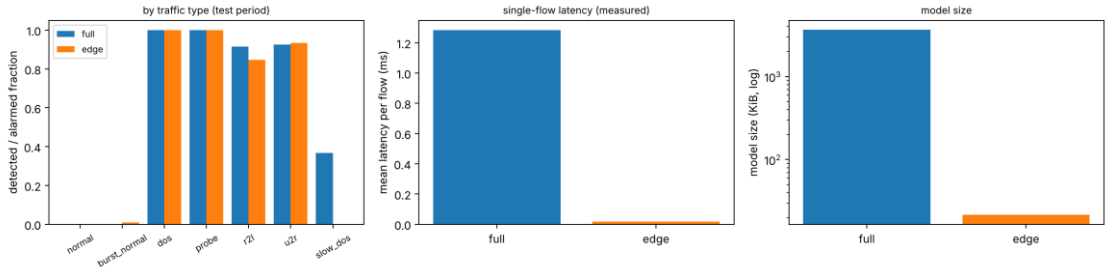


Figure 2 – Full against edge model: accuracy, node visits, model size and latency (B3.1, B3.4).

Interpretation. The edge model gives up 1.3 points of accuracy, 3.9 points of recall and 0.05 of AUC. Its precision is almost unchanged (0.993): it stays quiet and misses more. The AUC loss is larger than the accuracy loss because the cascade's probabilities are coarse (stage-1 leaves give a few fixed values). For an intrusion detector missing attacks is the costlier error, so the recall loss is the number to watch.

B3.2 Mis-predictions: categories and a demonstrated failure class (2 marks)

Predictions written before the analysis. P1: both models detect the seen attack classes (dos, probe, r2l, u2r) with recall ≥ 0.9, except u2r (2% of the data). P2: recall on the unseen slow_dos < 0.3 for both models. P3: the false-alarm rate on flash-crowd bursts is above that on ordinary normal flows, because bursts look like dos in the count feature.

Traffic type	n (test)	Full: detected	Edge: detected	Prediction check
normal	11,313	0.001 (false alarms)	0.002	–
burst_normal	1,307	0.000	0.010	**P3 wrong for the full model** (0.000, below normal); right in direction only for the edge model (0.010 > 0.002)
dos	2,202	1.000	1.000	P1 ✓

Traffic type	n (test)	Full: detected	Edge: detected	Prediction check
probe	1,406	1.000	0.999	P1 ✓
r2l	1,063	0.914	0.848	P1 ✓ full, ✗ **edge (0.848 < 0.9)**
u2r	365	0.926	0.934	P1 – I expected it lower; it is not, the u2r signature (compromised, logged_in) is easy here
slow_dos (unseen)	344	0.366	**0.000**	**P2 wrong for the full model** (0.366 > 0.3), right for the edge model

Categorised mis-predictions (seed 0, test period, 1 row = 1 flow; the full list is in Appendix B):

Category	Model, error	Example (count, serror, duration, failed_logins, compromised, logged_in)	P(attack)
1. Unseen attack type (slow_dos)	full and edge, FN	10, 0.05, 12.5, 0, 0, 1 – a slow flow that looks like a long normal session	0.325 full / 0.015 edge
2. Subtle remote-to-local (few failed logins)	full and edge, FN	5, 0.05, 5.8, 1, 0, 0	0.395 / 0.015
3. Rare class (u2r with a low compromise count)	full and edge, FN	3, 0.10, 2.3, 0, 0, 1	0.190 / 0.015
4. Flash crowd mistaken for an attack	edge, FP	74, 0.21, 1.0, 0, 0, 1 – a burst of legitimate traffic that resembles dos	0.999 (edge)
5. Normal flow with unusual values	full, FP and edge, FP	10, 0.05, 2.5, 3, 0, 1 (failed logins by a forgetful user) and 8, 0.09, 7.1, 0, 1, 1	0.785 / 0.577

Traffic type	Full errors	Edge errors
normal	16	20
burst_normal	0	13
dos / probe	0	0 / 1
r2l	91	162
u2r	27	24
slow_dos	218	344

Demonstrated failure class: an attack type never seen in training

The same models detect **100% of dos flows** seen in training but only **36.6% (full) and 0.0% (edge)** of the 344 slow_dos flows, whose features resemble long normal sessions (count 9, serror 0.25, duration 20). This is the failure class of a learned detector: it generalises inside the distribution it saw, not to a new attack strategy.

Retraining test. After adding 200 labelled slow_dos flows to the training data and retraining: the full model detects **0.953** of slow_dos (overall F1 0.986), but the edge cascade **still detects 0.000**. The reason is structural: the depth-3 stage 1 confidently labels these flows as normal (probability about 0.015, below the 0.03 early-exit threshold), so stage 2 never sees them and the new labels cannot change the answer. **Lesson: compression can remove the model's ability to learn a new class, not only its accuracy.** The remedy is to retrain stage 1 with the new class and to monitor the early-exit share per class.

Threshold sweep for the edge model (test period; costs of a missed attack and a false alarm decide the operating point):

Threshold	0.1	0.2	0.3	0.5	0.7	0.9
Recall	0.904	0.904	0.903	0.901	0.887	0.825
Precision	0.969	0.983	0.989	0.993	0.995	0.995
False-alarm rate	0.013	0.007	0.004	0.003	0.002	0.002

Lowering the threshold below 0.5 does **not** recover recall (0.904 at best), because the missed attacks leave through stage-1 early exits at probability ≈ 0.015, not through the threshold: another sign that the failure is in the cascade's structure. **Drift check:** the edge F1 in the thirds of the test period is 0.940, 0.949 and 0.946; the drift in this simulator is too mild to degrade the model noticeably, so I make no claim about drift robustness.

B3.3 PySpark-style RDD pipeline: sizes, partitions, caching and shuffles (2 marks)

> **Not PySpark**
> PySpark could not be installed here (no network route to the package index), so these measurements come from `minirdd.py`, a mini-RDD engine written for this assignment: partitioned numpy arrays, lazy lineage, narrow and wide dependencies, map-side combine, `cache()`, lineage recomputation, a process pool with `min(partitions, 2)` workers on a **2-core** container. It demonstrates the ideas and the shapes of the curves, not Spark's numbers. The same pipeline in PySpark is given in Appendix C (not executed).

Pipeline (the A4 design): clean (validity filter) → map to (sensor, (sum, count, max, sum of squares)) → reduceByKey (the one shuffle) → per-sensor mean, max, std → top-10 noisiest. **Prediction before running:** the time falls as the partitions rise until the number of partitions reaches the number of cores (2 here); beyond that no gain; for 1e5 and 1e6 records the fixed cost of starting tasks will exceed the work, so more partitions will be *slower*; map-side combine will cut the shuffle volume by about the ratio of records to sensors per partition; caching will help only if the cached step is expensive.

Records	1 partition	2	4	8	16	Best against 1 partition
10^5	0.001	0.010	0.014	0.012	0.014	1.00× (1 partition)
10^6	0.011	0.033	0.025	0.028	0.028	1.00× (1 partition)
10^7	0.221	0.201	0.142	0.116	0.097	**2.29×** (16 partitions)

Wall-clock seconds, median of 3 runs. The aggregate is identical for 1, 3 and 8 partitions (maximum absolute difference of the per-sensor std 4.5×10^{-13}): partitioning does not change the result.

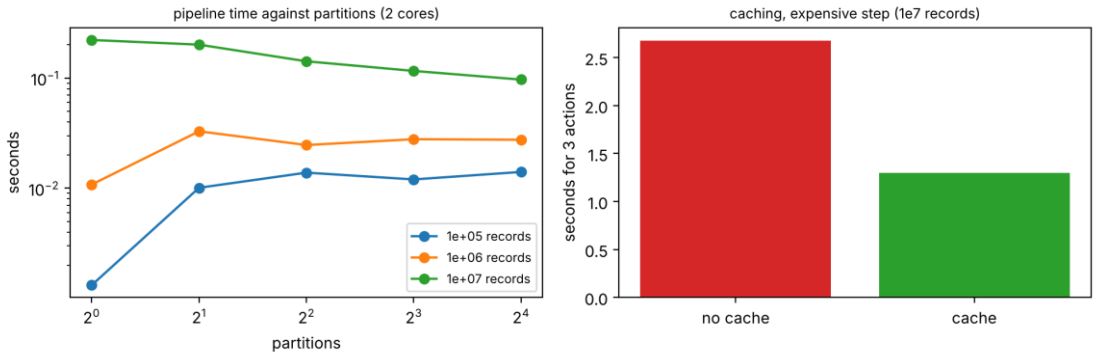


Figure 3 – Mini-RDD experiments: scaling with partitions, shuffle volume, caching (B3.3).

Experiment	Result	Reading
Shuffle: reduceByKey with map-side combine against shipping every record (1e6, 8 partitions, 1000 sensors)	1 shuffle in both; **0.26 MB against 15.68 MB**; 0.032 s against 0.113 s	same shuffle count, 60× less data: the combiner, not the shuffle count, is what matters
Caching, cheap intermediate step (validity filter), 3 actions, 1e7	no cache 0.278 s; **cache 0.584 s**	caching **hurt**: storing and fetching 8 partitions cost more than recomputing a cheap step
Caching, expensive step (filter + sort), 3 actions	no cache 2.678 s; **cache 1.298 s**	caching helps once the step is expensive and reused (2.1× faster)
Skew: one sensor holds 50% of the readings (1e7, 8 partitions)	combine: 0.113 s → 0.140 s; no combine: 1.029 s → 1.024 s; shuffle 0.3 MB and 156.8 MB	with the combiner the hot key costs little; without it the 156.8 MB would all flow to one reducer (here measured as volume and time only)
Lineage recovery: drop partition 3 of 8	1 partition recomputed, result identical (True), 0.096 s	only the lost partition is recomputed from its lineage

Where more partitions did not help, and why. (1) *1e5 and 1e6 records:* one partition is the fastest (0.001 s and 0.011 s); two or more cost 0.010–0.033 s. The work is about a millisecond, whereas starting the workers, sending a partition index and gathering the partial results costs several milliseconds, a fixed overhead per task; the extra partitions only multiply it. (2) *1e7 records beyond 2–4 partitions:* the container has 2 cores, so at most 2 tasks run at the same time; the gain from 4 to 16 partitions (0.142 → 0.097 s) comes from smaller partitions that fit better in the cache and balance the load, not from more parallelism, and it is a small effect. The 2.29× at 16 partitions on 2 cores is above the 2× that parallelism alone allows because of that cache effect. (3) *Skew:* more partitions would not help a hot key, because all the records of a key go to one reducer whatever the number of partitions; the combiner does.

B3.4 Compressed model: latency, throughput, size, accuracy change and the 50 ms verdict (2 marks)

Latency was measured per flow with the pure-Python flat-array engine (a software model of what the device would run), one flow at a time, on this container; the numbers show the **relative** cost of the two models, not the speed of a particular microcontroller.

Metric	Full forest (200 trees)	Edge cascade	Ratio
Nodes	208,438	15 + 3,133 = 3,148	66×
Model size	3,664 KiB (18 bytes/node)	21.5 KiB (7 bytes/node, 8-bit thresholds)	**170×**
Mean node visits per flow	2,666	18	**152×**
Latency mean / p95 / p99	1.284 / 2.244 / 2.437 ms	0.019 / 0.108 / 0.183 ms	69× (mean)
Sequential throughput	779 flows/s	54,003 flows/s	69×
Stage-1 early exits	–	0.871–0.903 (0.885 in the timing sample)	
F1 / AUC (test)	0.968 / 0.999	0.945 / 0.950	−0.023 / −0.049
Recall on unseen slow_dos	0.366	0.000	–

50 ms budget (p95). Both models meet it on this machine (2.244 ms and 0.108 ms). A device that is up to **22× slower** than this container would still be within the budget with the full model (2.244 ms × 22 ≤ 50 ms) and up to **462× slower** with the edge model. This is a tolerance factor, not a claim about any real device, because this container's speed was not compared with a specific microcontroller. **Verdict:** time is not what

rules out the full model here, **memory is** (3.6 MB against the assumed 512 KB, as estimated in A5.1); the edge model satisfies both with a wide margin, at the price of the losses listed above.

Task B4 – Conversational Interface and Deployment Decision (5 marks)

B4.1 Retrieval chatbot with an out-of-scope threshold (2 marks)

`chatbot.py` is a retrieval-based assistant for the intrusion-detection console. It has **9 intents** (list_alerts, alert_count, why_flagged, top_source, model_status, latency_info, attack_info, ack_alert, help) plus out_of_scope, 90 training utterances (10 per intent), a TF-IDF vector for every intent, cosine similarity, and an **out-of-scope (OOS) threshold**: if the best cosine is below the threshold the bot answers "I can only help with intrusion-detection questions". A small dialogue state handles missing slots ("Which alert number should I acknowledge?"). All utterances were written by me; the test sets (54 in-scope utterances with different wording and 20 OOS) are not from a public corpus.

Prediction. Accuracy about 0.8; OOS questions that share words with intents ("what is the time") will be accepted wrongly; I expected 20–30% false acceptances. **Result: 55%**, a bad miss.

Intent	Precision	Recall	F1	n
list_alerts	0.67	0.67	0.67	6
alert_count	0.83	0.83	0.83	6
why_flagged	0.50	0.83	0.62	6
top_source	0.75	1.00	0.86	6
model_status	0.80	0.67	0.73	6
latency_info	0.60	1.00	0.75	6
attack_info	1.00	1.00	1.00	6
ack_alert	0.86	1.00	0.92	6
help	0.83	0.83	0.83	6
out_of_scope	0.90	0.45	0.60	20

Version 1, threshold 0.30, clean text: accuracy 0.757 (n = 74), macro-F1 0.782, out-of-scope falsely accepted 0.55, in-scope wrongly rejected 0.02, mean latency 0.052 ms per utterance (pure Python, one core).

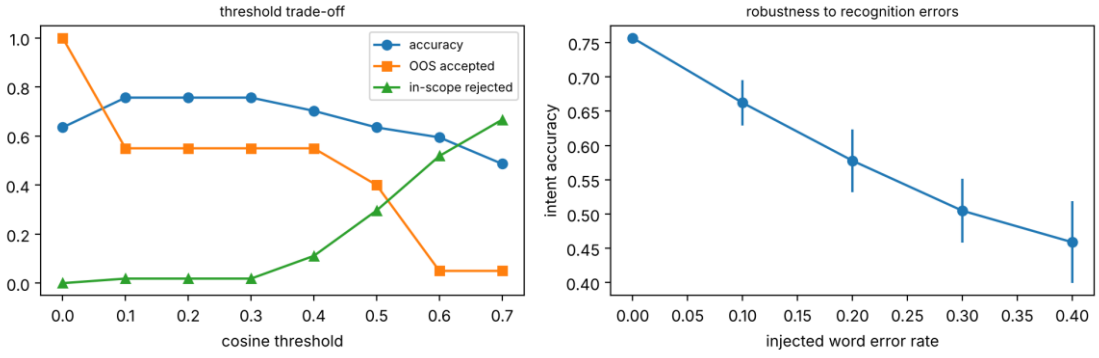


Figure 4 – Chatbot: threshold sweep, effect of speech-recognition errors, and the stop-word improvement (B4.1, B4.2).

Error analysis. 13 of the 18 mistakes are OOS questions accepted as an intent: "what is the capital of france" → latency_info, "tell me a joke" → why_flagged. The cause is that question words ("what", "who", "how", "tell", "is") occur in many training utterances and carry no intent. **Improvement (v2):** add question

words to the stop list. Since the change was motivated by test set 1, it was checked on a **second set of 30 fresh utterances** (18 in-scope, 12 OOS) written afterwards:

Version, threshold	Set 1 accuracy	Set 1 OOS accepted	Set 1 in-scope rejected	Set 2 accuracy	Set 2 OOS accepted	Set 2 in-scope rejected
v1, 0.3	0.757	0.55	0.02	0.633	0.83	0.00
v1, 0.4	0.703	0.55	0.11	0.633	0.83	0.06
v1, 0.5	0.635	0.40	0.30	0.567	0.58	0.33
v2, 0.3	**0.784**	**0.15**	0.07	**0.833**	**0.08**	0.00
v2, 0.4	0.784	0.15	0.07	0.833	0.08	0.11
v2, 0.5	0.649	0.10	0.31	0.733	0.08	0.39

v2 cuts OOS acceptance from 0.55 to 0.15 on set 1 and from 0.83 to 0.08 on the fresh set 2, so the improvement is not just fitted to set 1. The price is a slightly higher rejection of real questions on set 1 (0.02 → 0.07). Threshold 0.3 is chosen (0.4 gains nothing and rejects more real questions).

B4.2 Recognition errors: WER 0–30% and the threshold sweep (1 mark)

To simulate a speech front end, every word of every test utterance is substituted, deleted or inserted with probability equal to the word error rate (WER); 20 random seeds per WER (v1, threshold 0.3).

WER	Accuracy (mean ± sd)	OOS accepted (false-accept rate)	In-scope rejected
0.0	0.757 ± 0.000	0.55	0.02
0.1	0.662 ± 0.033	0.66	0.01
0.2	0.578 ± 0.046	0.76	0.01
0.3	0.505 ± 0.046	0.81	0.03
0.4	0.459 ± 0.060	0.86	0.03

Interpretation. Accuracy falls by about 0.09 per 10 points of WER and is down to 0.505 at WER 0.3, close to a coin flip. The false-accept rate rises (0.55 → 0.81) because garbled words that no longer match any term give the bot a few partial matches on common words, which the low threshold accepts. Threshold sweep on clean text (v1): raising the threshold to 0.6 cuts OOS acceptance to 0.05 but rejects 0.52 of real questions, so no threshold gives both; the cost is in the stop-word cleaning (v2), not in the threshold. A voice deployment would therefore need the v2 vocabulary, a confirmation question when the cosine is between 0.3 and 0.5, and a measured WER of the real microphone, which I do not have.

B4.3 Edge or cloud: the production decision (2 marks)

Decision: deploy the detector (edge cascade) and the chatbot on the edge device; keep a cloud service only for retraining and for the full model as a second opinion on flows that the edge model marks as uncertain.

Criterion	Own evidence	Edge	Cloud
Latency (50 ms p95 per flow)	edge model p95 0.108 ms (this machine); chatbot 0.052 ms per utterance	meets the budget with a factor of hundreds to spare	a round trip to a cloud service is **assumed** 40–100 ms; this alone exceeds a 50 ms budget before any computation
Memory	edge model 21.5 KiB; full forest 3.6 MiB (8 MiB pickled)	fits in 512 KB	no limit

Criterion	Own evidence	Edge	Cloud
Accuracy	F1 0.945 against 0.968; AUC 0.950 against 0.999; unseen attack 0.0 against 0.366	lower; blind to a new attack until retrained and **retraining the cascade did not help**	better
Availability	not measured; edge works when the uplink is down	works offline	depends on the link, which an attack may disturb
Privacy / data volume	flows would have to leave the site	stay local	all flows are sent out
Cost	**assumed**, not measured: per-request cloud fees against a one-off device cost	one-off	per request

Defence. The deciding constraint is the *latency budget plus the loss of the link under attack*: a detector that waits for the cloud is useless when the uplink is saturated by the very DoS it should detect. The accuracy gap (0.023 F1) is real but can be reduced by sending only the uncertain flows (the 10–12% that do not exit at stage 1) to the cloud model.

> **What would reverse the decision**
>
> **1.** The cloud round trip is measured below about 20 ms and is reliable (then the latency argument disappears and the better model wins).
>
> **2.** The attack mix drifts faster than devices can be updated, i.e. new attack types appear weekly: the cascade's structural blindness (0.0 on slow_dos even after retraining) would then be unacceptable; use the cloud model with a monitoring-driven retraining loop.
>
> **3.** The device memory is raised so that a model of at least a few MB fits: then the full forest meets the budget on the edge (22× tolerance) and no compression is needed.
>
> **4.** The cost of a missed attack is judged much greater than the cost of a false alarm: lower the edge threshold or move the decision to the cloud.

Assumptions that are not evidence: the 512 KB memory and the 50 ms budget come from the assignment/assumed device; the cloud round-trip and the cost figures are assumptions of mine and were not measured; the latency numbers are from this container, not from the device; the simulator is not real traffic. A real deployment would first replace each by a measurement.

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021 (agents and environments; expert systems; search).

[2] M. Tavallaee, E. Bagheri, W. Lu and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009 (feature schema used by the flow simulator).

[3] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.

[4] M. Zaharia et al., “Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing,” *Proc. USENIX NSDI*, 2012.

[5] P. E. Hart, N. J. Nilsson and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, pp. 100–107, 1968 (A*).

[6] M. Dorigo, E. Bonabeau and G. Theraulaz, “Ant algorithms and stigmergy,” *Future Generation Computer Systems*, vol. 16, pp. 851–871, 2000 (pheromone coordination).

[7] B. G. Buchanan and E. H. Shortliffe, *Rule-Based Expert Systems: The MYCIN Experiments*, Addison-Wesley, 1984 (explanation and knowledge acquisition).

[8] F. Pedregosa et al., “Scikit-learn: machine learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011 (forest training and metrics only).

Appendix A – Source Code

All sources, the raw result files (`t_b*_results.json`) and the figures are in the accompanying zip file. Every experiment was run with `PYTHONHASHSEED=0` and fixed seeds.

A.1 A* planner, hybrid drone agent and mission simulator (B1.1, B1.2) (drone.py)

```
"""Autonomous parcel-delivery drone: hybrid (reactive + deliberative) agent and grid mission simulator.
Agent logic (DroneAgent) is separated from the world (MissionEnv): the agent only sees percepts."""
import heapq, random
MOVES = {'N': (-1, 0), 'E': (0, 1), 'S': (1, 0), 'W': (0, -1)}
ORDER = ['N', 'E', 'S', 'W']                                  # fixed neighbour order = deterministic tie-break
def manhattan(a, b): return abs(a[0] - b[0]) + abs(a[1] - b[1])
def astar(start, goal, rows, cols, blocked):
    """shortest path as a list of cells (excluding start); None if unreachable.  ties: smaller g first, then expansion order N,E,S,W"""
    if start == goal: return []
    if goal in blocked: return None
    pq = [(manhattan(start, goal), 0, 0, start)]; came = {start: None}; g = {start: 0}; cnt = 0
    while pq:
        _, gc, _, cur = heapq.heappop(pq)
        if cur == goal: break
        if gc > g[cur]: continue
        for d in ORDER:
            nb = (cur[0] + MOVES[d][0], cur[1] + MOVES[d][1])
            if not (0 <= nb[0] < rows and 0 <= nb[1] < cols) or nb in blocked: continue
            if nb not in g or gc + 1 < g[nb]:
                g[nb] = gc + 1; came[nb] = cur; cnt += 1; heapq.heappush(pq, (gc + 1 + manhattan(nb, goal), gc + 1, cnt, nb))
    if goal not in came: return None
    path = []; c = goal
    while c != start: path.append(c); c = came[c]
    return path[::-1]
class DroneAgent:
    """Layers: L0 reactive safety (veto moves into sensed obstacles/no-fly cells, battery-return rule, landing),
    L1 deliberative planning (A* over the known map to goal or home).  Arbitration: L0 > L1; inside L0 safety > battery > mission."""
    def __init__(self, home, goal, rows, cols, nofly, reserve=3, adaptive=False):
        self.home, self.goal, self.rows, self.cols = home, goal, rows, cols
        self.nofly = set(nofly); self.known_obst = set(); self.reserve = reserve
        self.mode = 'MISSION'; self.path = []; self.delivered = False; self.replans = 0; self.vetoes = 0
        self.adaptive = adaptive; self.expected = None; self.moves = 0; self.drift_hits = 0     # wind estimate (state)
    def blocked(self): return self.nofly | self.known_obst
    def _plan(self, target):
        self.path = astar(self.pos, target, self.rows, self.cols, self.blocked()); self.replans += 1
        return self.path
    def decide(self, pct):
        """pct: dict(pos, battery, t, sensed) where sensed = set of neighbour cells that currently hold an obstacle.  returns (action,
layer, note)"""
        self.pos = pct['pos']; bat = pct['battery']
        if self.expected is not None:                                  # state update: did the last move end where it was intended?
            self.moves += 1; self.drift_hits += int(self.pos != self.expected)
        self.expected = None
        # ---- state update
        newly = pct['sensed'] - self.known_obst; self.known_obst |= pct['sensed']
        if self.mode == 'DONE': return 'Hold', 'L0', 'mission finished'
        # ---- L0a: landing at home in RETURN mode
        if self.mode == 'RETURN' and self.pos == self.home: self.mode = 'DONE'; return 'Land', 'L0', 'landed at home'
        # ---- L0b: battery rule (safety margin): can I still get home with the reserve?
        hp = astar(self.pos, self.home, self.rows, self.cols, self.blocked())
        dist_home = len(hp) if hp is not None else 10 ** 6
        note = ''
        p_est = self.drift_hits / (self.moves + 5) if self.adaptive else 0.0
        need = 1.6 * dist_home * (1 + 2 * p_est) / (1 - p_est) if dist_home < 10 ** 6 else dist_home       # expected moves per cell under
drift probability p: (1 + 2p)/(1 - p), +60 % safety
        if self.mode == 'MISSION' and bat <= need + self.reserve:
            self.mode = 'RETURN'; self.path = []; note = f'battery {bat} <= distance home {dist_home} + reserve {self.reserve}: RETURN'
            layer = 'L0'
        else: layer = None
        if self.mode == 'RETURN' and self.pos == self.home: self.mode = 'DONE'; return 'Land', 'L0', note + ' landed at home'
        # ---- deliver the parcel
        if self.mode == 'MISSION' and self.pos == self.goal:
            self.delivered = True; self.mode = 'RETURN'; self.path = []; return 'Drop', 'L1', 'parcel delivered; start return'
        # ---- L0c: veto a planned step into a newly sensed obstacle
        if self.path and (self.path[0] in self.blocked()):
            self.vetoes += 1; self.path = []; note += ' obstacle ahead: step vetoed, replan'; layer = 'L0'
        if self.path and manhattan(self.pos, self.path[0]) != 1:                # wind drift moved the drone off the plan: replan from the
real position
            self.path = []; note += ' off plan (drift): replan'
        # ---- L1: plan
        if not self.path:
            target = self.goal if self.mode == 'MISSION' else self.home
            if self._plan(target) is None:
                if self.mode == 'MISSION':
                    self.mode = 'RETURN'; note += ' goal unreachable: RETURN'; layer = 'L0'
                    if self._plan(self.home) is None: return 'Hold', 'L0', note + ' no path home: hold'
                else: return 'Hold', 'L0', note + ' no path home: hold'
            else: layer = layer or 'L1'
        nxt = self.path.pop(0)
        for d, (dr, dc) in MOVES.items():
            if (self.pos[0] + dr, self.pos[1] + dc) == nxt: self.expected = nxt; return d, layer or 'L1', note.strip() or f'follow plan,
{len(self.path)} cells left'
        return 'Hold', 'L0', 'inconsistent plan: hold'
class MissionEnv:
    def __init__(self, rows, cols, home, goal, nofly, static_obst=(), sudden=(), battery=40, wind=0.0, seed=0, max_t=200):
        self.rows, self.cols, self.home, self.goal = rows, cols, home, goal
        self.nofly = set(nofly); self.obst = set(static_obst); self.sudden = dict(sudden)   # {t: cell}
        self.battery = battery; self.wind = wind; self.rng = random.Random(seed); self.pos = home; self.t = 0; self.max_t = max_t
        self.incidents = 0; self.status = 'FLYING'
    def percept(self):
        s = set()
        for dr, dc in MOVES.values():
            c = (self.pos[0] + dr, self.pos[1] + dc)
            if c in self.obst: s.add(c)
```

```
        return dict(pos=self.pos, battery=self.battery, t=self.t, sensed=s)
    def step(self, a):
        self.t += 1
        if a == 'Land':
            self.status = 'LANDED_HOME' if self.pos == self.home else 'FORCED_LANDING'; return
        if a in ('Hold', 'Drop'): self.battery -= 1
        else:
            d = a
            if self.wind > 0 and self.rng.random() < self.wind: d = self.rng.choice([x for x in ORDER if x != a])       # wind drift
            c = (self.pos[0] + MOVES[d][0], self.pos[1] + MOVES[d][1]); self.battery -= 1
            if not (0 <= c[0] < self.rows and 0 <= c[1] < self.cols): self.incidents += 1
            elif c in self.obst or c in self.nofly: self.incidents += 1
            else: self.pos = c
        if self.t in self.sudden: self.obst.add(self.sudden[self.t])           # a sudden obstacle appears at the END of step t (visible
in the next percept)
        if self.battery <= 0 and self.status == 'FLYING': self.status = 'BATTERY_EXHAUSTED' if self.pos != self.home else 'LANDED_HOME'
        if self.t >= self.max_t and self.status == 'FLYING': self.status = 'TIMEOUT'
def run_mission(env, agent):
    log = []
    while env.status == 'FLYING':
        # the sudden obstacle of step t+1 is applied inside env.step; the percept shows the world as of now
        pct = env.percept(); a, layer, note = agent.decide(pct)
        log.append(dict(t=env.t, pos=pct['pos'], battery=pct['battery'], mode=agent.mode, sensed=sorted(pct['sensed']), layer=layer,
action=a, note=note))
        env.step(a)
        if env.sudden and env.t in env.sudden: pass
    outcome = dict(status=env.status, delivered=agent.delivered, steps=env.t, battery_left=env.battery, incidents=env.incidents,
replans=agent.replans, vetoes=agent.vetoes)
    return log, outcome
```

A.2 Swarm rule, central controller, field simulator (B1.3, B1.4) (swarm.py)

```
"""Flood search-and-rescue grid: decentralised pheromone/frontier swarm rule versus a central controller.
Agent rule (swarm_decide) sees only: its position, the pheromone marks of cells within sensing radius RS, and drones in the 4 neighbouring
cells."""
import random, math, time
from collections import deque
DIRS = [(-1, 0), (0, 1), (1, 0), (0, -1)]                       # U R D L - fixed tie-break order
class Field:
    def __init__(self, size, nofly_frac, seed):
        r = random.Random(seed); self.n = size
        self.nofly = {(i, j) for i in range(size) for j in range(size) if r.random() < nofly_frac and (i, j) != (0, 0)}
        self.free = size * size - len(self.nofly); self.mark = {}                   # pheromone: number of visits per cell
    def inside(self, c): return 0 <= c[0] < self.n and 0 <= c[1] < self.n and c not in self.nofly
def swarm_decide(pos, field, occupied, rng, RS=4, deterministic=False):
    """per-agent local rule: (1) neighbours = free, unoccupied 4-neighbours; (2) go to the one with the lowest pheromone mark (unvisited =
0);
    (3) if every neighbour is marked, BFS inside radius RS for the nearest unmarked cell and take its first step; (4) otherwise take the
least-marked neighbour."""
    cand = []
    for d in DIRS:
        c = (pos[0] + d[0], pos[1] + d[1])
        if field.inside(c) and c not in occupied: cand.append(c)
    if not cand: return pos
    best = min(field.mark.get(c, 0) for c in cand)
    if best == 0:
        opts = [c for c in cand if field.mark.get(c, 0) == 0]; return opts[0] if deterministic else rng.choice(opts)
    # frontier search within radius RS (marks of cells within RS are readable)
    seen = {pos: None}; q = deque([(pos, 0)]); found = None
    while q:
        cur, dist = q.popleft()
        if dist >= RS: continue
        for d in DIRS:
            c = (cur[0] + d[0], cur[1] + d[1])
            if c in seen or not field.inside(c): continue
            seen[c] = cur
            if field.mark.get(c, 0) == 0: found = c; break
            q.append((c, dist + 1))
        if found: break
    if found:
        c = found
        while seen[c] != pos: c = seen[c]
        if c in cand: return c
    opts = [c for c in cand if field.mark.get(c, 0) == best]; return opts[0] if deterministic else rng.choice(opts)
def central_targets(field, positions, alive, visited_unseen):
    """global controller: nearest not-yet-visited cell (Manhattan) per live drone, never the same cell twice"""
    taken = set(); tg = {}
    free_cells = [c for c in visited_unseen]
    for i in sorted(range(len(positions)), key=lambda k: k):
        if not alive[i]: continue
        p = positions[i]; best = None; bd = 10 ** 9
        for c in free_cells:
            if c in taken: continue
            d = abs(c[0] - p[0]) + abs(c[1] - p[1])
            if d < bd: bd = d; best = c
        if best is not None: taken.add(best); tg[i] = best
    return tg
def run(mode, N, seed, fail_frac=0.0, size=30, nofly=0.05, battery=120, T=150, link_loss=0.0, controller_fail_t=None):
    rng = random.Random(seed); f = Field(size, nofly, seed); n_free = f.free
    pos = [(0, 0)] * N; bat = [battery] * N; alive = [True] * N; moves = 0; redundant = 0
    fail_t = {}
    k = math.ceil(fail_frac * N) if fail_frac > 0 else 0
    for i in rng.sample(range(N), k): fail_t[i] = rng.randint(10, 60)
    f.mark[(0, 0)] = 1; visited = {(0, 0)}; cover = []; lat = []
    unseen = {(i, j) for i in range(size) for j in range(size) if (i, j) not in f.nofly} - visited; targets = {}
    for t in range(T):
        for i, ft in fail_t.items():
            if ft == t: alive[i] = False
        occ = set(p for i, p in enumerate(pos) if alive[i])
        if mode == 'central' and (controller_fail_t is None or t < controller_fail_t): targets = central_targets(f, pos, alive, unseen) if
t % 1 == 0 else targets
        order = list(range(N)); rng.shuffle(order)
        for i in order:
            if not alive[i] or bat[i] <= 0: continue
            occ.discard(pos[i]); t0 = time.perf_counter()
            if mode == 'swarm': new = swarm_decide(pos[i], f, occ, rng)
```

```
            else:
                new = pos[i]
                if controller_fail_t is not None and t >= controller_fail_t: new = pos[i]                    # no commands any more:
drone hovers
                elif rng.random() < link_loss: new = pos[i]                                                    # command lost
                elif i in targets:
                    tg = targets[i]; cands = []
                    for d in DIRS:
                        c = (pos[i][0] + d[0], pos[i][1] + d[1])
                        if f.inside(c) and c not in occ: cands.append((abs(c[0] - tg[0]) + abs(c[1] - tg[1]), c))
                    if cands: new = min(cands)[1]
            lat.append(time.perf_counter() - t0)
            if new != pos[i]:
                bat[i] -= 1; moves += 1
                if new in visited: redundant += 1
                else: visited.add(new); unseen.discard(new)
                f.mark[new] = f.mark.get(new, 0) + 1
            pos[i] = new; occ.add(new)
        cover.append(len(visited) / n_free)
    return dict(coverage=cover[-1], curve=cover, moves=moves, redundancy=redundant / max(1, moves), lat_us=1e6 * sum(lat) / max(1,
len(lat)), battery_used=moves / N, alive=sum(alive))
```

A.3 Generic expert-system shell (B2) (es.py)

```
"""Generic expert-system shell: knowledge base (rules as data), working memory (timestamped facts), forward-chaining inference engine with
pluggable conflict resolution, HOW / WHY explanation and run-time knowledge acquisition.  No domain knowledge is in the engine."""
import json, copy
class Rule:
    def __init__(self, rid, conds, concl, kind='diagnosis', priority=1, text=''):
        self.rid, self.conds, self.concl, self.kind, self.priority, self.text = rid, [tuple(c) for c in conds], concl, kind, priority, text
    def to_dict(self): return dict(rid=self.rid, conds=[list(c) for c in self.conds], concl=self.concl, kind=self.kind,
priority=self.priority, text=self.text)
class KnowledgeBase:
    def __init__(self, rules=None): self.rules = list(rules or [])
    @staticmethod
    def from_json(s): return KnowledgeBase([Rule(**d) for d in json.loads(s)])
    def to_json(self): return json.dumps([r.to_dict() for r in self.rules], indent=1)
    def add(self, rule): self.rules.append(rule)                                  # knowledge acquisition: add
    def edit(self, rid, **kw):                                                    # knowledge acquisition: edit
        for r in self.rules:
            if r.rid == rid:
                for k, v in kw.items(): setattr(r, k, [tuple(c) for c in v] if k == 'conds' else v)
                return
        raise KeyError(rid)
    def remove(self, rid): self.rules = [r for r in self.rules if r.rid != rid]
class WorkingMemory:
    def __init__(self, observations):
        self.facts = {}; self.time = {}; self.why = {}                        # attribute -> value ; insertion time ; justification
        for i, (a, v) in enumerate(observations.items(), 1): self.facts[a] = v; self.time[a] = i; self.why[a] = 'observed'
        self.clock = len(observations)
    def holds(self, c):
        a, op, v = c
        if op == '=': return self.facts.get(a) == v
        if op == '!=': return a in self.facts and self.facts[a] != v
        if op == 'in': return self.facts.get(a) in v
        raise ValueError(op)
    def assert_fact(self, a, v, rid): self.clock += 1; self.facts[a] = v; self.time[a] = self.clock; self.why[a] = rid
STRATEGIES = ('specificity', 'priority', 'recency', 'order')
def resolve(conflict, wm, strategy, kb_order):
    """pick one rule from the conflict set; ties are broken by position in the knowledge base (documented so results are reproducible)"""
    def key(r):
        pos = kb_order[r.rid]
        if strategy == 'specificity': return (-len(r.conds), pos)
        if strategy == 'priority': return (-r.priority, pos)
        if strategy == 'recency': return (-max(wm.time[c[0]] for c in r.conds), pos)
        if strategy == 'order': return (pos,)
    return min(conflict, key=key)
def diagnose(kb, observations, strategy='specificity'):
    wm = WorkingMemory(observations); order = {r.rid: i for i, r in enumerate(kb.rules)}; fired = []; trace = []
    # phase 1: derived-fact rules fire to closure (monotone), phase 2: the diagnosis conflict set is resolved by the strategy
    changed = True
    while changed:
        changed = False
        for r in kb.rules:
            if r.kind == 'derive' and r.rid not in fired and all(wm.holds(c) for c in r.conds):
                wm.assert_fact(r.concl[0], r.concl[1], r.rid); fired.append(r.rid); changed = True; trace.append(f"{r.rid} derives
{r.concl[0]} = {r.concl[1]}")
    conflict = [r for r in kb.rules if r.kind == 'diagnosis' and all(wm.holds(c) for c in r.conds)]
    if not conflict: return dict(diagnosis=None, fired=fired, conflict=[], trace=trace, wm=wm, rule=None)
    win = resolve(conflict, wm, strategy, order); fired.append(win.rid)
    return dict(diagnosis=win.concl, fired=fired, conflict=[r.rid for r in conflict], trace=trace, wm=wm, rule=win)
def how(kb, res):
    """HOW explanation: the chain of rules and the facts they used"""
    if res['diagnosis'] is None: return "HOW: no rule matched the observations, so no diagnosis was made."
    r = res['rule']; lines = [f"HOW: diagnosis '{r.concl}' was concluded by rule {r.rid} ({r.text})."]
    for c in r.conds:
        a = c[0]; lines.append(f"   because {a} {c[1]} {c[2]}  [{res['wm'].why[a]}]")
        if res['wm'].why[a] != 'observed':
            d = next(x for x in kb.rules if x.rid == res['wm'].why[a]); lines.append(f"      which came from {d.rid}: " + ' and
'.join(f"{x[0]} {x[1]} {x[2]}" for x in d.conds))
    others = [x for x in res['conflict'] if x != r.rid]
    if others: lines.append(f"   other rules that also matched and lost the conflict resolution: {others}")
    return '\n'.join(lines)
def why_ask(kb, observations):
    """WHY explanation: the question the shell would ask next and why: the missing attribute that appears in the largest number of rules
that are still possible"""
    wm = WorkingMemory(observations); best = None; derived = {r.concl[0] for r in kb.rules if r.kind == 'derive'}
    for r in kb.rules:
        if r.kind != 'diagnosis' or any(c[0] in derived for c in r.conds): continue          # only ask about directly observable
attributes
        if any(wm.facts.get(c[0]) is not None and not wm.holds(c) for c in r.conds): continue          # already contradicted
        if not any(wm.holds(c) for c in r.conds): continue                                # irrelevant rule: nothing supports it yet
        missing = [c for c in r.conds if c[0] not in wm.facts]
        if not missing or len(missing) == len(r.conds) and len(r.conds) > 2: continue
        cand = (len(missing), -r.priority, r.rid, missing[0][0], r)
        if best is None or cand[:3] < best[:3]: best = cand
```

```
    if best is None: return "WHY: no further question would help."
    r = best[4]; return f"WHY ask '{best[3]}'? Because rule {r.rid} ({r.text}) could conclude '{r.concl}' and it needs {best[3]} = {[c[2]
for c in r.conds if c[0] == best[3]][0]}; the other conditions are already satisfied or unknown."
```

A.4 Pump knowledge base and 30 labelled cases (B2) (pump_kb.py)

```
from es import Rule, KnowledgeBase
FAULTS = ['cavitation', 'bearing_wear', 'seal_leak', 'clogged_impeller', 'motor_overload', 'misalignment']
PRI = {'motor_overload': 5, 'seal_leak': 4, 'bearing_wear': 3, 'cavitation': 3, 'clogged_impeller': 2, 'misalignment': 1}
def D(rid, conds, f, text): return Rule(rid, conds, f, 'diagnosis', PRI[f], text)
def build_kb():
    return KnowledgeBase([
        Rule('D1', [('flow', 'in', ['low', 'zero'])], ('flow_deficit', 'yes'), 'derive', 0, 'low or zero flow is a flow deficit'),
        Rule('D2', [('motor_temp', '=', 'high'), ('current', '=', 'high')], ('electrical_stress', 'yes'), 'derive', 0, 'hot motor with high
current is electrical stress'),
        D('R1', [('noise', '=', 'rattling'), ('suction', '=', 'low')], 'cavitation', 'rattling noise with low suction pressure'),
        D('R2', [('noise', '=', 'rattling'), ('pressure', '=', 'low'), ('vibration', '=', 'high')], 'cavitation', 'rattling, low discharge
pressure and high vibration'),
        D('R3', [('noise', '=', 'rattling'), ('vibration', '=', 'high')], 'cavitation', 'rattling noise with high vibration'),
        D('R4', [('noise', '=', 'grinding'), ('bearing_temp', '=', 'high')], 'bearing_wear', 'grinding noise with hot bearing'),
        D('R5', [('noise', '=', 'grinding'), ('vibration', '=', 'high')], 'bearing_wear', 'grinding noise with high vibration'),
        D('R6', [('bearing_temp', '=', 'high'), ('vibration', '=', 'high')], 'bearing_wear', 'hot bearing with high vibration'),
        D('R7', [('leak', '=', 'visible')], 'seal_leak', 'visible leakage'),
        D('R8', [('leak', '=', 'visible'), ('pressure', '=', 'low')], 'seal_leak', 'visible leakage with low pressure'),
        D('R9', [('flow_deficit', '=', 'yes'), ('pressure', '=', 'high')], 'clogged_impeller', 'flow deficit with high discharge
pressure'),
        D('R10', [('flow_deficit', '=', 'yes'), ('current', '=', 'high'), ('pressure', '=', 'high')], 'clogged_impeller', 'flow deficit,
high current, high pressure'),
        D('R11', [('electrical_stress', '=', 'yes')], 'motor_overload', 'electrical stress'),
        D('R12', [('motor_temp', '=', 'high'), ('current', '=', 'high'), ('flow', '=', 'normal')], 'motor_overload', 'hot motor, high
current, normal flow'),
        D('R13', [('axial_vib', '=', 'high'), ('vibration', '=', 'high')], 'misalignment', 'high axial and radial vibration'),
        D('R14', [('axial_vib', '=', 'high'), ('bearing_temp', '=', 'normal')], 'misalignment', 'high axial vibration with cool bearing')])
# 30 labelled cases written by the student as a stand-in for expert-labelled data (cases D* are used to write the rules, H* are the hold-
out)
DEV = [
 ('D1', dict(vibration='high', noise='rattling', pressure='low', suction='low', flow='low'), 'cavitation'), ('D2', dict(vibration='high',
noise='rattling', suction='low', flow='normal', pressure='normal'), 'cavitation'),
 ('D3', dict(noise='rattling', pressure='low', suction='low', vibration='normal'), 'cavitation'),
 ('D4', dict(noise='grinding', vibration='high', bearing_temp='high', flow='normal'), 'bearing_wear'), ('D5', dict(noise='grinding',
bearing_temp='high', vibration='normal', current='normal'), 'bearing_wear'),
 ('D6', dict(vibration='high', bearing_temp='high', noise='normal', flow='normal'), 'bearing_wear'),
 ('D7', dict(leak='visible', pressure='low', vibration='normal', noise='normal'), 'seal_leak'), ('D8', dict(leak='visible', flow='normal',
pressure='normal', motor_temp='normal'), 'seal_leak'),
 ('D9', dict(flow='low', pressure='high', current='high', noise='normal'), 'clogged_impeller'), ('D10', dict(flow='zero', pressure='high',
current='normal', vibration='normal'), 'clogged_impeller'),
 ('D11', dict(flow='low', current='high', suction='normal', vibration='high', pressure='high'), 'clogged_impeller'),
 ('D12', dict(motor_temp='high', current='high', flow='normal', noise='hissing'), 'motor_overload'), ('D13', dict(motor_temp='high',
current='high', flow='low', vibration='normal'), 'motor_overload'),
 ('D14', dict(vibration='high', axial_vib='high', bearing_temp='normal', noise='normal'), 'misalignment'), ('D15', dict(axial_vib='high',
vibration='high', noise='normal', bearing_temp='normal', flow='normal'), 'misalignment')]
HOLD = [
 ('H1', dict(vibration='high', noise='rattling', suction='low', pressure='low', flow='low'), 'cavitation'), ('H2', dict(noise='rattling',
suction='low', flow='low', vibration='normal'), 'cavitation'),
 ('H3', dict(noise='grinding', vibration='high', bearing_temp='high', flow='normal'), 'bearing_wear'), ('H4', dict(bearing_temp='high',
vibration='high', noise='grinding', current='high'), 'bearing_wear'),
 ('H5', dict(noise='grinding', vibration='high', bearing_temp='normal'), 'bearing_wear'),
 ('H6', dict(leak='visible', pressure='low', flow='low'), 'seal_leak'), ('H7', dict(leak='visible', vibration='high', noise='normal'),
'seal_leak'),
 ('H8', dict(flow='low', pressure='high', current='high', noise='rattling'), 'clogged_impeller'), ('H9', dict(flow='zero', pressure='high',
current='high'), 'clogged_impeller'),
 ('H10', dict(motor_temp='high', current='high', flow='normal', vibration='normal'), 'motor_overload'), ('H11', dict(current='high',
motor_temp='high', flow='low', pressure='normal', bearing_temp='high'), 'motor_overload'),
 ('H12', dict(motor_temp='high', current='normal', flow='normal'), 'motor_overload'),
 ('H13', dict(vibration='high', axial_vib='high', bearing_temp='normal', noise='normal', flow='normal'), 'misalignment'), ('H14',
dict(axial_vib='high', vibration='high', noise='grinding'), 'misalignment'),
 ('H15', dict(vibration='high', noise='rattling', pressure='low', suction='normal'), 'cavitation')]
```

A.5 Flow simulator, full forest, cascade, flat-array engine (B3) (ids.py)

```
"""Edge network-intrusion detection: flow simulator (KDD-style features), full forest, compressed cascade, flat-array inference engine"""
import numpy as np, time
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
FEATS = ['duration', 'src_bytes', 'dst_bytes', 'count', 'srv_count', 'serror_rate', 'rerror_rate', 'same_srv_rate', 'diff_srv_rate',
'failed_logins', 'logged_in', 'num_compromised']
TYPES = ['normal', 'burst_normal', 'dos', 'probe', 'r2l', 'u2r', 'slow_dos']
def gen_stream(n, seed):
    """time-ordered flows.  normal traffic drifts (higher counts) and flash-crowd bursts become more common over time; slow_dos exists only
in the last 30 % of the stream"""
    r = np.random.RandomState(seed); t = np.sort(r.rand(n)); y = np.zeros(n, int); typ = []
    for i in range(n):
        u = r.rand(); late = t[i] > 0.7
        if u < 0.70: k = 'burst_normal' if r.rand() < (0.02 + 0.10 * t[i]) else 'normal'
        elif u < 0.82: k = 'dos'
        elif u < 0.90: k = 'probe'
        elif u < 0.96: k = 'r2l'
        elif u < 0.98: k = 'u2r'
        else: k = 'slow_dos' if late else 'dos'
        typ.append(k)
    typ = np.array(typ); X = np.zeros((n, 12)); c = lambda x: np.clip(x, 0, 1)
    for k in TYPES:
        m = typ == k; s = m.sum()
        if not s: continue
        tt = t[m]; g = r.normal
        if k == 'normal': X[m] = np.stack([r.lognormal(0.5, 1, s), g(6, 1.5, s), g(7, 1.5, s), r.poisson(8 + 6 * tt), r.poisson(6, s),
c(g(.03, .05, s)), c(g(.03, .05, s)), c(g(.85, .15, s)), c(g(.08, .08, s)), r.poisson(.1, s), r.binomial(1, .7, s), r.poisson(.01, s)], 1)
        if k == 'burst_normal': X[m] = np.stack([r.lognormal(0.2, 0.8, s), g(5, 1.2, s), g(6, 1.5, s), r.poisson(70, s), r.poisson(65, s),
c(g(.06, .06, s)), c(g(.03, .05, s)), c(g(.9, .08, s)), c(g(.05, .05, s)), r.poisson(.05, s), r.binomial(1, .8, s), np.zeros(s)], 1)
        if k == 'dos': X[m] = np.stack([r.lognormal(-2, .5, s), g(2, 1, s), g(1, 1, s), r.poisson(180, s), r.poisson(170, s), c(g(.85, .15,
s)), c(g(.03, .05, s)), c(g(.95, .05, s)), c(g(.03, .04, s)), np.zeros(s), np.zeros(s), np.zeros(s)], 1)
```

```
        if k == 'probe': X[m] = np.stack([r.lognormal(-1, .5, s), g(3, 1, s), g(2, 1, s), r.poisson(40, s), r.poisson(5, s), c(g(.1, .1,
s)), c(g(.4, .2, s)), c(g(.3, .2, s)), c(g(.6, .2, s)), np.zeros(s), np.zeros(s), np.zeros(s)], 1)
        if k == 'r2l': X[m] = np.stack([r.lognormal(1.5, 1, s), g(5, 1.5, s), g(5, 1.5, s), r.poisson(6, s), r.poisson(5, s), c(g(.05, .06,
s)), c(g(.08, .08, s)), c(g(.85, .15, s)), c(g(.1, .1, s)), r.poisson(3, s), r.binomial(1, .3, s), r.poisson(.3, s)], 1)
        if k == 'u2r': X[m] = np.stack([r.lognormal(1, 1, s), g(6.5, 1.5, s), g(6, 1.5, s), r.poisson(5, s), r.poisson(4, s), c(g(.04, .05,
s)), c(g(.04, .05, s)), c(g(.85, .15, s)), c(g(.08, .08, s)), r.poisson(.3, s), np.ones(s), r.poisson(3, s)], 1)
        if k == 'slow_dos': X[m] = np.stack([r.lognormal(3, .5, s), g(4, 1, s), g(3, 1, s), r.poisson(9, s), r.poisson(8, s), c(g(.25, .15,
s)), c(g(.05, .06, s)), c(g(.9, .1, s)), c(g(.05, .05, s)), np.zeros(s), r.binomial(1, .5, s), np.zeros(s)], 1)
    y = (typ != 'normal') & (typ != 'burst_normal'); return X, y.astype(int), typ, t
def split(X, y, typ, t): a = int(.6 * len(t)); b = int(.7 * len(t)); return (X[:a], y[:a], typ[:a]), (X[a:b], y[a:b], typ[a:b]), (X[b:],
y[b:], typ[b:])
class Flat:
    """a forest flattened into arrays (feature, threshold, left, right, value) - what an edge device would store; optional 8-bit
quantisation"""
    def __init__(self, trees, quant=None):
        self.F, self.T, self.L, self.R, self.V, self.root = [], [], [], [], [], []; off = 0; self.quant = quant
        for tr in trees:
            t = tr.tree_; n = t.node_count; self.root.append(off)
            self.F.append(t.feature.copy()); thr = t.threshold.copy()
            if quant is not None:
                lo, sc = quant; f = t.feature.copy(); f[f < 0] = 0; thr = np.floor((thr - lo[f]) / sc[f]).clip(0, 255)
            self.T.append(thr); self.L.append(np.where(t.children_left < 0, -1, t.children_left + off));
self.R.append(np.where(t.children_right < 0, -1, t.children_right + off))
            v = t.value[:, 0, :]; self.V.append(v[:, 1] / v.sum(1)); off += n
        self.F = np.concatenate(self.F); self.T = np.concatenate(self.T); self.L = np.concatenate(self.L); self.R = np.concatenate(self.R);
self.V = np.concatenate(self.V); self.root = np.array(self.root); self.nodes = off
    def prep(self, X):
        if self.quant is None: return X
        lo, sc = self.quant; return np.floor((X - lo) / sc).clip(0, 255)
    def predict_proba(self, X):
        Xq = self.prep(X); n = len(X); tot = np.zeros(n)
        for r in self.root:
            node = np.full(n, r)
            while True:
                f = self.F[node]; leaf = self.L[node] < 0
                if leaf.all(): break
                go = Xq[np.arange(n), np.where(leaf, 0, f)] <= self.T[node]
                node = np.where(leaf, node, np.where(go, self.L[node], self.R[node]))
            tot += self.V[node]
        return tot / len(self.root)
    def predict_one(self, x):
        """pure-python single-flow inference; returns (probability, node visits)"""
        xq = self.prep(x[None])[0] if self.quant is not None else x; tot = 0.0; visits = 0
        F, T, L, R, V = self.F, self.T, self.L, self.R, self.V
        for r in self.root:
            nd = r
            while L[nd] >= 0:
                nd = L[nd] if xq[F[nd]] <= T[nd] else R[nd]; visits += 1
            tot += V[nd]; visits += 1
        return tot / len(self.root), visits
    def bytes(self, full): return self.nodes * (18 if full else 7)
class Cascade:
    """stage 1: depth-3 tree on 5 cheap features, exits when the leaf is >= 97 % pure; stage 2: small quantised forest"""
    S1F = [3, 5, 8, 9, 11]
    def __init__(self, s1, s2): self.s1, self.s2 = s1, s2; self.s1flat = Flat([s1]); self.exit = 0
    def predict_proba(self, X):
        Xs = np.zeros_like(X); Xs[:, self.S1F] = X[:, self.S1F]; p1 = self.s1flat.predict_proba(X)
        conf = (p1 <= 0.03) | (p1 >= 0.97); out = p1.copy(); idx = np.where(~conf)[0]
        if len(idx): out[idx] = self.s2.predict_proba(X[idx])
        self.exit = conf.mean(); return out
    def predict_one(self, x):
        p, v = self.s1flat.predict_one(x)
        if p <= 0.03 or p >= 0.97: return p, v, True
        p2, v2 = self.s2.predict_one(x); return p2, v + v2, False
def train_models(Xtr, ytr, seed=0):
    full = RandomForestClassifier(n_estimators=200, random_state=seed, n_jobs=1).fit(Xtr, ytr); ff = Flat(full.estimators_)
    s1 = DecisionTreeClassifier(max_depth=3, random_state=seed).fit(Xtr[:, Cascade.S1F], ytr)
    # the stage-1 tree is trained on 5 columns; remap its feature ids to the full 12-column layout
    s1.tree_.feature[s1.tree_.feature >= 0] = np.array(Cascade.S1F)[s1.tree_.feature[s1.tree_.feature >= 0]]
    small = RandomForestClassifier(n_estimators=15, max_depth=8, random_state=seed, n_jobs=1).fit(Xtr, ytr)
    lo = Xtr.min(0); sc = np.maximum((Xtr.max(0) - lo) / 255.0, 1e-9); sf = Flat(small.estimators_, quant=(lo, sc))
    return full, ff, Cascade(s1, sf), small, sf
```

A.6 Mini-RDD engine (B3.3) (minirdd.py)

```
"""MiniRDD: a small stand-in for the Spark RDD model written for this assignment because PySpark could not be installed in the sandbox.
Partitioned data (numpy arrays), lazy transformations with lineage, narrow vs wide (shuffle) dependencies, cache(), actions that run one
task per partition in a process pool.
It reproduces the *ideas* being measured (partitioning, shuffles, map-side combine, caching, lineage recomputation); it is NOT Spark and
its numbers must not be read as Spark numbers."""
import numpy as np, os, time, multiprocessing as mp
_SRC = {}                                   # source partitions, inherited by forked workers (copy-on-write)
class Stats:
    shuffles = 0; shuffle_bytes = 0; tasks = 0; stages = 1
def _task(args):
    rdd, i = args; return rdd._compute(i)
class MiniRDD:
    def __init__(self, parent=None, fn=None, kind='source', nparts=None, key=None):
        self.parent, self.fn, self.kind, self.key = parent, fn, kind, key; self.n = nparts if nparts is not None else parent.n;
self.cache_on = False; self.cache = {}
        self.computed = 0
    # -------- transformations (lazy; narrow dependencies)
    def map_partitions(self, fn): return MiniRDD(self, fn, 'map')
    def filter_rows(self, pred): return MiniRDD(self, lambda arrs: tuple(a[pred(*arrs)] for a in arrs), 'map')
    def cache_it(self): self.cache_on = True; return self
    def _compute(self, i):
        if i in self.cache: return self.cache[i]
        if self.kind == 'source': out = _SRC[self.key][i]
        else: out = self.fn(self.parent._compute(i))
        self.computed += 1
        if self.cache_on and mp.current_process().name == 'MainProcess': self.cache[i] = out
        return out
    # -------- actions
    def _run(self, workers, finish=None):
        Stats.tasks += self.n; res = None
        if workers <= 1 or self.n == 1: res = [_task((self, i)) if finish is None else finish(_task((self, i))) for i in range(self.n)]
```

```
        else:
            ctx = mp.get_context('fork'); global _FIN, _CUR; _FIN = finish; _CUR = self                    # forked workers inherit the
lineage and the cache: only the partition number is sent
            with ctx.Pool(workers) as p: res = p.map(_run_one, list(range(self.n)))
        return res
    def materialise(self, workers):
        """run the lineage once and keep every partition of THIS rdd in the driver (the cache)"""
        outs = self._run(workers)
        for i, o in enumerate(outs): self.cache[i] = o
        return self
    def count(self, workers=2): return sum(self._run(workers, finish=_len))
    def reduce_by_key_dense(self, workers, nkeys, key_idx=0, val_idx=1, combine=True):
        """wide dependency: per-partition (map-side) combiners [sum, count, max, sumsq] per key, then a shuffle (driver merge).
combine=False ships every record instead"""
        Stats.shuffles += 1; Stats.stages += 1
        def finish(arrs):
            k = arrs[key_idx].astype(np.int64); v = arrs[val_idx].astype(np.float64)
            if not combine: return (k, v)
            return np.stack([np.bincount(k, v, nkeys), np.bincount(k, minlength=nkeys).astype(float), _gmax(k, v, nkeys), np.bincount(k, v
* v, nkeys)], 1)
        parts = self._run(workers, finish=finish)
        if combine: Stats.shuffle_bytes += sum(p.nbytes for p in parts); tot = np.sum([p[:, :2] for p in parts], 0); sq = np.sum([p[:, 3]
for p in parts], 0); mx = np.max([p[:, 2] for p in parts], 0)
        else:
            Stats.shuffle_bytes += sum(k.nbytes + v.nbytes for k, v in parts); k = np.concatenate([p[0] for p in parts]); v =
np.concatenate([p[1] for p in parts])
            tot = np.stack([np.bincount(k, v, nkeys), np.bincount(k, minlength=nkeys).astype(float)], 1); sq = np.bincount(k, v * v,
nkeys); mx = np.full(nkeys, -np.inf); np.maximum.at(mx, k, v)
        return tot[:, 0], tot[:, 1], mx, sq
def _gmax(k, v, n):
    m = np.full(n, -np.inf); np.maximum.at(m, k, v); return m
def _len(arrs): return len(arrs[0])
_FIN = None
_CUR = None
def _run_one(i):
    out = _CUR._compute(i); return _FIN(out) if _FIN is not None else out
def parallelize(key, parts): _SRC[key] = parts; return MiniRDD(kind='source', nparts=len(parts), key=key)
```

A.7 Retrieval chatbot (B4) (chatbot.py)

```
"""Operator chatbot for the edge IDS: TF-IDF + cosine nearest-example intent classifier written from scratch, slot extraction, response
templates"""
import math, re, random, time
from collections import Counter
TRAIN = {
 'list_alerts': ["show me the open alerts", "list all current alerts", "what alerts are active", "any new alerts", "display unresolved
alerts", "give me the alert list", "which alerts are open right now", "show active intrusion alerts", "list the alerts", "are there any
alerts"],
 'alert_count': ["how many alerts today", "count the alerts", "number of alerts in the last hour", "how many attacks were detected", "total
alerts so far", "what is the alert count", "how many intrusions were flagged today", "give me the number of detections", "how many alarms",
"count today's detections"],
 'why_flagged': ["why was this flow flagged", "explain alert 12", "why did you raise this alert", "what made this connection suspicious",
"reason for alert 7", "explain the detection", "why is flow 40 an attack", "what triggered the alarm", "tell me why it was flagged",
"justify this alert"],
 'top_source': ["which address sends the most attacks", "top attacking source", "who is the biggest offender", "most active attacker",
"show the top source ip", "which host generated most alerts", "worst source address", "most frequent attacker", "rank the attackers",
"which ip is attacking most"],
 'model_status': ["is the model running", "model status", "is the detector healthy", "show the model version", "is the ids online", "check
detector health", "when was the model last updated", "which model is deployed", "is the detection engine up", "report model status"],
 'latency_info': ["what is the detection latency", "how fast is the detector", "how long does a decision take", "show inference time",
"average response time of the model", "is the latency within budget", "report the processing delay", "how many milliseconds per flow",
"what is the throughput", "how quick are the alerts"],
 'attack_info': ["what is a dos attack", "explain port scanning", "what does a probe attack mean", "describe an r2l attack", "tell me about
privilege escalation", "what is slow dos", "define denial of service", "what is a remote to local attack", "explain u2r", "how does a syn
flood work"],
 'ack_alert': ["acknowledge alert 5", "mark alert 9 as handled", "close alert 3", "resolve alert 21", "dismiss alert 14", "i have handled
alert 8", "set alert 2 to resolved", "clear alert 30", "ack alert 11", "please acknowledge the alert"],
 'help': ["help", "what can you do", "how do i use this", "list your commands", "what can i ask", "show the help menu", "i need help",
"what are the options", "how does this assistant work", "commands please"]}
TEST = {
 'list_alerts': ["could you show which alerts are currently open", "I want to see every active alert", "pull up the outstanding alerts",
"list open intrusion warnings", "anything flagged at the moment", "show alerts"],
 'alert_count': ["how many alerts have we had this morning", "tell me the total number of alarms", "what's the count of detections today",
"number of flagged flows please", "how many intrusions in the last hour", "give the alarm total"],
 'why_flagged': ["why did alert 19 fire", "what is the reason this flow was flagged", "explain why the model said attack", "how did you
decide alert 3 was malicious", "what evidence triggered this alarm", "why was it marked suspicious"],
 'top_source': ["who is attacking us the most", "name the most aggressive source", "top offender address please", "which source address is
the worst", "show highest volume attacker", "biggest attacking host"],
 'model_status': ["is the intrusion detector alive", "model health check please", "what version of the detector is running", "has the model
been retrained recently", "status of the detection model", "is the classifier working"],
 'latency_info': ["how long does the detector take per flow", "what is the inference delay", "is detection fast enough", "show me the
response time", "how many flows can it handle per second", "tell me the latency"],
 'attack_info': ["what does a denial of service attack do", "explain what a port scan is", "tell me about user to root attacks", "describe
remote to local attacks", "what is a probe", "can you explain low and slow attacks"],
 'ack_alert': ["please close alert 17", "mark alert 4 as resolved", "acknowledge alert number 25", "i dealt with alert 13", "resolve alert
6", "dismiss alert 22"],
 'help': ["what can i say to you", "show me what you can do", "give me the command list", "help me please", "how do I talk to you", "what
features do you have"]}
OOS = ["what is the weather today", "book me a flight to delhi", "tell me a joke", "who won the cricket match", "play some music", "what is
two plus two", "order a pizza", "translate hello into french", "what time is it in tokyo", "recommend a movie", "how do i bake a cake",
"set an alarm for seven", "what is the capital of france", "call my mother", "turn on the lights", "tell me a story", "who is the prime
minister", "reserve a table for two", "what is the price of gold", "sing a song"]
STOP = {'the', 'a', 'an', 'of', 'is', 'to', 'me', 'please', 'i', 'it', 'this', 'you', 'at', 'in', 'on', 'for', 'and'}
STOP2 = STOP | {'what', 'who', 'how', 'tell', 'do', 'does', 'can', 'my', 'are', 'there', 'was', 'be', 'give', 'about', 'we', 'have', 'our',
'could', 'want', 'will', 'which', 'when'}
STOP_ACTIVE = [STOP]
def tok(s): return [w for w in re.findall(r"[a-z0-9]+", s.lower()) if w not in STOP_ACTIVE[0]]
class IntentModel:
    def __init__(self, train=TRAIN):
        self.ex = [(tok(u), k) for k, us in train.items() for u in us]; df = Counter(w for t, _ in self.ex for w in set(t)); n =
len(self.ex)
        self.idf = {w: math.log((1 + n) / (1 + c)) + 1 for w, c in df.items()}; self.vec = [(self.v(t), k) for t, k in self.ex]
    def v(self, toks):
        c = Counter(w for w in toks if w in self.idf); x = {w: n * self.idf[w] for w, n in c.items()}; nr = math.sqrt(sum(a * a for a in
x.values())); return {w: a / nr for w, a in x.items()} if nr else {}
    def classify(self, u, thr=0.3):
```

```
        q = self.v(tok(u)); best = (0.0, None)
        for x, k in self.vec:
            s = sum(a * x.get(w, 0) for w, a in q.items())
            if s > best[0]: best = (s, k)
        return (best[1] if best[0] >= thr else 'out_of_scope'), best[0]
def slots(u):
    m = re.search(r"(?:alert|flow)\s*(?:number\s*)?(\d+)", u.lower()); return {'id': int(m.group(1))} if m else {}
def respond(intent, u, state):
    s = slots(u)
    if intent == 'ack_alert': return f"Alert {s['id']} marked as resolved." if 'id' in s else "Which alert number should I acknowledge?"
    if intent == 'why_flagged': return (f"Alert {s['id']}: flagged because count and serror_rate were far above the normal range." if 'id'
in s else "Which alert should I explain? Say, for example, 'explain alert 12'.")
    return {'list_alerts': 'You have 4 open alerts.', 'alert_count': '17 alerts today.', 'top_source': 'Top source: 10.0.0.23.',
'model_status': 'Model v3 is running.', 'latency_info': 'Mean latency 0.02 ms per flow.', 'attack_info': 'A DoS attack floods a service
with connections.', 'help': 'You can ask about alerts, counts, sources, model status, latency or attack types.', 'out_of_scope': "Sorry, I
can only help with intrusion-detection questions. Say 'help' to see what I can do."}[intent]
def corrupt(u, wer, rng, vocab):
    """simulate speech-recognition errors: each word is substituted (50%), deleted (25%) or followed by an inserted word (25%) with
probability wer"""
    out = []
    for w in u.split():
        if rng.random() < wer:
            r = rng.random()
            if r < .5: out.append(rng.choice(vocab))
            elif r < .75: pass
            else: out += [w, rng.choice(vocab)]
        else: out.append(w)
    return ' '.join(out)
TEST2 = {  # a second, fresh set written AFTER the stop-word change, used only for the final check
 'list_alerts': ["display the pending warnings", "which intrusions are still open"], 'alert_count': ["what's today's alarm tally", "how
many warnings did we get"],
 'why_flagged': ["explain why alert 31 was raised", "what made you flag flow 8"], 'top_source': ["identify the most dangerous source host",
"which machine attacks us the most"],
 'model_status': ["is the detection model online", "when did the detector last restart"], 'latency_info': ["what is the average processing
delay", "can the detector keep up in real time"],
 'attack_info': ["explain what a syn flood is", "what is a privilege escalation attack"], 'ack_alert': ["close alert 40", "alert 18 is
resolved now"], 'help': ["what commands do you support", "help"]}
OOS2 = ["what is the stock price of apple", "how tall is mount everest", "tell me the news", "who is the president of the usa", "what
should i cook tonight", "play a video", "what is the meaning of life", "send an email to my boss", "when is the next holiday", "how far is
the moon", "what is machine learning", "who invented the telephone"]
```

A.8 Driver: B1.1/B1.2 (t_b11.py)

```
"""B1.1 / B1.2: validation scenarios and comparison with the hand trace of A1.5"""
import json
from drone import *
def show(title, log, out, maxrows=40):
    print(f"\n--- {title}"); print(f"{'t':>3} {'pos':>8} {'bat':>4} {'mode':>8} {'sensed':>14} {'layer':>5} {'action':>6}  note")
    for r in log[:maxrows]: print(f"{r['t']:>3} {str(r['pos']):>8} {r['battery']:>4} {r['mode']:>8} {str(r['sensed']):>14} {r['layer']:>5}
{r['action']:>6}  {r['note']}")
    print("   outcome:", out)
res = {}
print("B1.1 SCENARIOS (deterministic, wind = 0 unless stated)")
# S1 normal flight: 10x10, home (0,0), goal (6,6), one known no-fly block, battery 40
nf = {(0, 2), (0, 3), (1, 2), (1, 3)}
env = MissionEnv(10, 10, (0, 0), (6, 6), nf, battery=40); ag = DroneAgent((0, 0), (6, 6), 10, 10, nf); log, out = run_mission(env, ag);
show("S1 normal flight: goal (6,6), no-fly block rows 0-1, columns 2-3, battery 40", log, out, 40); res['S1'] = out
print("   verifies: the planner routes around known no-fly zones, delivers, returns and lands; expected: delivered = True, incidents = 0,
status LANDED_HOME, steps = 12 + 12 + drop + land")
# S2 sudden obstacle on the planned path
env = MissionEnv(10, 10, (0, 0), (0, 6), set(), sudden={3: (0, 4)}, battery=40); ag = DroneAgent((0, 0), (0, 6), 10, 10, set()); log, out =
run_mission(env, ag); show("S2 sudden obstacle: row-0 corridor, obstacle appears at (0,4) after step 3", log, out, 40); res['S2'] = out
print("   verifies: percept-driven veto (L0) and re-planning (L1) without collision; expected: delivered = True, incidents = 0, replans >=
2")
# S3 boundary: goal blocked permanently
env = MissionEnv(10, 10, (0, 0), (0, 5), set(), sudden={3: (0, 5)}, battery=40); ag = DroneAgent((0, 0), (0, 5), 10, 10, set()); log, out =
run_mission(env, ag); show("S3 boundary: the goal cell becomes an obstacle at t = 3", log, out, 40); res['S3'] = out
print("   verifies: goal unreachable -> mission aborted safely, returns and lands; expected: delivered = False, status LANDED_HOME,
incidents = 0")
# S4 boundary: battery too small for the round trip
env = MissionEnv(10, 10, (0, 0), (0, 9), set(), battery=14); ag = DroneAgent((0, 0), (0, 9), 10, 10, set()); log, out = run_mission(env,
ag); show("S4 boundary: battery 14, goal 9 cells away (round trip needs 18 + reserve)", log, out, 40); res['S4'] = out
print("   verifies: the battery rule turns the drone back while the reserve is still intact; expected: delivered = False, status
LANDED_HOME, battery_left >= 3 - 1")
# S5 wind drift: fixed reserve versus reserve scaled by the wind estimate
from collections import Counter
for battery, adaptive in [(b, a) for b in (36, 50) for a in (False, True)]:
    rs = []
    for sd in range(30):
        env = MissionEnv(10, 10, (0, 0), (6, 6), nf, battery=battery, wind=0.2, seed=sd); ag = DroneAgent((0, 0), (6, 6), 10, 10, nf,
adaptive=adaptive); _, out = run_mission(env, ag); rs.append(out)
    print("\n--- S5 wind drift 20%% per move, 30 seeds, battery %d, %s: delivered %d/30, landed home %d/30, statuses %s, mean steps %.1f,
mean incidents %.2f" % (battery, "adaptive return margin (wind estimated from own drift)" if adaptive else "fixed reserve 3",
sum(o['delivered'] for o in rs), sum(o['status'] == 'LANDED_HOME' for o in rs), dict(Counter(o['status'] for o in rs)), sum(o['steps'] for
o in rs) / 30, sum(o['incidents'] for o in rs) / 30))
    res[f'S5_{battery}_{adaptive}'] = dict(delivered=sum(o['delivered'] for o in rs), home=sum(o['status'] == 'LANDED_HOME' for o in rs),
statuses=dict(Counter(o['status'] for o in rs)))
# ---------------- B1.2 : the hand trace of A1.5 (8x8, home (0,0), goal (0,6), battery 14, reserve 3, sudden obstacle at (0,4) appearing at
t=3)
print("\nB1.2 PROGRAM TRACE OF THE A1.5 SCENARIO: 8x8 grid, home (0,0), goal (0,6), battery 14, reserve 3, obstacle appears at (0,4) before
the move of step 4")
env = MissionEnv(8, 8, (0, 0), (0, 6), set(), sudden={3: (0, 4)}, battery=14); ag = DroneAgent((0, 0), (0, 6), 8, 8, set()); log, out =
run_mission(env, ag); show("A1.5 scenario", log, out, 40); res['trace'] = log
json.dump(res, open('t_b11_results.json', 'w'), default=str, indent=1)
```

A.9 Driver: B1.3/B1.4 (t_b13.py)

```
import json, statistics as st, math
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from swarm import *
out = {}
```

```
print("A2.3  HAND-TRACE CHECK on a 5x5 grid, three drones A(0,0) B(0,4) C(4,2); deterministic tie-break U,R,D,L; drones move in the order
A,B,C; marks update immediately")
f = Field(5, 0.0, 0); pos = {'A': (0, 0), 'B': (0, 4), 'C': (4, 2)}; rng = random.Random(0)
for p in pos.values(): f.mark[p] = 1
vis = set(pos.values()); steps = []
for t in range(1, 7):
    row = []
    for k in 'ABC':
        occ = {v for kk, v in pos.items() if kk != k}; new = swarm_decide(pos[k], f, occ, rng, deterministic=True);
row.append(f"{k}:{pos[k]}->{new}")
        if new in vis: pass
        vis.add(new); f.mark[new] = f.mark.get(new, 0) + 1; pos[k] = new
    print(f"   step {t}:  " + "   ".join(row) + f"   | visited {len(vis)}/25 = {len(vis) / 25:.0%}")
steps.append(len(vis))
print("\nA2.4  FAILURE CONFIGURATION: every cell visited except (4,0); drones B(0,4) and C(0,3) both see no unmarked cell nearby... (RS = 8
so that the frontier is visible)")
f = Field(5, 0.0, 0)
for i in range(5):
    for j in range(5):
        if (i, j) != (4, 0): f.mark[(i, j)] = 1
pos = {'B': (0, 4), 'C': (0, 3)}; vis = {(i, j) for i in range(5) for j in range(5)} - {(4, 0)}; moves = 0
print("   PREDICTION (before running): both drones go for the same unvisited cell (4,0) and walk 7 and 6 steps almost in tandem; at most
one of them adds coverage, so about 13 of 13 moves are redundant.")
for t in range(1, 9):
    row = []
    for k in 'BC':
        occ = {v for kk, v in pos.items() if kk != k}; new = swarm_decide(pos[k], f, occ, rng, RS=8, deterministic=True);
row.append(f"{k}:{pos[k]}->{new}")
        moves += int(new != pos[k]); gained = new not in vis; vis.add(new); f.mark[new] = f.mark.get(new, 0) + 1; pos[k] = new
    print(f"   step {t}:  " + "   ".join(row) + f"   | visited {len(vis)}/25, moves so far {moves}")
    if (4, 0) in vis and len(vis) == 25: break
print("   -> total moves", moves, "to gain exactly one new cell: redundancy ratio", round((moves - 1) / moves, 3))
print("\nB1.4  SINGLE DRONE vs SWARM: 30 seeds x N in {1,5,10,25,50} x failures {0,10,30}%; field 30x30 (5% no-fly), battery 120 moves, T =
150 steps; success = coverage >= 70% of free cells")
SEEDS = range(30); res = {}
def summ(rs, key): return st.mean(r[key] for r in rs)
print(f"   {'N':>3} {'fail':>5} | {'success':>8} {'coverage':>9} {'t to 50%':>9} {'redundancy':>11} {'battery/drone':>14} {'latency
us':>11}")
for N in (1, 5, 10, 25, 50):
    for fr in (0.0, 0.1, 0.3):
        rs = [run('swarm', N, s, fail_frac=fr) for s in SEEDS]
        t50 = [next((i + 1 for i, c in enumerate(r['curve']) if c >= 0.5), None) for r in rs]; t50v = [x for x in t50 if x]
        res[(N, fr)] = dict(success=sum(r['coverage'] >= 0.7 for r in rs) / 30, cov=summ(rs, 'coverage'), cov_sd=st.pstdev([r['coverage']
for r in rs]), red=summ(rs, 'redundancy'), bat=summ(rs, 'battery_used'), lat=summ(rs, 'lat_us'), t50=(st.mean(t50v) if t50v else None),
n50=len(t50v), curve=[st.mean(r['curve'][i] for r in rs) for i in range(150)])
        x = res[(N, fr)]; print(f"   {N:>3} {int(fr * 100):>4}% | {x['success']:>8.2f} {x['cov']:>9.3f} {('%.0f (%d/30)' % (x['t50'],
x['n50'])) if x['t50'] else '   never':>9} {x['red']:>11.3f} {x['bat']:>14.1f} {x['lat']:>11.1f}")
out['swarm'] = {f"{k[0]}_{k[1]}": {kk: vv for kk, vv in v.items() if kk != 'curve'} for k, v in res.items()}
print("\nCENTRALISED CONTROLLER vs SWARM (N = 10, 30 seeds): link loss and controller failure")
cen = {}
print(f"   {'condition':>34} | {'design':>8} {'coverage':>9} {'success':>8}")
for name, kw in [('perfect links', {}), ('30% command loss', dict(link_loss=0.3)), ('controller fails at step 30',
dict(controller_fail_t=30)), ('10% drones fail', dict(fail_frac=0.1)), ('30% drones fail', dict(fail_frac=0.3))]:
    for mode in ('central', 'swarm'):
        kw2 = dict(kw)
        if mode == 'swarm': kw2 = {k: v for k, v in kw2.items() if k == 'fail_frac'}
        rs = [run(mode, 10, s, **kw2) for s in SEEDS]; cen[(name, mode)] = (summ(rs, 'coverage'), sum(r['coverage'] >= 0.7 for r in rs) /
30)
        print(f"   {name:>34} | {mode:>8} {cen[(name, mode)][0]:>9.3f} {cen[(name, mode)][1]:>8.2f}")
out['central'] = {f"{k[0]}|{k[1]}": v for k, v in cen.items()}
fig, ax = plt.subplots(1, 3, figsize=(14, 3.6))
for N, c in zip((1, 5, 10, 25, 50), ('k', 'C0', 'C1', 'C2', 'C3')): ax[0].plot(range(1, 151), res[(N, 0.0)]['curve'], c, label=f'N = {N}')
ax[0].set_xlabel('time step'); ax[0].set_ylabel('coverage of free cells'); ax[0].legend(fontsize=8); ax[0].set_title('coverage against time
(mean of 30 seeds, no failures)', fontsize=9); ax[0].axhline(.7, color='gray', ls=':')
for fr, ls in ((0.0, '-'), (0.1, '--'), (0.3, ':')): ax[1].plot([5, 10, 25, 50], [res[(N, fr)]['cov'] for N in (5, 10, 25, 50)], 'o' + ls,
label=f'{int(fr * 100)}% drones fail')
ax[1].set_xscale('log'); ax[1].set_xticks([5, 10, 25, 50]); ax[1].set_xticklabels([5, 10, 25, 50]); ax[1].set_xlabel('number of drones N');
ax[1].set_ylabel('final coverage'); ax[1].legend(fontsize=8); ax[1].set_title('robustness to drone failures', fontsize=9)
ax[2].plot([1, 5, 10, 25, 50], [res[(N, 0.0)]['red'] for N in (1, 5, 10, 25, 50)], 'o-', color='C4'); ax[2].set_xscale('log');
ax[2].set_xticks([1, 5, 10, 25, 50]); ax[2].set_xticklabels([1, 5, 10, 25, 50]); ax[2].set_xlabel('number of drones N');
ax[2].set_ylabel('redundancy ratio (revisits / moves)'); ax[2].set_title('redundancy grows with N', fontsize=9)
plt.tight_layout(); plt.savefig('fig/b1_swarm.png', dpi=200); plt.close()
json.dump(out, open('t_b13_results.json', 'w'), indent=1, default=str)
```

A.10 Driver: B2 (t_b2.py)

```
import json, copy
from es import *
from pump_kb import *
out = {}
kb = build_kb(); print(f"B2  PUMP-FAULT EXPERT SYSTEM: {len(kb.rules)} rules ({sum(r.kind == 'diagnosis' for r in kb.rules)} diagnosis +
{sum(r.kind == 'derive' for r in kb.rules)} derivation rules), {len(FAULTS)} fault types")
for r in kb.rules: print(f"   {r.rid:>3} [{r.kind[:4]}, pri {r.priority}] IF " + ' AND '.join(f"{c[0]} {c[1]} {c[2]}" for c in r.conds) +
f" THEN {r.concl}")
def evaluate(kb, cases, strategy):
    rows = []
    for cid, obs, truth in cases:
        res = diagnose(kb, obs, strategy); pred = res['diagnosis']; rows.append((cid, truth, pred, res))
    return rows
def metrics(rows):
    acc = sum(t == p for _, t, p, _ in rows) / len(rows); per = {}
    for f in FAULTS:
        tp = sum(t == f and p == f for _, t, p, _ in rows); fp = sum(t != f and p == f for _, t, p, _ in rows); fn = sum(t == f and p != f
for _, t, p, _ in rows)
        per[f] = (tp / (tp + fp) if tp + fp else float('nan'), tp / (tp + fn) if tp + fn else float('nan'), tp + fn)
    return acc, per
print("\nB2.1  RULES WRITTEN FROM THE 15 DEVELOPMENT CASES, EVALUATED ON THE 15 HOLD-OUT CASES (conflict resolution = specificity, ties by
rule order)")
d = evaluate(kb, DEV, 'specificity'); h = evaluate(kb, HOLD, 'specificity'); ad, _ = metrics(d); ah, per = metrics(h)
print(f"   development accuracy {ad:.3f} ({sum(t == p for _, t, p, _ in d)}/15);  HOLD-OUT accuracy {ah:.3f} ({sum(t == p for _, t, p, _ in
h)}/15)")
print(f"   {'fault':>18} | {'precision':>9} {'recall':>7} {'support':>8}");
for f in FAULTS: print(f"   {f:>18} | {per[f][0]:>9.2f} {per[f][1]:>7.2f} {per[f][2]:>8}")
print(f"   {'case':>4} {'truth':>17} {'predicted':>17} {'matched/fired':>14}  conflict set")
for cid, t, p, res in h: print(f"   {cid:>4} {t:>17} {str(p):>17} {len(res['conflict']) + sum(1 for x in res['fired'] if
x.startswith('D')):>7}/{len(res['fired']):<6}  {res['conflict']}" + ('' if t == p else '   <-- wrong'))
```

```
print("   mean rules fired per hold-out case: %.2f ; mean rules matched: %.2f" % (sum(len(r['fired']) for *_, r in h) / 15,
sum(len(r['conflict']) + sum(1 for x in r['fired'] if x.startswith('D')) for *_, r in h) / 15))
print("\n   SAMPLE EXPLANATIONS")
for cid in ('H3', 'H10', 'H5'):
    cid_, obs, truth = next(c for c in HOLD if c[0] == cid); res = diagnose(kb, obs, 'specificity'); print(f"   case {cid} {obs}:");
print('   ' + how(kb, res).replace('\n', '\n   '))
print("   " + why_ask(kb, dict(noise='rattling')) )
print("   " + why_ask(kb, dict(motor_temp='high', flow='normal')))
print("   " + why_ask(kb, dict(noise='grinding', vibration='high', bearing_temp='normal')))
out['holdout_acc'] = ah; out['dev_acc'] = ad
# B2.2
print("\nB2.2  ADVERSARIAL CASES: predictions written before the run, outcomes computed by the shell for the four strategies")
ADV = [
 ('X1 conflicting rules', dict(noise='grinding', vibration='high', bearing_temp='high', axial_vib='high'), 'bearing_wear',
dict(specificity='bearing_wear (R4 first among the tied size-2 rules)', priority='bearing_wear (priority 3 > 1)', recency='misalignment
(R13 uses axial_vib, the newest fact)', order='bearing_wear (R4 precedes R13)')),
 ('X2 missing evidence', dict(noise='rattling', vibration='high'), 'misalignment', dict(specificity='cavitation (only R3 matches)',
priority='cavitation', recency='cavitation', order='cavitation')),
 ('X3 order dependence', dict(leak='visible', motor_temp='high', current='high', flow='normal'), 'motor_overload',
dict(specificity='motor_overload (R12 has 3 conditions)', priority='motor_overload (5 > 4)', recency='motor_overload (R12 uses flow,
newest)', order='seal_leak (R7 precedes R11/R12)')),
 ('X4 dominance', dict(flow='zero', pressure='high', current='high', motor_temp='high'), 'clogged_impeller',
dict(specificity='clogged_impeller (R10 has 3 conditions)', priority='motor_overload (5 > 2)', recency='clogged_impeller (R10/R9 use
pressure/current, but R11 uses derived electrical_stress, newest)', order='clogged_impeller (R9 precedes R11)')),
 ('X5 no evidence', dict(noise='normal', vibration='normal'), None, dict(specificity='no diagnosis', priority='no diagnosis', recency='no
diagnosis', order='no diagnosis'))]
print(f"   {'case':>22} {'strategy':>11} | {'predicted (before)':>60} | {'outcome':>17} | match")
adv_rows = []
for name, obs, truth, pred in ADV:
    for s in STRATEGIES:
        res = diagnose(kb, obs, s); o = res['diagnosis']; ok = (o is not None and o in pred[s]) or (o is None and 'no diagnosis' in
pred[s]); adv_rows.append((name, s, o, ok, truth))
        print(f"   {name:>22} {s:>11} | {pred[s][:60]:>60} | {str(o):>17} | {'yes' if ok else 'NO'}")
print(f"   predictions confirmed: {sum(r[3] for r in adv_rows)} of {len(adv_rows)}")
print("\n   STRATEGY COMPARISON on the 15 hold-out cases and on the 5 adversarial cases (truth = my label; X5 truth = no diagnosis)")
print(f"   {'strategy':>12} | {'hold-out acc':>12} {'adversarial correct':>20}")
strat = {}
for s in STRATEGIES:
    hh = evaluate(kb, HOLD, s); a_h = sum(t == p for _, t, p, _ in hh) / 15
    aa = sum(diagnose(kb, obs, s)['diagnosis'] == truth for _, obs, truth, _ in ADV); strat[s] = (a_h, aa); print(f"   {s:>12} |
{a_h:>12.3f} {aa:>14}/5")
print("   rule-order sensitivity: reversing the order of the rules in the knowledge base (same rules) changes the diagnosis in these hold-
out + adversarial cases:")
kr = KnowledgeBase(list(reversed(build_kb().rules)))
for s in STRATEGIES:
    ch = [c[0] for c in HOLD + [(a[0], a[1], a[2]) for a in ADV] if diagnose(kb, c[1], s)['diagnosis'] != diagnose(kr, c[1],
s)['diagnosis']]; print(f"      {s:>12}: {ch}")
out['strategies'] = strat
# B2.3
print("\nB2.3  KNOWLEDGE ACQUISITION AT RUN TIME (no engine code changed): regression over all 30 cases")
def snap(k, strategy='specificity'): return {c[0]: diagnose(k, c[1], strategy)['diagnosis'] for c in DEV + HOLD}
base = snap(kb); truth = {c[0]: c[2] for c in DEV + HOLD}
def report(name, k, label):
    a = snap(k); ch = {c: (base[c], a[c]) for c in base if base[c] != a[c]}
    print(f"   {label}: accuracy on 30 cases {sum(base[c] == truth[c] for c in base) / 30:.3f} -> {sum(a[c] == truth[c] for c in a) /
30:.3f}; diagnoses changed: {ch}"); return ch
k2 = KnowledgeBase.from_json(kb.to_json()); k2.add(Rule('R15', [('motor_temp', '=', 'high'), ('flow', '=', 'normal')], 'motor_overload',
'diagnosis', 5, 'hot motor with normal flow'))
ch = report('e1', k2, "EDIT 1 (ADD rule R15: motor_temp = high and flow = normal -> motor_overload, to cover H12)")
k3 = KnowledgeBase.from_json(kb.to_json()); k3.edit('R5', conds=[('noise', '=', 'grinding'), ('vibration', '=', 'high'), ('axial_vib',
'!=', 'high')])
ch3 = report('e2', k3, "EDIT 2 (EDIT R5: add 'axial_vib != high' so that axial vibration is not read as bearing wear, to cover H14)")
k4 = KnowledgeBase.from_json(kb.to_json()); k4.edit('R6', conds=[('vibration', '=', 'high')])
ch4 = report('e3', k4, "EDIT 3 (careless EDIT of R6: the bearing-temperature condition is dropped, leaving 'vibration = high')")
k5 = KnowledgeBase.from_json(kb.to_json()); k5.remove('R14'); ch5 = report('e4', k5, "EDIT 4 (REMOVE rule R14)")
ch3 = {'e1': ch, 'e2': ch3, 'e3': ch4, 'e4': ch5}; ch4 = None
out['edits'] = ch3
json.dump(out, open('t_b2_results.json', 'w'), indent=1, default=str)
```

A.11 Driver: B3.1/B3.2/B3.4 (t_b3.py)

```
import json, time, numpy as np, pickle, io
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from ids import *
out = {}
print("B3  EDGE INTRUSION DETECTION (case (a)); data = student-written flow simulator with the KDD/NSL-KDD feature schema (no public
dataset could be downloaded in this environment)")
print("   stream of 60,000 time-ordered flows; split BY TIME: first 60% train, next 10% validation, last 30% test; normal traffic drifts
and flash-crowd bursts increase; the attack type slow_dos exists ONLY in the test period")
def report(name, p, y, thr=0.5):
    pr = (p >= thr).astype(int); return dict(acc=accuracy_score(y, pr), prec=precision_score(y, pr, zero_division=0), rec=recall_score(y,
pr), f1=f1_score(y, pr), auc=roc_auc_score(y, p))
print("\nB3.1  FULL MODEL (200-tree random forest) vs EDGE MODEL (stage-1 depth-3 tree + 15-tree depth-8 quantised forest), 3 stream
seeds")
rows = {}; lat = {}
for seed in (0, 1, 2):
    X, y, typ, t = gen_stream(60000, seed); (Xtr, ytr, _), (Xv, yv, _), (Xte, yte, tte) = split(X, y, typ, t)
    full, ff, casc, small, sf = train_models(Xtr, ytr, seed)
    pf = ff.predict_proba(Xte); pc = casc.predict_proba(Xte); chk = np.abs(ff.predict_proba(Xte[:500]) - full.predict_proba(Xte[:500])[:,
1]).max()
    rows[seed] = dict(full=report('full', pf, yte), edge=report('edge', pc, yte), exit=casc.exit, chk=chk, pf=pf, pc=pc, yte=yte, tte=tte,
ff=ff, casc=casc, sf=sf, Xte=Xte)
    print(f"   seed {seed}: flat-array engine equals scikit-learn probabilities (max abs diff {chk:.2e}); stage-1 early-exit share
{casc.exit:.3f}")
print(f"   {'model':>8} | {'accuracy':>9} {'precision':>10} {'recall':>7} {'F1':>6} {'ROC-AUC':>8}   (mean ± sd over 3 seeds, TEST
period)")
for m in ('full', 'edge'):
    s = {k: (np.mean([rows[sd][m][k] for sd in rows]), np.std([rows[sd][m][k] for sd in rows])) for k in ('acc', 'prec', 'rec', 'f1',
'auc')}; out[m] = {k: v for k, v in s.items()}
    print(f"   {m:>8} | " + ' '.join(f"{s[k][0]:.3f}±{s[k][1]:.3f}" for k in ('acc', 'prec', 'rec', 'f1', 'auc')))
print("\n   per seed (accuracy / recall / F1): " + '; '.join(f"seed {sd}: full
{rows[sd]['full']['acc']:.3f}/{rows[sd]['full']['rec']:.3f}/{rows[sd]['full']['f1']:.3f}, edge
{rows[sd]['edge']['acc']:.3f}/{rows[sd]['edge']['rec']:.3f}/{rows[sd]['edge']['f1']:.3f}" for sd in rows))
r0 = rows[0]; ff, casc, sf = r0['ff'], r0['casc'], r0['sf']; Xte, yte, tte = r0['Xte'], r0['yte'], r0['tte']
```

```
print("\nB3.2  RESOURCE EVIDENCE (seed 0): model size, node visits per flow, single-flow latency measured here (pure-Python flat-array
engine), 1 flow at a time")
print(f"   full: {ff.nodes} nodes in {len(ff.root)} trees = {ff.bytes(True) / 1024:.0f} KiB (18 bytes/node);   edge: stage 1
{casc.s1flat.nodes} nodes + stage 2 {sf.nodes} nodes in {len(sf.root)} trees = {(casc.s1flat.bytes(False) + sf.bytes(False)) / 1024:.1f}
KiB (7 bytes/node, 8-bit thresholds)   -> size ratio {ff.bytes(True) / (casc.s1flat.bytes(False) + sf.bytes(False)):.0f}x")
N = 1500; res = {}
tf = []; vf = []; te = []; ve = []; ex = 0
for i in range(N):
    x = Xte[i]; t0 = time.perf_counter(); p, v = ff.predict_one(x); tf.append(time.perf_counter() - t0); vf.append(v)
    t0 = time.perf_counter(); p2, v2, e = casc.predict_one(x); te.append(time.perf_counter() - t0); ve.append(v2); ex += e
for nm, tl, vl in (('full', tf, vf), ('edge', te, ve)):
    a = np.array(tl) * 1e3; res[nm] = dict(mean=a.mean(), p95=np.percentile(a, 95), p99=np.percentile(a, 99), visits=float(np.mean(vl)),
thr=1000 / a.mean())
    print(f"   {nm:>5}: mean {a.mean():.3f} ms, p95 {np.percentile(a, 95):.3f} ms, p99 {np.percentile(a, 99):.3f} ms, mean node visits
{np.mean(vl):.0f}, sequential throughput {1000 / a.mean():.0f} flows/s")
print(f"   stage-1 early exits in the timing sample: {ex / N:.3f}; the edge model needs {np.mean(vf) / np.mean(ve):.1f}x fewer node visits
and is {np.mean(tf) / np.mean(te):.1f}x faster on this machine")
bud = 50.0
for nm in ('full', 'edge'): print(f"   latency budget {bud:.0f} ms (p95): {nm} meets it on this machine; it would still meet it on a device
up to {bud / res[nm]['p95']:.0f}x slower than this container (p95 {res[nm]['p95']:.3f} ms x factor <= {bud:.0f} ms)")
out['res'] = res
# memory of Python objects is irrelevant; report serialised sizes of scikit-learn objects for reference
print("   (reference: pickled scikit-learn forests: full %.1f MiB)" % (len(pickle.dumps(ff.F)) * 5 / 2 ** 20))
print("\nB3.3  PREDICTIONS (written before the failure analysis) AND ERROR CATEGORIES on the TEST period, seed 0, threshold 0.5")
print("   P1: both models detect the attack classes seen in training (dos, probe, r2l, u2r) with recall >= 0.9 except u2r (rare, 2 %) which
is lower; P2: recall on the UNSEEN attack slow_dos is below 0.3 for both models (its features look like normal traffic); P3: false-positive
rate on flash-crowd bursts (burst_normal) is above that on ordinary normal flows because bursts resemble dos in the count feature.")
print(f"   {'traffic type':>13} {'n':>6} | {'full: detected':>14} {'edge: detected':>15}  (for the attack types 'detected' = recall; for
normal types it is the false-alarm rate)")
for k in TYPES:
    m = tte == k
    if m.sum() == 0: print(f"   {k:>13} {0:>6} | (absent in the test period)"); continue
    print(f"   {k:>13} {m.sum():>6} | {np.mean(r0['pf'][m] >= .5):>14.3f} {np.mean(r0['pc'][m] >= .5):>15.3f}")
    out.setdefault('types', {})[k] = (float(np.mean(r0['pf'][m] >= .5)), float(np.mean(r0['pc'][m] >= .5)), int(m.sum()))
cm = confusion_matrix(yte, r0['pf'] >= .5); cm2 = confusion_matrix(yte, r0['pc'] >= .5); print(f"   confusion matrix full [[TN, FP], [FN,
TP]] = {cm.tolist()};  edge = {cm2.tolist()}")
pre = {};
for k in ('dos', 'probe', 'r2l', 'u2r'):
    m = (tte == k)
print("\n   Threshold sweep for the edge model on the validation-free test period (cost of a missed attack vs a false alarm):")
thrs = [0.1, 0.2, 0.3, 0.5, 0.7, 0.9]; print(f"   {'threshold':>9} | {'recall':>7} {'precision':>10} {'false-alarm rate':>17}")
for th in thrs:
    pr = r0['pc'] >= th; print(f"   {th:>9} | {recall_score(yte, pr):>7.3f} {precision_score(yte, pr, zero_division=0):>10.3f}
{np.mean(pr[yte == 0]):>17.3f}")
print("\n   Concept-drift check: edge model F1 on the test period split into thirds (the later the more drift and the unseen slow_dos):")
q = np.array_split(np.arange(len(yte)), 3)
for i, ix in enumerate(q): print(f"      third {i + 1}: F1 {f1_score(yte[ix], r0['pc'][ix] >= .5):.3f}, recall {recall_score(yte[ix],
r0['pc'][ix] >= .5):.3f}, false-alarm rate {np.mean((r0['pc'][ix] >= .5)[yte[ix] == 0]):.3f}")
fig, ax = plt.subplots(1, 3, figsize=(14, 3.6))
tt = [k for k in TYPES if k in out['types']]; x = np.arange(len(tt)); ax[0].bar(x - .2, [out['types'][k][0] for k in tt], .4,
label='full'); ax[0].bar(x + .2, [out['types'][k][1] for k in tt], .4, label='edge'); ax[0].set_xticks(x); ax[0].set_xticklabels(tt,
rotation=30, fontsize=8); ax[0].set_ylabel('detected / alarmed fraction'); ax[0].legend(fontsize=8); ax[0].set_title('by traffic type (test
period)', fontsize=9)
ax[1].bar(['full', 'edge'], [res['full']['mean'], res['edge']['mean']], color=['C0', 'C1']); ax[1].set_ylabel('mean latency per flow
(ms)'); ax[1].set_title('single-flow latency (measured)', fontsize=9)
ax[2].bar(['full', 'edge'], [ff.bytes(True) / 1024, (casc.s1flat.bytes(False) + sf.bytes(False)) / 1024], color=['C0', 'C1']);
ax[2].set_yscale('log'); ax[2].set_ylabel('model size (KiB, log)'); ax[2].set_title('model size', fontsize=9); plt.tight_layout();
plt.savefig('fig/b3_edge.png', dpi=200); plt.close()
json.dump(out, open('t_b3_results.json', 'w'), indent=1, default=str)
```

A.12 Driver: mis-predictions and the slow_dos retraining demo (t_b3b.py)

```
"""B3.2: individual mis-predictions of the full model on the test period (seed 0), categorised"""
from sklearn.metrics import f1_score as f1
import numpy as np
from ids import *
X, y, typ, t = gen_stream(60000, 0); (Xtr, ytr, _), (Xv, yv, _), (Xte, yte, tte) = split(X, y, typ, t)
full, ff, casc, small, sf = train_models(Xtr, ytr, 0); pf = ff.predict_proba(Xte); pc = casc.predict_proba(Xte)
print("B3.2  MIS-PREDICTIONS (seed 0, test period, threshold 0.5): counts by traffic type and model")
print(f"   {'type':>13} | {'full errors':>11} {'edge errors':>11}")
for k in TYPES:
    m = tte == k; ef = int((((pf >= .5) != (yte == 1)) & m).sum()); ee = int((((pc >= .5) != (yte == 1)) & m).sum()); print(f"   {k:>13} |
{ef:>11} {ee:>11}")
print("\n   Individual examples (feature values: count, serror_rate, duration, failed_logins, num_compromised, logged_in) and the category
assigned")
cats = [('unseen attack type (slow_dos)', 'slow_dos', 'FN'), ('subtle remote-to-local (few failed logins)', 'r2l', 'FN'), ('rare class (u2r
with low compromise count)', 'u2r', 'FN'), ('flash crowd mistaken for an attack', 'burst_normal', 'FP'), ('normal flow with unusual
counts', 'normal', 'FP'), ('dos flow while the edge cascade exits early', 'dos', 'FN')]
rows = []
for name, k, kind in cats:
    for model, p in (('full', pf), ('edge', pc)):
        if k == 'dos' and model == 'full': continue
        wrong = np.where((tte == k) & (((p >= .5) & (yte == 0)) if kind == 'FP' else ((p < .5) & (yte == 1))))[0]
        if len(wrong) == 0: continue
        i = wrong[0]; x = Xte[i]; print(f"   [{name}] {model} model, {kind}, test row {i}: count {x[3]:.0f}, serror {x[5]:.2f}, duration
{x[0]:.1f}, failed_logins {x[9]:.0f}, compromised {x[11]:.0f}, logged_in {x[10]:.0f}; P(attack) = {p[i]:.3f}")
        rows.append((name, model))
    if (tte == k).sum() and False: pass
# demonstrate the class of inputs on which the model fails
m = tte == 'slow_dos'; print(f"\n   FAILURE CLASS DEMONSTRATED: unseen attack slow_dos: full model detects {np.mean(pf[m] >= .5):.3f} of
{m.sum()}, edge model {np.mean(pc[m] >= .5):.3f}; the same model detects 1.000 of the dos flows seen in training. Adding 200 labelled
slow_dos flows to the training data and retraining restores:")
Xa, ya, ta, _ = gen_stream(60000, 7); mm = ta == 'slow_dos'
if mm.sum() < 200:
    Xa, ya, ta, _ = gen_stream(300000, 7); mm = ta == 'slow_dos'
Xadd = Xa[mm][:200]; yadd = np.ones(len(Xadd), int); full2, ff2, casc2, _, _ = train_models(np.vstack([Xtr, Xadd]), np.concatenate([ytr,
yadd]), 0)
print(f"      after adding {len(Xadd)} slow_dos examples: full model detects {np.mean(ff2.predict_proba(Xte)[m] >= .5):.3f}, edge model
{np.mean(casc2.predict_proba(Xte)[m] >= .5):.3f}; overall F1 full {f1(yte, ff2.predict_proba(Xte) >= .5):.3f}")
```

A.13 Driver: B3.3 RDD experiments (t_b34.py)

```
import numpy as np, time, json, os
from minirdd import *
import minirdd
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
NK = 1000; out = {}
def make(n, nparts, seed=0, skew=False):
    r = np.random.RandomState(seed); parts = []; per = n // nparts
    for p in range(nparts):
        if skew: k = np.where(r.rand(per) < 0.5, 7, r.randint(0, NK, per)).astype(np.int16)          # one hot sensor receives about 50%
of the readings
        else: k = r.randint(0, NK, per).astype(np.int16)
        v = (20 + 5 * np.sin(k / 50.0) + r.normal(0, 1 + (k % 10) * 0.3, per)).astype(np.float32); bad = r.rand(per) < 0.02; v[bad] =
np.where(r.rand(bad.sum()) < 0.5, -999.0, 9999.0)   # 2% faulty readings
        parts.append((k, v))
    return parts
def clean_fn(arrs):                                                 # the costly narrow step: validity filter + a derived column
    k, v = arrs; m = (v >= 0) & (v <= 500); k, v = k[m], v[m]; return (k, np.log1p(v) * 0 + v)
def pipeline(rdd, W, combine=True):
    """A4 pipeline: clean -> (sensor, value) -> per-sensor mean / max / std (reduceByKey) -> top-10 noisiest sensors (driver-side
takeOrdered) + count of valid readings"""
    s, c, mx, sq = rdd.reduce_by_key_dense(W, NK, combine=combine); mean = s / np.maximum(c, 1); std = np.sqrt(np.maximum(sq /
np.maximum(c, 1) - mean ** 2, 0)); top = np.argsort(-std)[:10]; return mean, mx, std, top
print("B3.4  DISTRIBUTED PROCESSING WITH A MINI-RDD ENGINE (PySpark is not installable in this sandbox, see the note in the report);
machine cores:", os.cpu_count())
print("\n(a) SCALING: records 1e5, 1e6, 1e7 x partitions {1, 2, 4, 8, 16}; workers = min(partitions, 2); wall-clock seconds for the A4
pipeline (clean + reduceByKey + top-10), median of 3 runs")
W2 = 2; rows = {}
print(f"   {'records':>9} | " + ' '.join(f"{p:>7}" for p in (1, 2, 4, 8, 16)) + " | speed-up of best vs 1 partition")
for n in (10 ** 5, 10 ** 6, 10 ** 7):
    line = [];
    for P in (1, 2, 4, 8, 16):
        rdd = parallelize(f'd{n}_{P}', make(n, P)); r2 = rdd.map_partitions(clean_fn); ts = []
        for rep in range(3):
            t0 = time.perf_counter(); res = pipeline(r2, min(P, W2)); ts.append(time.perf_counter() - t0)
        rows[(n, P)] = float(np.median(ts)); line.append(rows[(n, P)])
        minirdd._SRC.clear()
    print(f"   {n:>9} | " + ' '.join(f"{x:>7.3f}" for x in line) + f" | {line[0] / min(line):.2f}x at {(1, 2, 4, 8,
16)[int(np.argmin(line))]} partitions")
out['scaling'] = {f"{k[0]}_{k[1]}": v for k, v in rows.items()}
# invariance test with identical data re-partitioned
r = np.random.RandomState(1); k = r.randint(0, NK, 600000).astype(np.int16); v = r.normal(20, 3, 600000).astype(np.float32); vals = []
for P in (1, 3, 8):
    idx = np.array_split(np.arange(len(k)), P); rdd = parallelize(f'i{P}', [(k[i], v[i]) for i in idx]);
vals.append(pipeline(rdd.map_partitions(clean_fn), min(P, 2))[2]); minirdd._SRC.clear()
print("   re-partitioning the SAME 600,000 records into 1, 3 and 8 partitions gives per-sensor std vectors equal to %.1e (max abs
difference): results do not depend on the partitioning" % max(np.abs(vals[0] - vals[1]).max(), np.abs(vals[0] - vals[2]).max()))
print("\n(b) SHUFFLE: map-side combine vs shipping every record (1e6 records, 8 partitions, 1000 sensors)")
for combine in (True, False):
    Stats.shuffle_bytes = 0; Stats.shuffles = 0; rdd = parallelize('s', make(10 ** 6, 8)); r2 = rdd.map_partitions(clean_fn); t0 =
time.perf_counter(); pipeline(r2, 2, combine=combine); dt = time.perf_counter() - t0
    print(f"   {'reduceByKey (map-side combine)' if combine else 'groupByKey-style (all records shuffled)':>40}: shuffles {Stats.shuffles},
shuffle volume {Stats.shuffle_bytes / 1e6:.2f} MB, time {dt:.3f} s"); out[f'shuffle_{combine}'] = (Stats.shuffle_bytes, dt);
minirdd._SRC.clear()
def heavy_fn(arrs):                                                 # an expensive narrow step: clean + rank-based feature (sort of the
partition)
    k, v = clean_fn(arrs); o = np.argsort(v, kind='stable'); rank = np.empty(len(v), np.float32); rank[o] = np.arange(len(v)) / max(len(v),
1); return (k, (v + 0 * rank).astype(np.float32))
print("\n(c) CACHING: three actions (stats, top-10 on std, count of readings above 30) on the same intermediate RDD, 1e7 records, 8
partitions; a CHEAP intermediate step (validity filter) and an EXPENSIVE one (filter + sort of every partition)")
for name, fn in (('cheap step', clean_fn), ('expensive step', heavy_fn)):
    for cache in (False, True):
        rdd = parallelize('k', make(10 ** 7, 8)); r2 = rdd.map_partitions(fn)
        t0 = time.perf_counter()
        if cache: r2.cache_it(); r2.materialise(2)
        pipeline(r2, 2); pipeline(r2, 2); n_hi = r2.map_partitions(lambda a: (a[0][a[1] > 30], a[1][a[1] > 30])).count(2); dt =
time.perf_counter() - t0
        print(f"   {name:>14}, cache = {str(cache):>5}: total {dt:.3f} s (cached partitions held in the driver: {len(r2.cache)}); readings
above 30: {n_hi}"); out[f'cache_{name}_{cache}'] = dt; minirdd._SRC.clear()
print("\n(d) SKEW: one hot sensor holds ~50% of the readings (1e7 records, 8 partitions)")
for skew in (False, True):
    for combine in (True, False):
        Stats.shuffle_bytes = 0; rdd = parallelize('z', make(10 ** 7, 8, skew=skew)); r2 = rdd.map_partitions(clean_fn); t0 =
time.perf_counter(); pipeline(r2, 2, combine=combine); dt = time.perf_counter() - t0
        print(f"   skew = {str(skew):>5}, {'combine ' if combine else 'no-combine'}: {dt:.3f} s, shuffle {Stats.shuffle_bytes / 1e6:.1f}
MB"); minirdd._SRC.clear()
print("\n(e) FAULT TOLERANCE BY LINEAGE: lose one cached partition and recompute only that partition (1e7 records, 8 partitions)")
rdd = parallelize('f', make(10 ** 7, 8)); r2 = rdd.map_partitions(clean_fn).cache_it(); r2.materialise(2); before = pipeline(r2, 2)[2]
t0 = time.perf_counter(); del r2.cache[3]; r2.computed = 0; after = pipeline(r2, 1)[2]; dt = time.perf_counter() - t0
print(f"   after dropping partition 3 of 8: recomputed {r2.computed} partition(s) from the lineage; result identical:
{bool(np.allclose(before, after))}"); time {dt:.3f} s"); out['lineage'] = dict(recomputed=r2.computed, same=bool(np.allclose(before, after)))
fig, ax = plt.subplots(1, 2, figsize=(10, 3.4))
for n in (10 ** 5, 10 ** 6, 10 ** 7): ax[0].plot([1, 2, 4, 8, 16], [rows[(n, P)] for P in (1, 2, 4, 8, 16)], 'o-', label=f'{n:.0e}
records')
ax[0].set_xscale('log', base=2); ax[0].set_yscale('log'); ax[0].set_xlabel('partitions'); ax[0].set_ylabel('seconds');
ax[0].legend(fontsize=8); ax[0].set_title('pipeline time against partitions (2 cores)', fontsize=9)
ax[1].bar(['no cache', 'cache'], [out['cache_expensive step_False'], out['cache_expensive step_True']], color=['C3', 'C2']);
ax[1].set_ylabel('seconds for 3 actions'); ax[1].set_title('caching, expensive step (1e7 records)', fontsize=9); plt.tight_layout();
plt.savefig('fig/b3_rdd.png', dpi=200); plt.close()
json.dump(out, open('t_b34_results.json', 'w'), indent=1, default=str)
```

A.14 Driver: B4 (t_b4.py)

```
import random, time, json, numpy as np
from chatbot import *
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
out = {}
import chatbot
m = IntentModel(); intents = list(TRAIN) + ['out_of_scope']
print(f"B4  CHATBOT: {len(TRAIN)} intents, {sum(len(v) for v in TRAIN.values())} training utterances, {sum(len(v) for v in TEST.values())}
in-scope test utterances (different wording) and {len(OOS)} out-of-scope test utterances; all written by the student")
def evaluate(thr, wer=0.0, seed=0):
    rng = random.Random(seed); vocab = sorted(m.idf); y = []; p = []
    for k, us in TEST.items():
```

```
        for u in us: y.append(k); p.append(m.classify(corrupt(u, wer, rng, vocab), thr)[0])
    for u in OOS: y.append('out_of_scope'); p.append(m.classify(corrupt(u, wer, rng, vocab), thr)[0])
    return y, p
def scores(y, p):
    acc = np.mean([a == b for a, b in zip(y, p)]); per = {}
    for k in intents:
        tp = sum(a == k and b == k for a, b in zip(y, p)); fp = sum(a != k and b == k for a, b in zip(y, p)); fn = sum(a == k and b != k
for a, b in zip(y, p)); pr = tp / (tp + fp) if tp + fp else 0; rc = tp / (tp + fn) if tp + fn else 0; per[k] = (pr, rc, 2 * pr * rc / (pr +
rc) if pr + rc else 0, tp + fn)
    oos_far = np.mean([b != 'out_of_scope' for a, b in zip(y, p) if a == 'out_of_scope']); inscope_rej = np.mean([b == 'out_of_scope' for
a, b in zip(y, p) if a != 'out_of_scope'])
    return acc, per, oos_far, inscope_rej
y, p = evaluate(0.3); acc, per, far, rej = scores(y, p)
print(f"\nB4.1  THRESHOLD 0.30, clean text: accuracy {acc:.3f} (n = {len(y)}), macro-F1 {np.mean([v[2] for v in per.values()]):.3f}; out-
of-scope falsely accepted {far:.2f}; in-scope wrongly rejected {rej:.2f}")
print(f"   {'intent':>13} | {'precision':>9} {'recall':>7} {'F1':>6} {'n':>3}")
for k, v in per.items(): print(f"   {k:>13} | {v[0]:>9.2f} {v[1]:>7.2f} {v[2]:>6.2f} {v[3]:>3}")
print("   misclassified utterances:");
for u, a, b in [(u, a, b) for (a, u), b in zip([(k, u) for k, us in TEST.items() for u in us] + [('out_of_scope', u) for u in OOS], p) if a
!= b]: print(f"      '{u}': true {a}, predicted {b}")
out['clean'] = dict(acc=acc, far=far, rej=rej)
print("\nB4.2  THRESHOLD SWEEP (clean text): the trade-off between accepting out-of-scope questions and rejecting real ones")
print(f"   {'threshold':>9} | {'accuracy':>8} {'OOS accepted':>13} {'in-scope rejected':>18}")
sw = []
for thr in (0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7):
    yy, pp = evaluate(thr); a, _, f, r = scores(yy, pp); sw.append((thr, a, f, r)); print(f"   {thr:>9} | {a:>8.3f} {f:>13.2f} {r:>18.2f}")
out['sweep'] = sw
print("\nB4.3  SIMULATED SPEECH-RECOGNITION ERRORS (word error rate injected into every test utterance, 20 random seeds each, threshold
0.30)")
print(f"   {'WER':>5} | {'accuracy (mean ± sd)':>22} {'OOS accepted':>13} {'in-scope rejected':>18}")
wr = []
for wer in (0.0, 0.1, 0.2, 0.3, 0.4):
    rs = [scores(*evaluate(0.3, wer, s)) for s in range(20)]; a = [r[0] for r in rs]; wr.append((wer, np.mean(a), np.std(a), np.mean([r[2]
for r in rs]), np.mean([r[3] for r in rs])))
    print(f"   {wer:>5.1f} | {np.mean(a):>13.3f} ± {np.std(a):.3f} {np.mean([r[2] for r in rs]):>13.2f} {np.mean([r[3] for r in
rs]):>18.2f}")
out['wer'] = wr
print("\nB4.3b  IMPROVEMENT: question words (what, who, how, tell, can, ...) carry no intent and were matching many intents; they are added
to the stop list (the change was motivated by the clean-text errors above, so it is checked on a SECOND, fresh set written afterwards: 18
in-scope + 12 out-of-scope utterances)")
def eval2(thr):
    y2 = [k for k, us in TEST2.items() for u in us] + ['out_of_scope'] * len(OOS2); u2 = [u for k, us in TEST2.items() for u in us] + OOS2;
p2 = [m.classify(u, thr)[0] for u in u2]
    return np.mean([a == b for a, b in zip(y2, p2)]), np.mean([b != 'out_of_scope' for a, b in zip(y2, p2) if a == 'out_of_scope']),
np.mean([b == 'out_of_scope' for a, b in zip(y2, p2) if a != 'out_of_scope'])
print(f"   {'version':>26} {'thr':>4} | {'set 1 accuracy':>14} {'OOS accepted':>13} {'in-scope rejected':>18} | {'set 2 accuracy':>14}
{'OOS accepted':>13} {'in-scope rejected':>18}")
cmp = {}
for name, st in (('v1 basic stop list', chatbot.STOP), ('v2 + question words', chatbot.STOP2)):
    chatbot.STOP_ACTIVE[0] = st; m = IntentModel()
    for thr in (0.3, 0.4, 0.5):
        yy, pp = evaluate(thr); a, _, f, r = scores(yy, pp); a2, f2, r2 = eval2(thr); cmp[(name, thr)] = (a, f, r, a2, f2, r2); print(f"
{name:>26} {thr:>4} | {a:>14.3f} {f:>13.2f} {r:>18.2f} | {a2:>14.3f} {f2:>13.2f} {r2:>18.2f}")
out['cmp'] = {f"{k[0]}|{k[1]}": v for k, v in cmp.items()}
chatbot.STOP_ACTIVE[0] = chatbot.STOP2; m = IntentModel()
print("\nB4.4  LATENCY: mean time per utterance (classification + response)")
us = [u for v in TEST.values() for u in v] * 20; t0 = time.perf_counter()
for u in us: i, _ = m.classify(u); respond(i, u, {})
print(f"   {1e3 * (time.perf_counter() - t0) / len(us):.3f} ms per utterance over {len(us)} utterances (pure Python, one core)");
out['lat_ms'] = 1e3 * (time.perf_counter() - t0) / len(us)
print("\nB4.5  EXAMPLE DIALOGUE (state: pending clarification)")
def chat(u):
    i, s = m.classify(u); print(f"   user: {u!r:48} -> intent {i} (cosine {s:.2f})"); print(f"   bot : {respond(i, u, {})}")
for u in ["acknowledge alert", "ack alert 5", "why was it flagged", "explain alert 12", "what is the weather", "how many alerts today"]:
chat(u)
fig, ax = plt.subplots(1, 2, figsize=(10, 3.4)); s_ = np.array(sw); ax[0].plot(s_[:, 0], s_[:, 1], 'o-', label='accuracy');
ax[0].plot(s_[:, 0], s_[:, 2], 's-', label='OOS accepted'); ax[0].plot(s_[:, 0], s_[:, 3], '^-', label='in-scope rejected');
ax[0].set_xlabel('cosine threshold'); ax[0].legend(fontsize=8); ax[0].set_title('threshold trade-off', fontsize=9)
w_ = np.array(wr); ax[1].errorbar(w_[:, 0], w_[:, 1], w_[:, 2], fmt='o-'); ax[1].set_xlabel('injected word error rate');
ax[1].set_ylabel('intent accuracy'); ax[1].set_title('robustness to recognition errors', fontsize=9); plt.tight_layout();
plt.savefig('fig/b4_chat.png', dpi=200); plt.close()
json.dump(out, open('t_b4_results.json', 'w'), indent=1, default=str)
```

Appendix B – Complete Console Output

B.1 B1.1 and B1.2 (t_b11_output.txt)

```
B1.1 SCENARIOS (deterministic, wind = 0 unless stated)


--- S1 normal flight: goal (6,6), no-fly block rows 0-1, columns 2-3, battery 40
  t      pos  bat     mode          sensed layer action  note
  0   (0, 0)   40  MISSION              []    L1      E  follow plan, 11 cells left
  1   (0, 1)   39  MISSION              []    L1      S  follow plan, 10 cells left
  2   (1, 1)   38  MISSION              []    L1      S  follow plan, 9 cells left
  3   (2, 1)   37  MISSION              []    L1      E  follow plan, 8 cells left
  4   (2, 2)   36  MISSION              []    L1      E  follow plan, 7 cells left
  5   (2, 3)   35  MISSION              []    L1      E  follow plan, 6 cells left
  6   (2, 4)   34  MISSION              []    L1      E  follow plan, 5 cells left
  7   (2, 5)   33  MISSION              []    L1      E  follow plan, 4 cells left
  8   (2, 6)   32  MISSION              []    L1      S  follow plan, 3 cells left
  9   (3, 6)   31  MISSION              []    L1      S  follow plan, 2 cells left
 10   (4, 6)   30  MISSION              []    L1      S  follow plan, 1 cells left
 11   (5, 6)   29  MISSION              []    L1      S  follow plan, 0 cells left
 12   (6, 6)   28    RETURN             []    L1   Drop  parcel delivered; start return
 13   (6, 6)   27    RETURN             []    L1      N  follow plan, 11 cells left
 14   (5, 6)   26    RETURN             []    L1      N  follow plan, 10 cells left
 15   (4, 6)   25    RETURN             []    L1      N  follow plan, 9 cells left
 16   (3, 6)   24    RETURN             []    L1      N  follow plan, 8 cells left
 17   (2, 6)   23    RETURN             []    L1      W  follow plan, 7 cells left
 18   (2, 5)   22    RETURN             []    L1      W  follow plan, 6 cells left
 19   (2, 4)   21    RETURN             []    L1      W  follow plan, 5 cells left
 20   (2, 3)   20    RETURN             []    L1      W  follow plan, 4 cells left
 21   (2, 2)   19    RETURN             []    L1      W  follow plan, 3 cells left
 22   (2, 1)   18    RETURN             []    L1      N  follow plan, 2 cells left
 23   (1, 1)   17    RETURN             []    L1      N  follow plan, 1 cells left
 24   (0, 1)   16    RETURN             []    L1      W  follow plan, 0 cells left
 25   (0, 0)   15      DONE             []    L0   Land  landed at home
   outcome: {'status': 'LANDED_HOME', 'delivered': True, 'steps': 26, 'battery_left': 15, 'incidents': 0, 'replans': 2, 'vetoes': 0}
   verifies: the planner routes around known no-fly zones, delivers, returns and lands; expected: delivered = True, incidents = 0, status LANDED_HOME,
steps = 12 + 12 + drop + land


--- S2 sudden obstacle: row-0 corridor, obstacle appears at (0,4) after step 3
  t      pos  bat     mode          sensed layer action  note
  0   (0, 0)   40  MISSION              []    L1      E  follow plan, 5 cells left
  1   (0, 1)   39  MISSION              []    L1      E  follow plan, 4 cells left
  2   (0, 2)   38  MISSION              []    L1      E  follow plan, 3 cells left
  3   (0, 3)   37  MISSION       [(0, 4)]    L0      S  obstacle ahead: step vetoed, replan
  4   (1, 3)   36  MISSION              []    L1      E  follow plan, 3 cells left
  5   (1, 4)   35  MISSION       [(0, 4)]    L1      E  follow plan, 2 cells left
  6   (1, 5)   34  MISSION              []    L1      N  follow plan, 1 cells left
  7   (0, 5)   33  MISSION       [(0, 4)]    L1      E  follow plan, 0 cells left
  8   (0, 6)   32    RETURN             []    L1   Drop  parcel delivered; start return
  9   (0, 6)   31    RETURN             []    L1      W  follow plan, 7 cells left
 10   (0, 5)   30    RETURN       [(0, 4)]    L1      S  follow plan, 6 cells left
 11   (1, 5)   29    RETURN             []    L1      W  follow plan, 5 cells left
 12   (1, 4)   28    RETURN       [(0, 4)]    L1      W  follow plan, 4 cells left
 13   (1, 3)   27    RETURN             []    L1      N  follow plan, 3 cells left
 14   (0, 3)   26    RETURN       [(0, 4)]    L1      W  follow plan, 2 cells left
 15   (0, 2)   25    RETURN             []    L1      W  follow plan, 1 cells left
 16   (0, 1)   24    RETURN             []    L1      W  follow plan, 0 cells left
 17   (0, 0)   23      DONE             []    L0   Land  landed at home
   outcome: {'status': 'LANDED_HOME', 'delivered': True, 'steps': 18, 'battery_left': 23, 'incidents': 0, 'replans': 3, 'vetoes': 1}
   verifies: percept-driven veto (L0) and re-planning (L1) without collision; expected: delivered = True, incidents = 0, replans >= 2


--- S3 boundary: the goal cell becomes an obstacle at t = 3
  t      pos  bat     mode          sensed layer action  note
  0   (0, 0)   40  MISSION              []    L1      E  follow plan, 4 cells left
  1   (0, 1)   39  MISSION              []    L1      E  follow plan, 3 cells left
  2   (0, 2)   38  MISSION              []    L1      E  follow plan, 2 cells left
  3   (0, 3)   37  MISSION              []    L1      E  follow plan, 1 cells left
  4   (0, 4)   36    RETURN       [(0, 5)]    L0      W  obstacle ahead: step vetoed, replan goal unreachable: RETURN
  5   (0, 3)   35    RETURN             []    L1      W  follow plan, 2 cells left
  6   (0, 2)   34    RETURN             []    L1      W  follow plan, 1 cells left
  7   (0, 1)   33    RETURN             []    L1      W  follow plan, 0 cells left
  8   (0, 0)   32      DONE             []    L0   Land  landed at home
   outcome: {'status': 'LANDED_HOME', 'delivered': False, 'steps': 9, 'battery_left': 32, 'incidents': 0, 'replans': 3, 'vetoes': 1}
   verifies: goal unreachable -> mission aborted safely, returns and lands; expected: delivered = False, status LANDED_HOME, incidents = 0


--- S4 boundary: battery 14, goal 9 cells away (round trip needs 18 + reserve)
  t      pos  bat     mode          sensed layer action  note
  0   (0, 0)   14  MISSION              []    L1      E  follow plan, 8 cells left
  1   (0, 1)   13  MISSION              []    L1      E  follow plan, 7 cells left
  2   (0, 2)   12  MISSION              []    L1      E  follow plan, 6 cells left
  3   (0, 3)   11  MISSION              []    L1      E  follow plan, 5 cells left
  4   (0, 4)   10  MISSION              []    L1      E  follow plan, 4 cells left
  5   (0, 5)    9    RETURN             []    L0      W  battery 9 <= distance home 5 + reserve 3: RETURN
  6   (0, 4)    8    RETURN             []    L1      W  follow plan, 3 cells left
  7   (0, 3)    7    RETURN             []    L1      W  follow plan, 2 cells left
  8   (0, 2)    6    RETURN             []    L1      W  follow plan, 1 cells left
  9   (0, 1)    5    RETURN             []    L1      W  follow plan, 0 cells left
 10   (0, 0)    4      DONE             []    L0   Land  landed at home
   outcome: {'status': 'LANDED_HOME', 'delivered': False, 'steps': 11, 'battery_left': 4, 'incidents': 0, 'replans': 2, 'vetoes': 0}
   verifies: the battery rule turns the drone back while the reserve is still intact; expected: delivered = False, status LANDED_HOME, battery_left >=
3 - 1


--- S5 wind drift 20% per move, 30 seeds, battery 36, fixed reserve 3: delivered 3/30, landed home 28/30, statuses {'LANDED_HOME': 28,
'BATTERY_EXHAUSTED': 2}, mean steps 31.9, mean incidents 0.93


--- S5 wind drift 20% per move, 30 seeds, battery 36, adaptive return margin (wind estimated from own drift): delivered 1/30, landed home 30/30,
statuses {'LANDED_HOME': 30}, mean steps 26.2, mean incidents 1.00


--- S5 wind drift 20% per move, 30 seeds, battery 50, fixed reserve 3: delivered 29/30, landed home 29/30, statuses {'LANDED_HOME': 29,
'BATTERY_EXHAUSTED': 1}, mean steps 36.6, mean incidents 1.23


--- S5 wind drift 20% per move, 30 seeds, battery 50, adaptive return margin (wind estimated from own drift): delivered 16/30, landed home 30/30,
statuses {'LANDED_HOME': 30}, mean steps 33.7, mean incidents 1.00
```

```
B1.2 PROGRAM TRACE OF THE A1.5 SCENARIO: 8x8 grid, home (0,0), goal (0,6), battery 14, reserve 3, obstacle appears at (0,4) before the move of step 4


--- A1.5 scenario
  t      pos  bat     mode         sensed layer action  note
  0   (0, 0)   14  MISSION             []    L1      E  follow plan, 5 cells left
  1   (0, 1)   13  MISSION             []    L1      E  follow plan, 4 cells left
  2   (0, 2)   12  MISSION             []    L1      E  follow plan, 3 cells left
  3   (0, 3)   11  MISSION       [(0, 4)]    L0      S  obstacle ahead: step vetoed, replan
  4   (1, 3)   10  MISSION             []    L1      E  follow plan, 3 cells left
  5   (1, 4)    9   RETURN       [(0, 4)]    L0      W  battery 9 <= distance home 5 + reserve 3: RETURN
  6   (1, 3)    8   RETURN             []    L1      N  follow plan, 3 cells left
  7   (0, 3)    7   RETURN       [(0, 4)]    L1      W  follow plan, 2 cells left
  8   (0, 2)    6   RETURN             []    L1      W  follow plan, 1 cells left
  9   (0, 1)    5   RETURN             []    L1      W  follow plan, 0 cells left
 10   (0, 0)    4     DONE             []    L0   Land  landed at home
   outcome: {'status': 'LANDED_HOME', 'delivered': False, 'steps': 11, 'battery_left': 4, 'incidents': 0, 'replans': 3, 'vetoes': 1}
```

B.2 B1.3 and B1.4 (t_b13_output.txt)

```
A2.3  HAND-TRACE CHECK on a 5x5 grid, three drones A(0,0) B(0,4) C(4,2); deterministic tie-break U,R,D,L; drones move in the order A,B,C; marks update
immediately
   step 1:  A:(0, 0)->(0, 1)   B:(0, 4)->(1, 4)   C:(4, 2)->(3, 2)   | visited 6/25 = 24%
   step 2:  A:(0, 1)->(0, 2)   B:(1, 4)->(2, 4)   C:(3, 2)->(2, 2)   | visited 9/25 = 36%
   step 3:  A:(0, 2)->(0, 3)   B:(2, 4)->(3, 4)   C:(2, 2)->(1, 2)   | visited 12/25 = 48%
   step 4:  A:(0, 3)->(1, 3)   B:(3, 4)->(4, 4)   C:(1, 2)->(1, 1)   | visited 15/25 = 60%
   step 5:  A:(1, 3)->(2, 3)   B:(4, 4)->(4, 3)   C:(1, 1)->(2, 1)   | visited 18/25 = 72%
   step 6:  A:(2, 3)->(3, 3)   B:(4, 3)->(4, 2)   C:(2, 1)->(3, 1)   | visited 20/25 = 80%


A2.4  FAILURE CONFIGURATION: every cell visited except (4,0); drones B(0,4) and C(0,3) both see no unmarked cell nearby... (RS = 8 so that the frontier
is visible)
   PREDICTION (before running): both drones go for the same unvisited cell (4,0) and walk 7 and 6 steps almost in tandem; at most one of them adds
coverage, so about 13 of 13 moves are redundant.
   step 1:  B:(0, 4)->(1, 4)   C:(0, 3)->(1, 3)   | visited 24/25, moves so far 2
   step 2:  B:(1, 4)->(2, 4)   C:(1, 3)->(2, 3)   | visited 24/25, moves so far 4
   step 3:  B:(2, 4)->(3, 4)   C:(2, 3)->(3, 3)   | visited 24/25, moves so far 6
   step 4:  B:(3, 4)->(4, 4)   C:(3, 3)->(4, 3)   | visited 24/25, moves so far 8
   step 5:  B:(4, 4)->(3, 4)   C:(4, 3)->(4, 2)   | visited 24/25, moves so far 10
   step 6:  B:(3, 4)->(4, 4)   C:(4, 2)->(4, 1)   | visited 24/25, moves so far 12
   step 7:  B:(4, 4)->(4, 3)   C:(4, 1)->(4, 0)   | visited 25/25, moves so far 14
   -> total moves 14 to gain exactly one new cell: redundancy ratio 0.929


B1.4  SINGLE DRONE vs SWARM: 30 seeds x N in {1,5,10,25,50} x failures {0,10,30}%; field 30x30 (5% no-fly), battery 120 moves, T = 150 steps; success =
coverage >= 70% of free cells
     N  fail |  success  coverage  t to 50%  redundancy  battery/drone  latency us
     1    0% |     0.00     0.136     never       0.043          120.0         2.5
     1   10% |     0.00     0.044     never       0.038           38.0         2.4
     1   30% |     0.00     0.044     never       0.038           38.0         2.5
     5    0% |     0.00     0.592 102 (30/30)       0.159          120.0         3.6
     5   10% |     0.00     0.518 114 (21/30)       0.141          102.8         3.5
     5   30% |     0.00     0.453     never       0.104           86.3         2.8
    10    0% |     1.00     0.944 61 (30/30)       0.329          120.0         5.4
    10   10% |     1.00     0.902 62 (30/30)       0.308          111.2         5.3
    10   30% |     1.00     0.814 69 (30/30)       0.256           93.3         5.1
    25    0% |     1.00     1.000 46 (30/30)       0.716          120.0        15.4
    25   10% |     1.00     1.000 48 (30/30)       0.686          108.9        15.8
    25   30% |     1.00     0.998 49 (30/30)       0.622           90.2        11.8
    50    0% |     1.00     1.000 46 (30/30)       0.857          119.6        16.9
    50   10% |     1.00     1.000 46 (30/30)       0.844          109.5        14.8
    50   30% |     1.00     1.000 48 (30/30)       0.809           89.6        14.5


CENTRALISED CONTROLLER vs SWARM (N = 10, 30 seeds): link loss and controller failure
                          condition |   design  coverage  success
                      perfect links |  central     0.897     0.97
                      perfect links |    swarm     0.944     1.00
                   30% command loss |  central     0.811     0.83
                   30% command loss |    swarm     0.944     1.00
         controller fails at step 30 |  central     0.208     0.00
         controller fails at step 30 |    swarm     0.944     1.00
                    10% drones fail |  central     0.866     0.97
                    10% drones fail |    swarm     0.902     1.00
                    30% drones fail |  central     0.755     0.83
                    30% drones fail |    swarm     0.814     1.00
```

B.3 B2 (t_b2_output.txt)

```
B2  PUMP-FAULT EXPERT SYSTEM: 16 rules (14 diagnosis + 2 derivation rules), 6 fault types
    D1 [deri, pri 0] IF flow in ['low', 'zero'] THEN ('flow_deficit', 'yes')
    D2 [deri, pri 0] IF motor_temp = high AND current = high THEN ('electrical_stress', 'yes')
    R1 [diag, pri 3] IF noise = rattling AND suction = low THEN cavitation
    R2 [diag, pri 3] IF noise = rattling AND pressure = low AND vibration = high THEN cavitation
    R3 [diag, pri 3] IF noise = rattling AND vibration = high THEN cavitation
    R4 [diag, pri 3] IF noise = grinding AND bearing_temp = high THEN bearing_wear
    R5 [diag, pri 3] IF noise = grinding AND vibration = high THEN bearing_wear
    R6 [diag, pri 3] IF bearing_temp = high AND vibration = high THEN bearing_wear
    R7 [diag, pri 4] IF leak = visible THEN seal_leak
    R8 [diag, pri 4] IF leak = visible AND pressure = low THEN seal_leak
    R9 [diag, pri 2] IF flow_deficit = yes AND pressure = high THEN clogged_impeller
   R10 [diag, pri 2] IF flow_deficit = yes AND current = high AND pressure = high THEN clogged_impeller
   R11 [diag, pri 5] IF electrical_stress = yes THEN motor_overload
   R12 [diag, pri 5] IF motor_temp = high AND current = high AND flow = normal THEN motor_overload
   R13 [diag, pri 1] IF axial_vib = high AND vibration = high THEN misalignment
   R14 [diag, pri 1] IF axial_vib = high AND bearing_temp = normal THEN misalignment


B2.1  RULES WRITTEN FROM THE 15 DEVELOPMENT CASES, EVALUATED ON THE 15 HOLD-OUT CASES (conflict resolution = specificity, ties by rule order)
   development accuracy 1.000 (15/15);  HOLD-OUT accuracy 0.867 (13/15)
                fault | precision  recall  support
            cavitation |      1.00    1.00        3
         bearing_wear |      0.75    1.00        3
            seal_leak |      1.00    1.00        2
     clogged_impeller |      1.00    1.00        2
       motor_overload |      1.00    0.67        3
         misalignment |      1.00    0.50        2
   case           truth          predicted  matched/fired  conflict set
     H1       cavitation        cavitation       4/2       ['R1', 'R2', 'R3']
     H2       cavitation        cavitation       2/2       ['R1']
     H3     bearing_wear      bearing_wear       3/1       ['R4', 'R5', 'R6']
```

```
     H4      bearing_wear      bearing_wear       3/1       ['R4', 'R5', 'R6']
     H5      bearing_wear      bearing_wear       1/1       ['R5']
     H6         seal_leak         seal_leak       3/2       ['R7', 'R8']
     H7         seal_leak         seal_leak       1/1       ['R7']
     H8  clogged_impeller  clogged_impeller       3/2       ['R9', 'R10']
     H9  clogged_impeller  clogged_impeller       3/2       ['R9', 'R10']
    H10    motor_overload    motor_overload       3/2       ['R11', 'R12']
    H11    motor_overload    motor_overload       3/3       ['R11']
    H12    motor_overload              None       0/0       []   <-- wrong
    H13      misalignment      misalignment       2/1       ['R13', 'R14']
    H14      misalignment      bearing_wear       2/1       ['R5', 'R13']   <-- wrong
    H15        cavitation        cavitation       2/1       ['R2', 'R3']
   mean rules fired per hold-out case: 1.47 ; mean rules matched: 2.33


   SAMPLE EXPLANATIONS
   case H3 {'noise': 'grinding', 'vibration': 'high', 'bearing_temp': 'high', 'flow': 'normal'}:
   HOW: diagnosis 'bearing_wear' was concluded by rule R4 (grinding noise with hot bearing).
      because noise = grinding  [observed]
      because bearing_temp = high  [observed]
      other rules that also matched and lost the conflict resolution: ['R5', 'R6']
   case H10 {'motor_temp': 'high', 'current': 'high', 'flow': 'normal', 'vibration': 'normal'}:
   HOW: diagnosis 'motor_overload' was concluded by rule R12 (hot motor, high current, normal flow).
      because motor_temp = high  [observed]
      because current = high  [observed]
      because flow = normal  [observed]
      other rules that also matched and lost the conflict resolution: ['R11']
   case H5 {'noise': 'grinding', 'vibration': 'high', 'bearing_temp': 'normal'}:
   HOW: diagnosis 'bearing_wear' was concluded by rule R5 (grinding noise with high vibration).
      because noise = grinding  [observed]
      because vibration = high  [observed]
   WHY ask 'suction'? Because rule R1 (rattling noise with low suction pressure) could conclude 'cavitation' and it needs suction = low; the other
conditions are already satisfied or unknown.
   WHY ask 'current'? Because rule R12 (hot motor, high current, normal flow) could conclude 'motor_overload' and it needs current = high; the other
conditions are already satisfied or unknown.
   WHY ask 'axial vib'? Because rule R13 (high axial and radial vibration) could conclude 'misalignment' and it needs axial vib = high; the other
conditions are already satisfied or unknown.


B2.2  ADVERSARIAL CASES: predictions written before the run, outcomes computed by the shell for the four strategies
                    case    strategy |                                                predicted (before) |           outcome | match
     X1 conflicting rules specificity |          bearing_wear (R4 first among the tied size-2 rules) |      bearing_wear | yes
     X1 conflicting rules    priority |                                  bearing_wear (priority 3 > 1) |      bearing_wear | yes
     X1 conflicting rules     recency |           misalignment (R13 uses axial_vib, the newest fact) |      misalignment | yes
     X1 conflicting rules       order |                                bearing_wear (R4 precedes R13) |      bearing_wear | yes
      X2 missing evidence specificity |                                  cavitation (only R3 matches) |        cavitation | yes
      X2 missing evidence    priority |                                                   cavitation |        cavitation | yes
      X2 missing evidence     recency |                                                   cavitation |        cavitation | yes
      X2 missing evidence       order |                                                   cavitation |        cavitation | yes
      X3 order dependence specificity |                       motor_overload (R12 has 3 conditions) |    motor_overload | yes
      X3 order dependence    priority |                                  motor_overload (5 > 4) |    motor_overload | yes
      X3 order dependence     recency |                      motor_overload (R12 uses flow, newest) |    motor_overload | yes
      X3 order dependence       order |                             seal_leak (R7 precedes R11/R12) |         seal_leak | yes
            X4 dominance specificity |                     clogged_impeller (R10 has 3 conditions) |  clogged_impeller | yes
            X4 dominance    priority |                                  motor_overload (5 > 2) |    motor_overload | yes
            X4 dominance     recency | clogged impeller (R10/R9 use pressure/current, but R11 uses  |    motor_overload | NO
            X4 dominance       order |                          clogged_impeller (R9 precedes R11) |  clogged_impeller | yes
          X5 no evidence specificity |                                                no diagnosis |              None | yes
          X5 no evidence    priority |                                                no diagnosis |              None | yes
          X5 no evidence     recency |                                                no diagnosis |              None | yes
          X5 no evidence       order |                                                no diagnosis |              None | yes
   predictions confirmed: 19 of 20


   STRATEGY COMPARISON on the 15 hold-out cases and on the 5 adversarial cases (truth = my label; X5 truth = no diagnosis)
       strategy | hold-out acc  adversarial correct
    specificity |        0.867              4/5
       priority |        0.867              3/5
        recency |        0.867              2/5
          order |        0.867              3/5
   rule-order sensitivity: reversing the order of the rules in the knowledge base (same rules) changes the diagnosis in these hold-out + adversarial
cases:
       specificity: ['H14', 'X1 conflicting rules']
          priority: []
           recency: ['X4 dominance']
             order: ['H14', 'X1 conflicting rules', 'X3 order dependence', 'X4 dominance']


B2.3  KNOWLEDGE ACQUISITION AT RUN TIME (no engine code changed): regression over all 30 cases
   EDIT 1 (ADD rule R15: motor_temp = high and flow = normal -> motor_overload, to cover H12): accuracy on 30 cases 0.933 -> 0.967; diagnoses changed:
{'H12': (None, 'motor_overload')}
   EDIT 2 (EDIT R5: add 'axial_vib != high' so that axial vibration is not read as bearing wear, to cover H14): accuracy on 30 cases 0.933 -> 0.933;
diagnoses changed: {'H5': ('bearing_wear', None), 'H14': ('bearing_wear', 'misalignment')}
   EDIT 3 (careless EDIT of R6: the bearing-temperature condition is dropped, leaving 'vibration = high'): accuracy on 30 cases 0.933 -> 0.900;
diagnoses changed: {'H7': ('seal_leak', 'bearing_wear')}
   EDIT 4 (REMOVE rule R14): accuracy on 30 cases 0.933 -> 0.933; diagnoses changed: {}
```

B.4 B3.1 and B3.2 (t_b3_output.txt)

```
B3  EDGE INTRUSION DETECTION (case (a)); data = student-written flow simulator with the KDD/NSL-KDD feature schema (no public dataset could be
downloaded in this environment)
   stream of 60,000 time-ordered flows; split BY TIME: first 60% train, next 10% validation, last 30% test; normal traffic drifts and flash-crowd
bursts increase; the attack type slow_dos exists ONLY in the test period


B3.1  FULL MODEL (200-tree random forest) vs EDGE MODEL (stage-1 depth-3 tree + 15-tree depth-8 quantised forest), 3 stream seeds
   seed 0: flat-array engine equals scikit-learn probabilities (max abs diff 0.00e+00); stage-1 early-exit share 0.871
   seed 1: flat-array engine equals scikit-learn probabilities (max abs diff 0.00e+00); stage-1 early-exit share 0.900
   seed 2: flat-array engine equals scikit-learn probabilities (max abs diff 0.00e+00); stage-1 early-exit share 0.903
      model |  accuracy  precision  recall     F1  ROC-AUC   (mean ± sd over 3 seeds, TEST period)
       full | 0.981±0.001 0.997±0.000 0.940±0.002 0.968±0.001 0.999±0.000
       edge | 0.968±0.000 0.993±0.001 0.901±0.001 0.945±0.000 0.950±0.004


   per seed (accuracy / recall / F1): seed 0: full 0.980/0.938/0.966, edge 0.969/0.901/0.945; seed 1: full 0.981/0.940/0.968, edge 0.968/0.900/0.944;
seed 2: full 0.982/0.943/0.969, edge 0.969/0.903/0.945


B3.2  RESOURCE EVIDENCE (seed 0): model size, node visits per flow, single-flow latency measured here (pure-Python flat-array engine), 1 flow at a time
   full: 208438 nodes in 200 trees = 3664 KiB (18 bytes/node);   edge: stage 1 15 nodes + stage 2 3133 nodes in 15 trees = 21.5 KiB (7 bytes/node, 8-
bit thresholds)   -> size ratio 170x
    full: mean 1.284 ms, p95 2.244 ms, p99 2.437 ms, mean node visits 2666, sequential throughput 779 flows/s
    edge: mean 0.019 ms, p95 0.108 ms, p99 0.183 ms, mean node visits 18, sequential throughput 54003 flows/s
   stage-1 early exits in the timing sample: 0.885; the edge model needs 151.6x fewer node visits and is 69.3x faster on this machine
```

```
   latency budget 50 ms (p95): full meets it on this machine; it would still meet it on a device up to 22x slower than this container (p95 2.244 ms x
factor <= 50 ms)
   latency budget 50 ms (p95): edge meets it on this machine; it would still meet it on a device up to 462x slower than this container (p95 0.108 ms x
factor <= 50 ms)
   (reference: pickled scikit-learn forests: full 8.0 MiB)


B3.3  PREDICTIONS (written before the failure analysis) AND ERROR CATEGORIES on the TEST period, seed 0, threshold 0.5
   P1: both models detect the attack classes seen in training (dos, probe, r2l, u2r) with recall >= 0.9 except u2r (rare, 2 %) which is lower; P2:
recall on the UNSEEN attack slow_dos is below 0.3 for both models (its features look like normal traffic); P3: false-positive rate on flash-crowd
bursts (burst_normal) is above that on ordinary normal flows because bursts resemble dos in the count feature.
    traffic type      n | full: detected  edge: detected  (for the attack types 'detected' = recall; for normal types it is the false-alarm rate)
          normal  11313 |          0.001           0.002
    burst_normal   1307 |          0.000           0.010
             dos   2202 |          1.000           1.000
           probe   1406 |          1.000           0.999
             r2l   1063 |          0.914           0.848
             u2r    365 |          0.926           0.934
       slow_dos    344 |          0.366           0.000
   confusion matrix full [[TN, FP], [FN, TP]] = [[12604, 16], [336, 5044]];  edge = [[12587, 33], [531, 4849]]


   Threshold sweep for the edge model on the validation-free test period (cost of a missed attack vs a false alarm):
   threshold |  recall  precision  false-alarm rate
         0.1 |   0.904      0.969             0.013
         0.2 |   0.904      0.983             0.007
         0.3 |   0.903      0.989             0.004
         0.5 |   0.901      0.993             0.003
         0.7 |   0.887      0.995             0.002
         0.9 |   0.825      0.995             0.002


   Concept-drift check: edge model F1 on the test period split into thirds (the later the more drift and the unseen slow_dos):
      third 1: F1 0.940, recall 0.895, false-alarm rate 0.004
      third 2: F1 0.949, recall 0.907, false-alarm rate 0.002
      third 3: F1 0.946, recall 0.903, false-alarm rate 0.002
```

B.5 B3 mis-predictions (t_b3b_output.txt)

```
B3.2  MIS-PREDICTIONS (seed 0, test period, threshold 0.5): counts by traffic type and model
          type | full errors edge errors
        normal |          16          20
    burst_normal |           0          13
           dos |           0           0
         probe |           0           1
           r2l |          91         162
           u2r |          27          24
      slow_dos |         218         344


   Individual examples (feature values: count, serror_rate, duration, failed_logins, num_compromised, logged_in) and the category assigned
   [unseen attack type (slow_dos)] full model, FN, test row 245: count 10, serror 0.05, duration 12.5, failed_logins 0, compromised 0, logged_in 1;
P(attack) = 0.325
   [unseen attack type (slow_dos)] edge model, FN, test row 245: count 10, serror 0.05, duration 12.5, failed_logins 0, compromised 0, logged_in 1;
P(attack) = 0.015
   [subtle remote-to-local (few failed logins)] full model, FN, test row 322: count 5, serror 0.05, duration 5.8, failed_logins 1, compromised 0,
logged_in 0; P(attack) = 0.395
   [subtle remote-to-local (few failed logins)] edge model, FN, test row 146: count 6, serror 0.06, duration 2.2, failed_logins 1, compromised 0,
logged_in 0; P(attack) = 0.015
   [rare class (u2r with low compromise count)] full model, FN, test row 151: count 3, serror 0.10, duration 2.3, failed_logins 0, compromised 0,
logged_in 1; P(attack) = 0.190
   [rare class (u2r with low compromise count)] edge model, FN, test row 151: count 3, serror 0.10, duration 2.3, failed_logins 0, compromised 0,
logged_in 1; P(attack) = 0.015
   [flash crowd mistaken for an attack] edge model, FP, test row 911: count 74, serror 0.21, duration 1.0, failed_logins 0, compromised 0, logged_in 1;
P(attack) = 0.999
   [normal flow with unusual counts] full model, FP, test row 1012: count 10, serror 0.05, duration 2.5, failed_logins 3, compromised 0, logged_in 1;
P(attack) = 0.785
   [normal flow with unusual counts] edge model, FP, test row 275: count 8, serror 0.09, duration 7.1, failed_logins 0, compromised 1, logged_in 1;
P(attack) = 0.577


   FAILURE CLASS DEMONSTRATED: unseen attack slow_dos: full model detects 0.366 of 344, edge model 0.000; the same model detects 1.000 of the dos flows
seen in training. Adding 200 labelled slow_dos flows to the training data and retraining restores:
      after adding 200 slow_dos examples: full model detects 0.953, edge model 0.000; overall F1 full 0.986
```

B.6 B3.3 (mini-RDD) (t_b34_output.txt)

```
B3.4  DISTRIBUTED PROCESSING WITH A MINI-RDD ENGINE (PySpark is not installable in this sandbox, see the note in the report); machine cores: 2


(a) SCALING: records 1e5, 1e6, 1e7 x partitions {1, 2, 4, 8, 16}; workers = min(partitions, 2); wall-clock seconds for the A4 pipeline (clean +
reduceByKey + top-10), median of 3 runs
     records |       1       2       4       8      16 | speed-up of best vs 1 partition
      100000 |   0.001   0.010   0.014   0.012   0.014 | 1.00x at 1 partitions
     1000000 |   0.011   0.033   0.025   0.028   0.028 | 1.00x at 1 partitions
    10000000 |   0.221   0.201   0.142   0.116   0.097 | 2.29x at 16 partitions
   re-partitioning the SAME 600,000 records into 1, 3 and 8 partitions gives per-sensor std vectors equal to 4.5e-13 (max abs difference): results do
not depend on the partitioning


(b) SHUFFLE: map-side combine vs shipping every record (1e6 records, 8 partitions, 1000 sensors)
            reduceByKey (map-side combine): shuffles 1, shuffle volume 0.26 MB, time 0.032 s
    groupByKey-style (all records shuffled): shuffles 1, shuffle volume 15.68 MB, time 0.113 s


(c) CACHING: three actions (stats, top-10 on std, count of readings above 30) on the same intermediate RDD, 1e7 records, 8 partitions; a CHEAP
intermediate step (validity filter) and an EXPENSIVE one (filter + sort of every partition)
       cheap step, cache = False: total 0.278 s (cached partitions held in the driver: 0); readings above 30: 63787
       cheap step, cache =  True: total 0.584 s (cached partitions held in the driver: 8); readings above 30: 63787
   expensive step, cache = False: total 2.678 s (cached partitions held in the driver: 0); readings above 30: 63787
   expensive step, cache =  True: total 1.298 s (cached partitions held in the driver: 8); readings above 30: 63787


(d) SKEW: one hot sensor holds ~50% of the readings (1e7 records, 8 partitions)
   skew = False, combine : 0.113 s, shuffle 0.3 MB
   skew = False, no-combine: 1.029 s, shuffle 156.8 MB
   skew =  True, combine : 0.140 s, shuffle 0.3 MB
   skew =  True, no-combine: 1.024 s, shuffle 156.8 MB
```

```
(e) FAULT TOLERANCE BY LINEAGE: lose one cached partition and recompute only that partition (1e7 records, 8 partitions)
   after dropping partition 3 of 8: recomputed 1 partition(s) from the lineage; result identical: True; time 0.096 s
```

B.7 B4 (t_b4_output.txt)

```
B4  CHATBOT: 9 intents, 90 training utterances, 54 in-scope test utterances (different wording) and 20 out-of-scope test utterances; all written by the
student


B4.1  THRESHOLD 0.30, clean text: accuracy 0.757 (n = 74), macro-F1 0.782; out-of-scope falsely accepted 0.55; in-scope wrongly rejected 0.02
          intent | precision  recall     F1   n
     list_alerts |      0.67    0.67   0.67   6
     alert_count |      0.83    0.83   0.83   6
     why_flagged |      0.50    0.83   0.62   6
      top_source |      0.75    1.00   0.86   6
    model_status |      0.80    0.67   0.73   6
    latency_info |      0.60    1.00   0.75   6
     attack_info |      1.00    1.00   1.00   6
       ack_alert |      0.86    1.00   0.92   6
            help |      0.83    0.83   0.83   6
    out_of_scope |      0.90    0.45   0.60  20
   misclassified utterances:
      'pull up the outstanding alerts': true list_alerts, predicted model_status
      'anything flagged at the moment': true list_alerts, predicted why_flagged
      'give the alarm total': true alert_count, predicted why_flagged
      'how did you decide alert 3 was malicious': true why_flagged, predicted ack_alert
      'is the intrusion detector alive': true model_status, predicted list_alerts
      'is the classifier working': true model_status, predicted out_of_scope
      'give me the command list': true help, predicted list alerts
      'what is the weather today': true out_of_scope, predicted alert_count
      'tell me a joke': true out_of_scope, predicted why_flagged
      'who won the cricket match': true out_of_scope, predicted top_source
      'what is two plus two': true out_of_scope, predicted latency_info
      'what time is it in tokyo': true out_of_scope, predicted latency_info
      'how do i bake a cake': true out_of_scope, predicted help
      'set an alarm for seven': true out_of_scope, predicted why_flagged
      'what is the capital of france': true out_of_scope, predicted latency_info
      'tell me a story': true out_of_scope, predicted why_flagged
      'who is the prime minister': true out_of_scope, predicted top_source
      'what is the price of gold': true out_of_scope, predicted latency_info


B4.2  THRESHOLD SWEEP (clean text): the trade-off between accepting out-of-scope questions and rejecting real ones
   threshold | accuracy  OOS accepted  in-scope rejected
         0.0 |    0.635          1.00              0.00
         0.1 |    0.757          0.55              0.02
         0.2 |    0.757          0.55              0.02
         0.3 |    0.757          0.55              0.02
         0.4 |    0.703          0.55              0.11
         0.5 |    0.635          0.40              0.30
         0.6 |    0.595          0.05              0.52
         0.7 |    0.486          0.05              0.67


B4.3  SIMULATED SPEECH-RECOGNITION ERRORS (word error rate injected into every test utterance, 20 random seeds each, threshold 0.30)
     WER |   accuracy (mean ± sd)  OOS accepted  in-scope rejected
     0.0 |         0.757 ± 0.000          0.55              0.02
     0.1 |         0.662 ± 0.033          0.66              0.01
     0.2 |         0.578 ± 0.046          0.76              0.01
     0.3 |         0.505 ± 0.046          0.81              0.03
     0.4 |         0.459 ± 0.060          0.86              0.03


B4.3b  IMPROVEMENT: question words (what, who, how, tell, can, ...) carry no intent and were matching many intents; they are added to the stop list
(the change was motivated by the clean-text errors above, so it is checked on a SECOND, fresh set written afterwards: 18 in-scope + 12 out-of-scope
utterances)
                  version  thr | set 1 accuracy  OOS accepted  in-scope rejected | set 2 accuracy  OOS accepted  in-scope rejected
          v1 basic stop list  0.3 |          0.757          0.55              0.02 |          0.633          0.83              0.00
          v1 basic stop list  0.4 |          0.703          0.55              0.11 |          0.633          0.83              0.06
          v1 basic stop list  0.5 |          0.635          0.40              0.30 |          0.567          0.58              0.33
         v2 + question words  0.3 |          0.784          0.15              0.07 |          0.833          0.08              0.00
         v2 + question words  0.4 |          0.784          0.15              0.07 |          0.833          0.08              0.11
         v2 + question words  0.5 |          0.649          0.10              0.31 |          0.733          0.08              0.39


B4.4  LATENCY: mean time per utterance (classification + response)
   0.052 ms per utterance over 1080 utterances (pure Python, one core)


B4.5  EXAMPLE DIALOGUE (state: pending clarification)
   user: 'acknowledge alert'                          -> intent ack_alert (cosine 1.00)
   bot : Which alert number should I acknowledge?
   user: 'ack alert 5'                                -> intent ack_alert (cosine 0.59)
   bot : Alert 5 marked as resolved.
   user: 'why was it flagged'                         -> intent why_flagged (cosine 1.00)
   bot : Which alert should I explain? Say, for example, 'explain alert 12'.
   user: 'explain alert 12'                           -> intent why_flagged (cosine 1.00)
   bot : Alert 12: flagged because count and serror_rate were far above the normal range.
   user: 'what is the weather'                        -> intent out_of_scope (cosine 0.00)
   bot : Sorry, I can only help with intrusion-detection questions. Say 'help' to see what I can do.
   user: 'how many alerts today'                      -> intent alert_count (cosine 1.00)
   bot : 17 alerts today.
```

Appendix C – The Same Pipeline in PySpark (not executed here)

This listing is the PySpark equivalent of the pipeline of A4 and B3.3. It was **not run** in the sandbox because PySpark could not be installed; run it on Google Colab (`pip install pyspark`) before the viva and replace Figure 3 with the numbers it prints.

```
"""PySpark version of the A4 / B3.3 pipeline.  NOT EXECUTED in the sandbox (PySpark could not be installed there); run it on Colab:  pip
install pyspark
Reads CSV lines 'sensorId,timestamp,value', computes per-sensor mean / max / std and the 10 noisiest sensors."""
import sys, math, time
from pyspark.sql import SparkSession


def parse(line):                                          # narrow transformation helper
    try:
        sid, ts, v = line.split(","); v = float(v)
        return [(sid, (v, 1, v, v * v))] if 0.0 <= v <= 500.0 else []
    except ValueError:
        return []


def main(path, partitions):
    spark = SparkSession.builder.master("local[*]").appName("sensor-stats").getOrCreate()
    sc = spark.sparkContext
    raw = sc.textFile(path, minPartitions=partitions)                        # transformation (lazy)
    pairs = raw.flatMap(parse)                                               # narrow: parse + validity filter + key
    # pairs.cache()      # enable ONLY if two or more actions reuse 'pairs' and the parse step is expensive (see B3.3 (c))
    stats = pairs.reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1], max(a[2], b[2]), a[3] + b[3]))   # wide: ONE shuffle, map-side
combine
    per_sensor = stats.mapValues(lambda s: (s[0] / s[1], s[2], math.sqrt(max(s[3] / s[1] - (s[0] / s[1]) ** 2, 0.0))))   # (mean, max, std)
    t0 = time.time()
    result = per_sensor.collect()                                            # ACTION 1: the job runs here
    top10 = per_sensor.takeOrdered(10, key=lambda kv: -kv[1][2])            # ACTION 2: ten noisiest sensors (largest std)
    print("sensors:", len(result), "seconds:", round(time.time() - t0, 3))
    print("shuffle stages: see the Spark UI (stage 2 'Shuffle Read')")
    print(raw.toDebugString().decode())                                      # lineage
    print(top10)
    spark.stop()


if __name__ == "__main__":
    main(sys.argv[1], int(sys.argv[2]) if len(sys.argv) > 2 else 8)
```